

Fondamenti d'Intelligenza Artificiale-Metodi

Luca Tortoriello

January 2026

Indice

1	Machine learning and deep learning	7
1.1	Programmazione tradizionale vs Machine Learning	7
1.2	Processo di Machine Learning	7
1.3	Tipi di Machine Learning	8
1.4	Machine learning supervisionato	9
1.4.1	Classificazione	10
1.5	Feature scaling	10
1.5.1	Dummy variables	11
1.6	KNN	11
1.6.1	Scelta del valore di K e considerazioni sul metodo	14
1.7	Training Set, Test Set e Validazione	15
1.8	Valutazione dei modelli	18
1.8.1	ROC	20
1.8.2	Media delle curve ROC	21
2	Apprendimento supervisionato	22
2.1	Teoria della decisione e probabilità	22
2.1.1	Probabilità a priori	22
2.2	Densità di probabilità condizionata a una classe	23
2.2.1	Rischio condizionato	24
2.3	Classificatore Bayesiano	29
2.4	Alberi di decisione	32
2.4.1	CART (Classification and Regression Tree)	33
2.4.2	Impurità di Gini	34
2.4.3	Entropia	35
2.4.4	Varianza	35
2.4.5	Misclassificazione	36
2.4.6	Complessità	36
2.4.7	C4.5	37
2.4.8	Ensemble Learning	38
2.4.9	Albero per regressione	38
3	Dimensionality Reduction (Feature Selection)	38
3.1	Tassonomie	38
3.1.1	Supervised Feature Selection	39
3.1.2	Feature Ranking: Pair Wise (<i>a coppie</i>)	39
3.1.3	Feature ranking: simultaneous	41
3.2	Subset Selection: Configuration	43
3.3	Subset Selection: Evaluation	44
3.3.1	Metodi model-specifier (wrapper)	44
3.3.2	Metodi model-independent	44
3.3.3	Subset Evaluation Complete: Branch and bound	45
3.3.4	Subset Evaluation Heuristic: Best First	46
3.3.5	Subset Evaluation Heuristic: SFS e SBS	47

3.3.6	Bidirectional Elimination (Stepwise Bidirectional Selection)	49
3.4	Embedded	50
3.4.1	Embedded: Regularization-Regression Lasso	50
4	Hyperparameter Tuning	51
4.1	Grid Search	52
4.2	Random Search	52
5	Data Cleaning	53
5.1	Gestione dei valori mancanti	54
5.2	Tecniche di imputazione	54
5.2.1	Imputazione tramite k-Nearest Neighbours	56
5.3	Cleaning di valori non corretti	56
5.4	Scaling e Normalizzazione	57
5.4.1	Standardizzazione (Z-score)	57
5.4.2	Min-Max Scaling	58
6	Data Reduction and Transformation	58
6.1	Data Sampling per Dati Statici	59
6.1.1	Unbiased Sampling (Campionamento non polarizzato)	59
6.1.2	Biased Sampling (Campionamento polarizzato)	60
6.2	Data Sampling per Data Streams	60
6.2.1	Algoritmo di Reservoir Sampling (<i>serbatoio</i>)	60
6.3	Data reduction with type transformation	61
6.3.1	Haar Wavelet Transform	61
7	Reti Neurali	64
7.1	Funzioni di attivazione	66
7.2	Processi di apprendimento	68
7.2.1	Error-correcting learning	69
7.3	La soluzione dell'AND logico	70
7.4	Teorema di convergenza del perceptrone	71
7.5	Error Correcting Learning e LMS	76
7.5.1	Paradigma di apprendimento della LMS	79
7.5.2	Curva di apprendimento	81
7.5.3	Gradiente discendente stocastico	83
7.6	Tecniche per mitigare l'overfitting	84
7.6.1	Semplificazione del modello	84
7.6.2	Early stopping	84
7.6.3	Data Augmentation	85
7.6.4	Regolarizzazione	85
7.6.5	Dropout	86
7.6.6	Inizializzazione della rete	86
7.6.7	Batch Normalization	87
7.7	Multi-Layer Perceptrons (MLP)	87
7.7.1	Il problema dello XOR	89

7.8	Error backpropagation: le equazioni	92
7.8.1	1. Errore locale nello strato di uscita ($l = L$)	95
7.8.2	2. Errore locale nello strato interno l	96
7.8.3	3. Variazione della Loss rispetto a θ di apprendimento	98
7.8.4	4. Variazione della Loss rispetto ai pesi w_{jk}	98
7.9	Due funzioni di perdita: Cross entropy e Softmax	99
7.9.1	Cross-Entropy	99
7.9.2	Softmax	100
8	Autoencoder	100
8.1	Overcomplete hidden layers	102
8.2	Sparse Autoencoder	103
8.3	Denoising Autoencoders	104
8.4	Stacked Autoencoders	104
9	Support Vector Machine (SVM)	105
10	Clustering	106
10.1	Nomenclatura dei metodi di clustering	107
10.2	Clustering gerarchico (hierarchical)	110
10.2.1	Clustering gerarchico agglomerativo	110
10.3	K-Means Clustering	116
10.3.1	Elbow method	118
10.4	K-Means ++	119
10.5	DBSCAN	121
10.6	Indici per valutare i modelli di clustering	124
10.6.1	Silhouette index (interno)	125
10.6.2	Adjusted Rand Index (esterno)	126
11	Regressione	126
11.1	Regressione multipla	127
11.2	Potenziati problemi di regressione	128
11.2.1	Problema della relazione non lineare	128
11.2.2	Errore non riducibile a varianza non costante	129
11.2.3	Outlier	129
11.2.4	High-leverage points	131
11.2.5	Collinearità	132
11.3	Regressione isotonica	132
11.4	Regressione logistica	132
12	Metodi di decomposizione	133
12.1	Matrice di decomposizione	134
12.2	One-per-class (OpC) (One-Against-All)	135
12.3	PairWise Coupling (PWC) (One vs One)	135
12.3.1	Distributed Output Code (ECOC)	136

13 Time Series Mining	138
13.1 Definizioni	139
13.2 Scale diverse dei campioni	140
13.3 Missing Values	140
13.4 Rimozione del rumore	140
13.4.1 Binning	141
13.4.2 Smoothing	141
13.5 Rappresentazione delle time series	143
13.6 Symbolic Aggregate Approximation	144
13.7 Forecasting	145
13.7.1 Modello autoregressivo	145
13.8 Classificazione	146
13.8.1 Dynamic Time Warping	147
13.9 Reti neurali per Time Series	149
14 Reti neurali ricorrenti (RNN)	150
14.1 Implementazione Vanilla	152
14.2 Gradiente evanescente nelle RNN	154
15 Ensemble Learning	154
15.1 Matrice profilo di decisione	155
15.2 Metriche per misurare la diversità	156
15.2.1 Q-statistic	156
15.2.2 Coefficiente di correlazione	157
15.2.3 Disaccordo	157
15.2.4 Double fault	158
15.3 Random Forest	158
15.4 Bagging (Bootstrap Aggregating)	159
15.5 AdaBoost	159
16 Reinforcement Learning	161
16.1 Processo decisionale sequenziale	163
16.2 Descrizione del modello dell'ambiente	164
16.3 Proprietà di Markov	166
16.4 Processi di Markov	166
16.5 Processo decisionale di Markov	168
16.5.1 Tipi di processi decisionali di Markov	168
16.5.2 Esempi di Processi Decisionali di Markov	169
16.6 Agente	170
16.6.1 Agente: Policy	170
16.6.2 Agente: Funzione Valore	170
16.6.3 Agente: Modello	171
16.7 Dilemma dell'esplorazione e dello sfruttamento	171
16.7.1 Strategie di Sfruttamento (Exploitation)	171
16.8 Tassonomie degli agenti	172
16.9 Algoritmi Value-Based	174

16.9.1 Value Iteration	174
16.10 Algoritmi Policy-Based	177
16.10.1 Policy Iteration	177
16.11 Temporal Difference Learning	181
16.11.1 Q-Learning	183
16.12 Paradigmi di Apprendimento: Off-policy vs. On-policy	188
16.12.1 Paradigma On-Policy: Approccio SARSA	188

1 Machine learning and deep learning

Il deep learning si basa su modelli di **reti neurali**, strutture computazionali ispirate al funzionamento del cervello umano. Una delle caratteristiche fondamentali è la capacità di operare direttamente sul **dato grezzo**, senza richiedere una preventiva fase manuale di estrazione delle caratteristiche. Nel **machine learning** i modelli lavorano tipicamente su dati in forma tabellare: il dato grezzo è trasformato, tramite una fase di **feature extraction**, in un insieme di attributi significativi organizzati in una tabella di caratteristiche.

Nel **deep learning**, la fase di estrazione delle caratteristiche è incorporata all'interno del modello stesso: la rete neurale apprende autonomamente le rappresentazioni rilevanti dei dati, costruendo livelli progressivamente più astratti e informativi.

1.1 Programmazione tradizionale vs Machine Learning

La programmazione tradizionale si basa sulla **definizione esplicita di regole**. Data una specifica, il programmatore analizza il problema, individua le condizioni rilevanti e costruisce un insieme di istruzioni che il computer deve seguire. Il processo è iterativo: si scrivono le regole, si valuta il comportamento del programma e, in caso di errori, si modificano o si aggiungono ulteriori regole.

Le tecniche di Machine Learning spostano l'intelligenza dal programmatore al modello. Invece di definire manualmente le regole, si fornisce al sistema un insieme di dati rappresentativi del problema: il modello apprende automaticamente le relazioni rilevanti e costruisce internamente le proprie regole implicite. Il processo è iterativo, ma l'intervento umano non consiste più nello scrivere nuove condizioni: si agisce sui dati, sull'architettura o sul metodo di addestramento. Il risultato è un programma generalmente più corto, più facile da mantenere e capace di gestire problemi complessi e variabili, per i quali sarebbe irrealistico definire manualmente tutte le regole operative.

1.2 Processo di Machine Learning

La tipica pipeline di Machine Learning può essere suddivisa in tre fasi:

1. **Data pre-processing**: l'analisi preliminare dei dati e le operazioni di pulizia, trasformazione e organizzazione del dato grezzo
2. **Modellazione**: selezione dell'algoritmo, addestramento del modello tramite stima dei parametri e nella generazione delle predizioni, successivamente utilizzate nelle fasi di validazione
3. **Valutazione**: analisi delle prestazioni del modello mediante opportuni indicatori, verificando la capacità di generalizzare su dati non noti

Fornire al modello dati non correttamente preparati compromette l'intero processo, poiché esso non dispone dei mezzi per compensare errori o anomalie presenti nel dato. La fase di pre-processing riveste un ruolo centrale e spesso risulta la più determinante dell'intera pipeline.

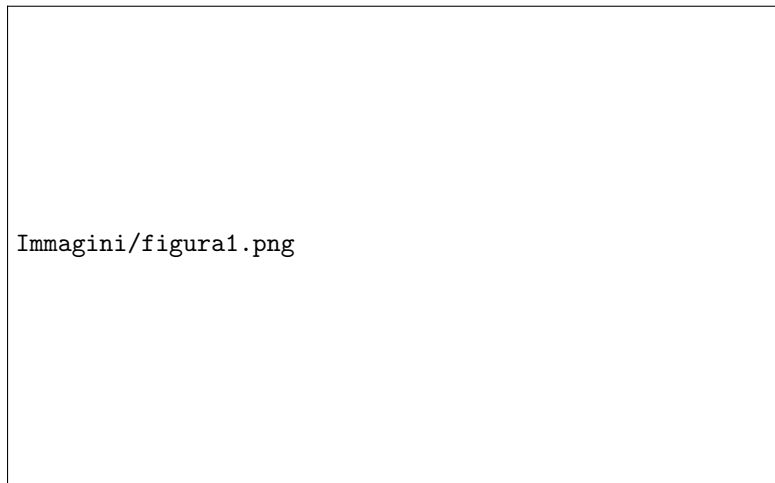


Figura 1.1: Distribuzione del tempo nelle principali attività di un progetto di Machine Learning e fasi del ciclo di vita di sviluppo, dal data management al deployment

1.3 Tipi di Machine Learning

In base al paradigma di apprendimento, i modelli di Machine Learning possono essere distinti in tre categorie principali:

- **Supervisionati:** nella fase di addestramento il modello utilizza un dataset contenente sia gli **input** sia gli **output corretti**. L'obiettivo è **apprendere la relazione** che lega le variabili indipendenti all'output, così da effettuare previsioni su nuovi dati. I principali task sono:
 - **Classificazione:** l'output appartiene a una classe discreta; non esiste un ordinamento naturale tra le categorie. Un insieme di **etichette**
 - **Regressione:** l'output è un valore continuo o ordinabile
- **Non supervisionati:** il modello non dispone di un output etichettato e deve individuare autonomamente strutture, pattern o relazioni nei dati. Le attività principali sono:
 - **Clustering:** raggruppamento dei campioni in insiemi omogenei; i gruppi non hanno un significato semantico assegnato a priori

- **Dimensionality reduction:** riduzione del numero di feature mantenendo le informazioni essenziali. Consente di ottenere una rappresentazione più compatta e interpretabile del dato, migliorando efficienza e qualità del modello
- **Reinforcement Learning:** un agente interagisce con un ambiente prendendo decisioni sequenziali con l'obiettivo di massimizzare una ricompensa cumulata. L'agente impara attraverso tentativi ed errori, valutando l'effetto delle proprie azioni in funzione dell'obiettivo da raggiungere



Figura 1.2: Panoramica dei principali paradigmi del Machine Learning (supervised, unsupervised e reinforcement learning) e delle loro applicazioni tipiche

1.4 Machine learning supervisionato

Si consideri un insieme di training set composto da n campioni. Per ciascun campione si osserva un vettore di caratteristiche (features), indicato come

$$\underline{x}^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_d^{(i)}),$$

dove d rappresenta il numero totale di caratteristiche. In particolare, $\underline{x}^{(i)}$ è il vettore delle feature del i -esimo campione. Per ogni campione è inoltre associato un output (o etichetta):

$$y^{(i)}, \quad i = 1, \dots, n.$$

Si definisce poi una funzione di apprendimento $f(\underline{x}, \underline{w})$, dove:

- $\underline{w}$ è il vettore dei pesi da ottimizzare
- $f(\underline{x}, \underline{w})$ restituisce l'output corretto per il campione di input $\underline{x}$

Per determinare il valore ottimale dei pesi si utilizzano i campioni etichettati $\{(\underline{x}^{(i)}, y^{(i)})\}_{i=1}^n$, in modo tale che $f(\underline{x}, \underline{w})$ fornisca l'etichetta corretta per ciascun esempio. Questo processo costituisce un problema di **ottimizzazione**: occorre scegliere la forma funzionale di $f(\underline{x}, \underline{w})$ e determinare i pesi che minimizzano l'errore di predizione.

L'insieme dei campioni etichettati $\{\underline{x}^{(1)}, \underline{x}^{(2)}, \dots, \underline{x}^{(n)}\}$ è suddiviso in due sottoinsiemi distinti:

- **Training set**, utilizzato per insegnare al modello, per stimare i pesi ottimali $\underline{w}$
- **Test set**, utilizzato per valutare le prestazioni del modello su esempi mai visti in precedenza

La fase di testing consente di stimare la capacità del modello e fornisce un'indicazione realistica delle prestazioni attese in applicazioni pratiche.

1.4.1 Classificazione

La classificazione è una tecnica di ML che ha l'obiettivo di identificare la **categoria** a cui appartiene una nuova osservazione, sulla base dei dati utilizzati durante la fase di addestramento. A **differenza della regressione**, che predice un valore numerico continuo, la classificazione associa **ciascun campione** a una classe **discreta**.

1.5 Feature scaling

Si vuole che le caratteristiche fornite in input abbiano lo stesso ordine di grandezza, in modo tale che ogni feature contribuisca in maniera equilibrata al modello. A questo scopo si ricorre alla standardizzazione dei dati, con l'obiettivo di riportare tutte le caratteristiche all'interno di uno stesso intervallo di valori. Esistono due modalità principali di feature scaling:

1. **Normalizzazione**: si definisce una nuova variabile X' come

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \quad (1.1)$$

Tale trasformazione riporta i valori nel range:

$$[0; 1]$$

particolarmente utile quando il **range** delle variabili è **noto a priori**.

2. **Standardizzazione**: si sottrae al valore originale la media μ e si divide il risultato per la deviazione standard σ :

$$X' = \frac{X - \mu}{\sigma} \quad (1.2)$$

la procedura risulta generalmente **più robusta** alla presenza di **outlier**.

1.5.1 Dummy variables

Quando un dataset contiene **variabili categoriali**, tali informazioni non possono essere utilizzate direttamente nei modelli di regressione o nelle tecniche di Machine Learning, poiché questi richiedono **input numerici**. Per incorporare correttamente le categorie si utilizza il metodo delle dummy variables, che consiste nel trasformare ogni categoria in una o più variabili binarie.

Ad esempio, se la variabile **State** può assumere i valori **New York** o **California**, si creano due nuove variabili:

- **New York**: vale 1 se il campione appartiene a questa categoria, altrimenti 0
- **California**: vale 1 se il campione appartiene a questa categoria, altrimenti 0

In questo modo il modello può utilizzare tali informazioni all'interno della sua struttura funzionale. Una regressione lineare che include variabili dummy assume la forma:

$$y = b_0 + b_1X_1 + b_2X_2 + b_3X_3 + \dots + b_kD_k \quad (1.3)$$

dove i termini D_k sono le variabili dummy associate alle diverse categorie.

Quando si utilizzano variabili dummy è fondamentale evitare la **dummy variable trap**, un problema di **multicollinearità** perfetta che si verifica quando tutte le categorie di una variabile categoriale vengono incluse contemporaneamente nel modello.

Per esempio, considerando due categorie (**New York** e **California**), costruendo due variabili dummy D_1 e D_2 , esse risultano linearmente dipendenti:

$$D_2 = 1 - D_1$$

Una colonna può essere espressa come combinazione lineare dell'altra, generando multicollinearità e rendendo impossibile stimare correttamente i coefficienti del modello di regressione. Per questo motivo, nel modello si includono solamente $k - 1$ variabili dummy per rappresentare k categorie. Nel caso di due categorie, si utilizza una sola dummy. Il modello assume pertanto la forma:

$$y = b_0 + b_1X_1 + b_2X_2 + b_3X_3 + b_4D_1 \quad ,$$

evitando così la dipendenza lineare.

1.6 KNN

Il metodo K-Nearest Neighbors (K-NN) è una tecnica di **classificazione** basata sulla vicinanza geometrica tra i punti nello spazio delle feature. L'algoritmo non apprende un modello parametrico, ma effettua la previsione confrontando

direttamente il nuovo dato con i campioni presenti nel dataset. Un nuovo punto viene assegnato alla categoria che risulta prevalente tra i suoi K vicini più prossimi.

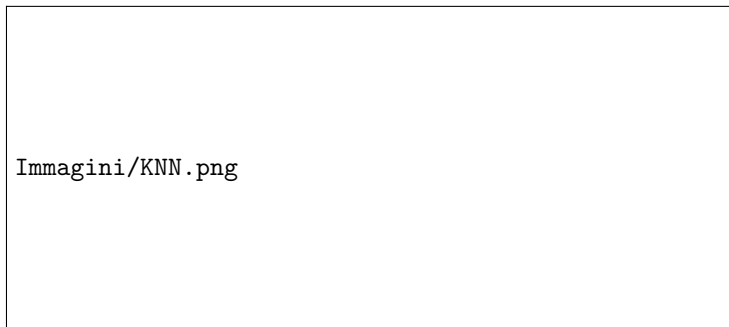


Figura 1.3: Esempio del funzionamento del metodo K-NN: un nuovo punto viene classificato in base ai punti più vicini nello spazio delle feature.

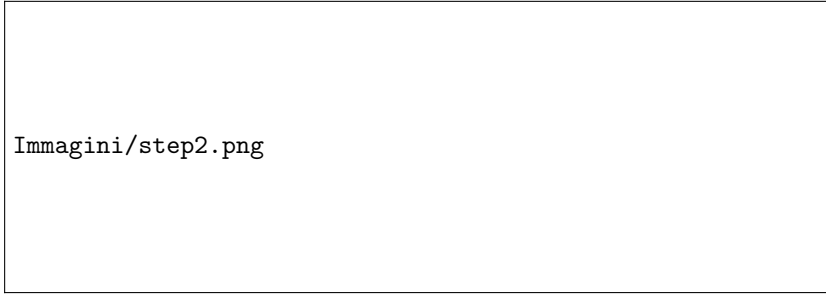
K è l'unico **iperparametro** del modello, un parametro da fissare a priori prima dell'addestramento.

1. **Scelta del numero K di vicini.** Si seleziona il valore di K , che indica quanti punti del training set verranno utilizzati per determinare la classe del nuovo campione.



Figura 1.4: Step 1: scelta del numero K di vicini da considerare

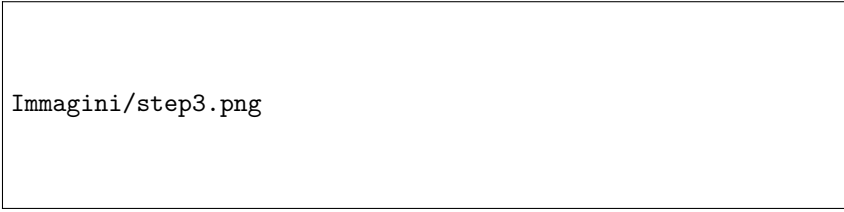
2. **Calcolo delle distanze e selezione dei K vicini più prossimi.** Si calcola la distanza da tutti i campioni presenti nel dataset. Le distanze ottenute vengono ordinate e si selezionano i K punti più vicini.



Immagini/step2.png

Figura 1.5: Step 2: calcolo della distanza euclidea e ordinamento dei campioni in base alla distanza

3. **Conteggio delle categorie nei K vicini selezionati.** Tra i K vicini più prossimi si contano le occorrenze delle diverse classi presenti. La classe con il maggior numero di rappresentanti risulta quella candidata per l'assegnazione del nuovo punto.



Immagini/step3.png

Figura 1.6: Step 3: individuazione della classe prevalente all'interno dei K vicini

4. **Assegnazione della classe (o del valore, nel caso di regressione).** In fase di classificazione il nuovo punto viene assegnato alla classe maggiormente rappresentata tra i K vicini. Nel caso della regressione K-NN, la previsione del valore viene ottenuta mediante la media dei valori associati ai K vicini selezionati.



Immagini/step4.png

Figura 1.7: Step 4: assegnazione della classe dominante oppure calcolo della media nel caso di regressione

Un altro **iperparametro** del KNN è la **scelta della metrica della distanza**. In base alla tipologia di distanza cambia il vicinato.

- **Distanza di Manhattan** Misura la distanza come somma dei valori assoluti delle differenze tra le coordinate:

$$d(\mathbf{x}, \mathbf{y}) = |x_1 - y_1| + |x_2 - y_2| \quad (1.4)$$

Corrisponde al percorso minimo lungo una griglia. È adatta quando le feature rappresentano grandezze indipendenti o quando lo spazio ha vincoli simili a una griglia.

- **Distanza Euclidea** È la distanza in linea retta tra due punti:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2} \quad (1.5)$$

Funziona bene quando le feature sono scalate correttamente. Sensibile a outlier e differenze di scala.

- **Distanza di Chebyshev** Definisce la distanza come il massimo tra le differenze assolute:

$$d(\mathbf{x}, \mathbf{y}) = \max(|x_1 - y_1|, |x_2 - y_2|) \quad (1.6)$$

Permette di muoversi in tutte le direzioni con costo uniforme. È utile quando si vuole che un'unica grande differenza domini la misura di vicinanza.

- **Distanza di Minkowski** È una generalizzazione delle tre precedenti:

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}, \quad p \geq 1 \quad (1.7)$$

- $p = 1$: Manhattan
- $p = 2$: Euclidea
- $p \rightarrow \infty$: Chebyshev

Permette di modellare in modo flessibile la sensibilità alle differenze tra le coordinate.

1.6.1 Scelta del valore di K e considerazioni sul metodo

La scelta del parametro K è cruciale per le prestazioni dell'algoritmo K-NN. In pratica, si valutano diversi valori di K e si seleziona quello che minimizza l'errore sul *test set* (o in validazione). Un valore troppo piccolo, ad esempio $K = 1$, rende il modello molto sensibile al rumore e agli outlier, producendo confini decisionali irregolari. Al contrario, un valore troppo grande tende a sovrasemplificare la struttura dei dati, generando confini eccessivamente lisci e riducendo la capacità discriminativa.

Una tecnica ampiamente utilizzata per individuare il valore ottimale è la **k-fold cross-validation**, che permette di bilanciare accuratezza e capacità di generalizzazione.

Il K-NN **non richiede una vera fase di training**, poiché le decisioni vengono prese direttamente a partire dai dati memorizzati. Tuttavia, l'algoritmo presenta alcune limitazioni: il costo computazionale in fase di predizione cresce con la dimensione del training set; è sensibile al rumore, ai valori mancanti e agli outlier; infine, richiede un'adeguata scalatura delle feature per evitare che alcune variabili dominino il calcolo delle distanze. La scelta di K rimane quindi un passaggio determinante per ottenere buone prestazioni.

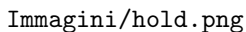
1.7 Training Set, Test Set e Validazione

Quando si sviluppa un modello di Machine Learning, il dataset iniziale viene suddiviso in due sottoinsiemi distinti: il **training set** e il **test set**. Il training set, che tipicamente rappresenta circa l'80% dei dati disponibili, è utilizzato per costruire il modello. Il restante 20% costituisce il **test set**, che ha il ruolo di valutare la capacità del modello di generalizzare su dati mai visti prima.

L'idea è verificare che il modello non apprenda solo la struttura specifica dei dati di training, ma sia in grado di fare previsioni corrette su nuovi dati. Un modello che performa bene sul training set ma male sul test set soffre di **overfitting**, mentre un modello con prestazioni scarse su entrambi i set potrebbe essere troppo semplice e quindi soggetto a **underfitting**.

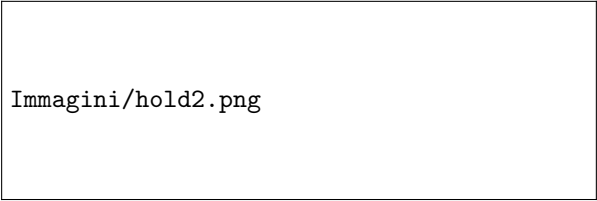
La **validazione** fornisce un insieme di strumenti per evitare tali errori di valutazione, permettendo di stimare in maniera più affidabile la reale capacità predittiva del modello su dati nuovi e non osservati. Si possono valutare diversi approcci per la scelta ottimale di training set e test set a partire dai dati:

- **Holdout:** il dataset è suddiviso in modo casuale in due sottoinsiemi distinti. Le partizioni comuni prevedono divisioni 70/30 o 80/20. Tale metodo è semplice e veloce, ma la qualità della valutazione dipende fortemente da come viene effettuata la suddivisione casuale. **Si utilizza l'holdout con un dataset molto grande.**



Immagini/hold.png

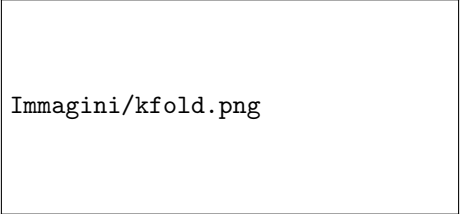
- **Random Subsampling:** si definisce un iperparametro n e si eseguono n esperimenti holdout. Ogni esperimento utilizza una diversa suddivisione casuale del dataset in train set e test set, una stima più robusta delle prestazioni medie del modello. Si ottengono n valori di errore, che possono essere mediati per ottenere un indicatore più stabile.



Immagini/hold2.png

Per $n = 1$ si recupera lo schema dell'holdout. Effettuare più suddivisioni casuali consente di avere una **visione complessiva** del comportamento del modello su dati diversi

- **K-Fold Cross Validation:** per ridurre la variabilità dovuta alle suddivisioni casuali, il dataset viene diviso in k blocchi di uguale dimensione. Si eseguono quindi k esperimenti: a rotazione, un blocco viene utilizzato come test set mentre gli altri $k-1$ blocchi formano il train set. Ogni campione svolge esattamente una volta il ruolo di test, evitando sovrapposizioni tra i set di valutazione.

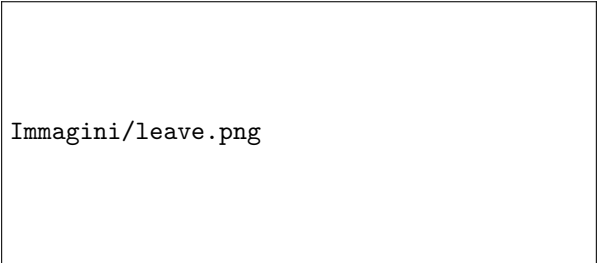


Immagini/kfold.png

- **Leave-One-Out Cross Validation:** caso particolare del k-fold in cui

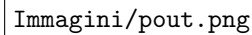
$$k = \text{\#campioni}$$

Ogni esperimento prevede di addestrare il modello su tutti i campioni tranne uno, che viene utilizzato per il test. Si ripete l'operazione per ogni campione. Approccio utile quando il **dataset è ridotto**, poiché permette di sfruttare tutti i dati per l'addestramento e quando il campione rappresenta un'entità importante. E' computazionalmente molto pesante.



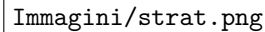
Immagini/leave.png

- **Leave-p-Out Cross Validation:** si sceglie un valore p e, per ogni possibile combinazione di p campioni, questi ultimi vengono esclusi e utilizzati come test set. Tutti gli altri campioni costituiscono il train set. Il metodo è molto potente, ma cresce con il numero di campioni e può diventare computazionalmente pesante



Immagini/pout.png

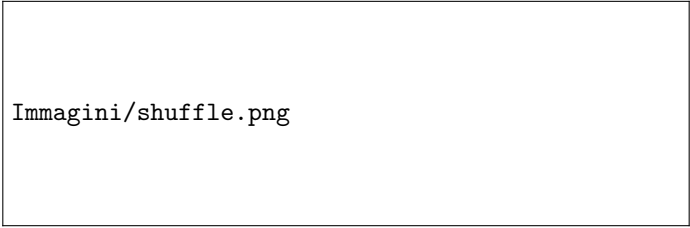
- **Stratified Cross Validation:** in presenza di dataset sbilanciati, una suddivisione casuale può produrre partizioni non rappresentative della distribuzione originale delle classi. La stratified cross validation garantisce che ciascuna fold mantenga approssimativamente la stessa proporzione di classi del dataset originale, evitando perdite di informazione e valutazioni distorte.



Immagini/strat.png

L'idea è di effettuare una suddivisione k-fold indipendente per ciascuna classe, combinando poi i risultati in modo coerente.

- **Stratified Shuffle Split:** rappresenta una variante stratificata del *random subsampling*. Ogni suddivisione conserva le proporzioni originali delle classi (stratified), ma i campioni vengono mescolati casualmente a ogni iterazione (shuffle). Permette di generare molteplici split indipendenti mantenendo comunque la rappresentatività delle classi.



Immagini/shuffle.png

- **Bootstrap**: si estrae dal dataset un campione di dimensione pari al dataset stesso utilizzando un processo di campionamento con reinserimento (sampling with replacement). I campioni non selezionati costituiscono automaticamente il test set (out-of-bag samples). L'operazione viene ripetuta per k iterazioni, ottenendo differenti partizioni train-test. Il bootstrap consente di stimare la variabilità delle prestazioni ed è **utile** soprattutto quando il **dataset è limitato**.



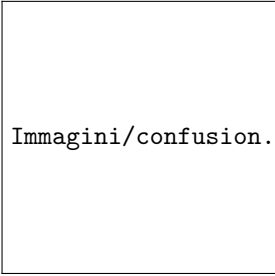
Immagini/boot.png

1.8 Valutazione dei modelli

Nella valutazione dei modelli di classificazione si distinguono quattro possibili esiti delle predizioni del classificatore. Quando la predizione è corretta si parla di **true positive** o **true negative**, mentre gli errori vengono suddivisi in **falsi positivi** e **falsi negativi**. In particolare:

1. **Falsi positivi (FP)**: il modello predice positivamente un campione che appartiene in realtà alla classe negativa
2. **Falsi negativi (FN)**: il modello predice negativamente un campione che appartiene in realtà alla classe positiva

L'insieme di queste quattro quantità permette di costruire la **matrice di confusione** (*confusion matrix*):



Immagini/confusion.png

Supponiamo di disporre di un insieme di 100 campioni classificati dal modello:

Immagini/ese.png

A partire dalla matrice di confusione è possibile calcolare due indicatori di base: l'**accuratezza** e il suo complementare, l'**error rate**:

$$AR = \frac{\text{Correct}}{\text{Total}} = \frac{TN+TP}{\text{Total}} = \frac{84}{100} = 84\% \quad (1.8)$$

$$ER = \frac{\text{Incorrect}}{\text{Total}} = \frac{FN+FP}{\text{Total}} = \frac{16}{100} = 16\% \quad (1.9)$$

Tuttavia, l'accuratezza non sempre rappresenta una misura affidabile, soprattutto in casi di dataset sbilanciati, quando la percentuale di campioni appartenenti ad una classe è molto maggiore rispetto alla percentuale di dati appartenenti ad un'altra classe. Per una valutazione più significativa è preferibile analizzare metriche che separano il comportamento del modello sulle classi positiva e negativa. Le principali sono:

$$TPR = \frac{TP}{TP+FN} \quad (\text{sensitivity o true positive rate}) \quad (1.10)$$

$$TNR = \frac{TN}{TN+FP} \quad (\text{specificity o true negative rate}) \quad (1.11)$$

Per sintetizzare in un'unica metrica la capacità del modello di riconoscere correttamente sia i positivi sia i negativi, si può utilizzare la **geometric mean**:

$$g = \sqrt{TPR \cdot TNR} \quad (1.12)$$

Altre metriche di largo uso sono la **precisione** e la **recall**. La prima misura quanto spesso le predizioni positive del modello siano corrette; la seconda misura quanto spesso il modello riesca a individuare correttamente i campioni realmente positivi:

$$\text{Precision} = \frac{TP}{TP+FP} \quad (1.13)$$

$$\text{Recall} = \frac{TP}{TP+FN} \quad (1.14)$$

Per combinare precision e recall in un'unica quantità si utilizza lo **F-score**, e in particolare l' F_1 -**score**, che rappresenta la loro media armonica:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (1.15)$$

1.8.1 ROC

La curva **ROC** (Receiver Operating Characteristic) rappresenta graficamente il comportamento del modello al variare della soglia di decisione utilizzata per trasformare la probabilità stimata in un'etichetta di classe. Mette in relazione il:

- il **True Positive Rate (TPR)**, anche detto *sensitivity*:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}},$$

che corrisponde all'asse verticale del grafico

- il **False Positive Rate (FPR)**:

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = 1 - \text{specificity},$$

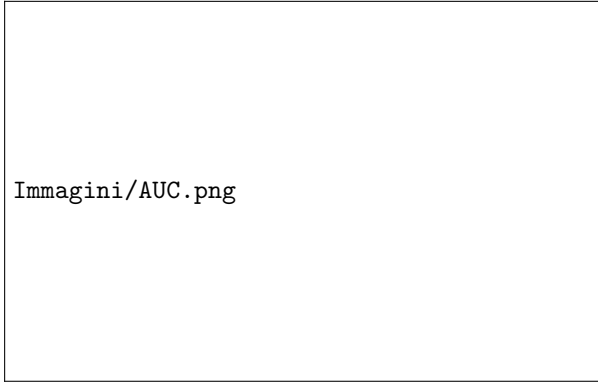
rappresentato sull'asse orizzontale.

Al variare della soglia di classificazione, il modello produce diverse coppie (FPR, TPR) che, collegate tra loro, formano la curva ROC. Questo approccio consente di osservare come cambiano le prestazioni del classificatore in funzione della soglia scelta:



L'informazione fornita dalla curva ROC viene sintetizzata tramite l'**AUC** (Area Under the Curve), l'area sottesa alla curva. L'AUC misura la capacità del classificatore di separare le due classi:

- un valore vicino a 1 indica un'ottima capacità discriminante
- un valore pari a 0.5 corrisponde al comportamento di un classificatore casuale
- valori inferiori a 0.5 indicano un modello peggiore del caso.



Immagini/AUC.png

La curva ROC è inoltre utile per confrontare modelli differenti: a parità di condizioni, il modello con curva ROC più alta e con AUC maggiore è da considerarsi complessivamente migliore, poiché mostra una maggiore sensibilità a parità di tasso di falsi positivi.

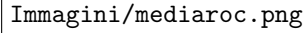
1.8.2 Media delle curve ROC

Quando si valutano modelli di classificazione attraverso diverse ripetizioni sperimentali, oppure tramite tecniche di suddivisione del dataset, è spesso necessario ottenere una rappresentazione sintetica delle prestazioni. In questi contesti le singole curve ROC, prodotte per ciascun esperimento, possono differire tra loro a causa della variabilità indotta dai diversi campioni di training e test.

Una strategia consiste nel combinare tutte le evidenze in un'unica curva ROC media. Se le curve provengono da differenti suddivisioni del test set, è possibile aggregare i risultati considerando, per ciascun valore di soglia, il punto medio tra le coppie (FPR, TPR) generate dai diversi esperimenti. La procedura tipica adottata è la seguente:

- per ogni soglia considerata, si determinano i corrispondenti valori di TPR e FPR ottenuti in ciascun esperimento
- tali valori vengono mediati punto per punto, ottenendo così una curva ROC rappresentativa del comportamento medio del modello

Oltre alla media, risulta spesso utile quantificare la variabilità delle curve. A tal fine si utilizzano misure di **varianza bidimensionale**, che rappresentano la dispersione dei punti ROC attorno alla curva media. Questo approccio consente non solo di osservare il valore atteso delle prestazioni, ma anche di valutarne la stabilità, aspetto fondamentale nelle analisi comparative tra modelli.



2 Apprendimento supervisionato

L'apprendimento supervisionato rappresenta il paradigma in cui un modello viene addestrato a partire da esempi etichettati. Per ogni campione del dataset, è nota la **verità** (o *target*) associata. Si distinguono due principali tipologie di task:

1. **Classificazione**: il problema consiste nell'assegnare ogni campione a una classe appartenente a un insieme discreto e finito di **etichette** (*labels*)
2. **Regressione**: il modello deve predire un valore numerico continuo, tipicamente appartenente a un intervallo dei reali

2.1 Teoria della decisione e probabilità

Alla base dell'apprendimento supervisionato vi è un'interpretazione probabilistica dell'informazione contenuta nei dati.

2.1.1 Probabilità a priori

Si consideri un dataset costituito da campioni discreti, ciascuno associato a una determinata classe. L'idea fondamentale è che la struttura statistica del dataset consenta di stimare la **probabilità a priori** delle classi.

Sia dunque definito lo spazio delle classi come:

$$\Omega = \{\omega_1, \omega_2, \dots, \omega_c\},$$

dove ogni ω_i rappresenta una classe distinta.

Supponiamo che n_i sia il numero di campioni appartenenti alla classe ω_i e che il numero totale di campioni nel dataset sia N . La **probabilità a priori** della classe ω_i è data da:

$$P(\omega_i) = \frac{n_i}{N} \quad (2.1)$$

Tale quantità esprime la probabilità di osservare un campione della classe ω_i prima ancora di analizzare le sue caratteristiche.

Si consideri ora il caso di una **dicotomia**, un problema di classificazione binaria. Se disponessi di un dataset molto ampio, potrei costruire un semplice sistema di decisione basato esclusivamente sulle probabilità a priori delle classi: scegliere la classe più frequente nel dataset.

Indicando con

$$\underline{x} \in \mathbb{R}^n,$$

il vettore delle *features*, una possibile regola decisionale potrebbe essere formalizzata come:

$$O(\underline{x}) = \begin{cases} \omega_1, & \text{se } P(\omega_1) > P(\omega_2), \\ \omega_2, & \text{altrimenti} \end{cases}$$

Una decisione di questo tipo, tuttavia, non utilizza alcuna informazione contenuta nelle caratteristiche del campione: ignora completamente $\underline{x}$ e si limita a scegliere la classe più probabile nel dataset. Per ottenere una classificazione realmente informata, conviene invece considerare le **probabilità condizionate** dei dati rispetto alle classi. In particolare:

$$p(\underline{x} \mid \omega_1), \quad (2.2)$$

rappresenta la **densità di probabilità condizionata** di osservare il campione $\underline{x}$ sapendo che esso appartiene alla classe ω_1 .

2.2 Densità di probabilità condizionata a una classe

Data una collezione di campioni etichettati, è possibile stimare la **densità di probabilità condizionata** di osservare un certo vettore di misura rispetto a una classe specifica. Per ciascuna classe si considerano tutti i campioni che le appartengono e si analizza come essi si distribuiscono nello spazio delle features.

Nel caso di una singola variabile, la stima può essere ottenuta contando quanti campioni assumono un determinato intervallo di valori e normalizzando tale conteggio rispetto al numero totale di campioni della classe. Questo procedimento consente di ricostruire, in forma discreta, una curva che approssima la densità di probabilità condizionata:

$$p(\underline{x} \mid \omega_i),$$

la quale descrive quanto sia probabile osservare il campione $\underline{x}$ sapendo di trovarsi nella classe ω_i .

Da Bayes, sappiamo che la probabilità congiunta si può sviluppare come il rapporto di una probabilità condizionata per un altro termine. La probabilità congiunta di una determinata classe ω_j data un'osservazione $\underline{x}$ si può scrivere:

$$P(\omega_j, \underline{x}) = \underbrace{p(\underline{x} \mid \omega_j)}_{\text{verosimiglianza}} \cdot \underbrace{P(\omega_j)}_{\text{p. a priori}} = \underbrace{P(\omega_j \mid \underline{x})}_{\text{p. a posteriori}} \underbrace{P(\underline{x})}_{\text{evidenza}}$$

dove l'evidenza è pari a:

$$P(\underline{x}) = \sum_j P(\omega_j) p(\underline{x} | \omega_j)$$

$$P(\omega_j | \underline{x}) = \frac{p(\underline{x} | \omega_j) P(\omega_j)}{P(\underline{x})} \quad (2.3)$$

L'equazione (2.3) definisce la **probabilità a posteriori**: essa quantifica la probabilità che il campione appartenga alla classe ω_j **dopo aver osservato** la misura $\underline{x}$. In altre parole, ricevuto il vettore di feature $\underline{x}$, l'obiettivo è determinare a quale classe esso sia più plausibilmente associato.

Nel caso di un problema binario, la regola decisionale assume la forma:

$$O(\underline{x}) = \begin{cases} \omega_1, & \text{se } P(\omega_1 | \underline{x}) > P(\omega_2 | \underline{x}), \\ \omega_2, & \text{altrimenti} \end{cases}$$

La scelta finale consiste quindi nel selezionare la classe associata al valore più elevato della probabilità a posteriori. Sostituendo con il teorema di Bayes si ottiene:

$$\frac{p(\underline{x} | \omega_1) P(\omega_1)}{P(\underline{x})} > \frac{p(\underline{x} | \omega_2) P(\omega_2)}{P(\underline{x})}$$

$$\implies p(\underline{x} | \omega_1) P(\omega_1) > p(\underline{x} | \omega_2) P(\omega_2)$$

2.2.1 Rischio condizionato

Le azioni che il decisore può intraprendere non coincidono necessariamente con le classi presenti nel problema. Per tener conto di questa distinzione, si introduce:

$$\lambda(\alpha_i | \omega_j), \quad (2.4)$$

che rappresenta la perdita associata al compiere l'azione α_i quando la classe osservata è ω_j . Nota una decisione da prendere, è di utile interesse analizzare il rischio di avere una determinata azione avendo osservato una certa classe. Esso si esprime come:

$$R(\alpha_i | \underline{x}) = \sum_{j=1}^c \lambda(\alpha_i | \omega_j) P(\omega_j | \underline{x}) \quad \text{rischio condizionato} \quad (2.5)$$

Di maggiore interesse è il **rischio totale**, il rischio di tutte le azioni che si possono compiere:

$$R_{\text{tot}} = \int R(\alpha_i | \underline{x}) p(\underline{x}) d\underline{x} \implies \sum_{\underline{x}} R(\alpha_i | \underline{x}) p(\underline{x}) \quad (2.6)$$

L'obiettivo è chiaramente avere un sistema di decisione che minimizzi il rischio totale per qualunque azione compibile:

$$\min_{\alpha_i} R_{\text{tot}}$$

Esempio Due classi e due azioni:

$$\omega_1, \omega_2 \quad \alpha_1, \alpha_2$$

$$\alpha_1 \longrightarrow \text{decido } \omega_1$$

$$\alpha_2 \longrightarrow \text{decido } \omega_2$$

$$\lambda(\alpha_i | \omega_j) = \lambda_{ij}$$

Calcoliamo i due rischi:

$$R(\alpha_1 | \underline{x}) = \lambda_{11} P(\omega_1 | \underline{x}) + \lambda_{12} P(\omega_2 | \underline{x})$$

$$R(\alpha_2 | \underline{x}) = \lambda_{21} P(\omega_1 | \underline{x}) + \lambda_{22} P(\omega_2 | \underline{x})$$

$$O(\underline{x}) = \begin{cases} \alpha_1, & \text{se } R(\alpha_1 | \underline{x}) < R(\alpha_2 | \underline{x}), \\ \alpha_2, & \text{altrimenti} \end{cases}$$

Il primo outcome si può riscrivere come:

$$\implies \lambda_{11} P(\omega_1 | \underline{x}) + \lambda_{12} P(\omega_2 | \underline{x}) < \lambda_{21} P(\omega_1 | \underline{x}) + \lambda_{22} P(\omega_2 | \underline{x})$$

$$P(\omega_1 | \underline{x}) [\lambda_{11} - \lambda_{21}] < P(\omega_2 | \underline{x}) [\lambda_{22} - \lambda_{12}]$$

$$\lambda_{11} = \lambda(\alpha_1 | \omega_1), \quad \lambda_{21} = \lambda(\alpha_2 | \omega_1), \quad \alpha_1 \rightarrow \text{decido } \omega_1$$

Ricordando i valori della perdita, le quantità che moltiplicano i valori della probabilità a posteriori sono entrambi negative. Posso invertire i segni:

$$P(\omega_1 | \underline{x}) [\lambda_{21} - \lambda_{11}] > P(\omega_2 | \underline{x}) [\lambda_{12} - \lambda_{22}]$$

Riscriviamo le probabilità a posteriori con il teorema di Bayes:

$$\begin{aligned} \frac{p(\underline{x} | \omega_1) P(\omega_1)}{P(\underline{x})} [\lambda_{21} - \lambda_{11}] &> \frac{p(\underline{x} | \omega_2) P(\omega_2)}{P(\underline{x})} [\lambda_{12} - \lambda_{22}] \\ \implies \underbrace{\frac{p(\underline{x} | \omega_1)}{p(\underline{x} | \omega_2)}}_{\text{rapporto di verosimiglianza}} &> \underbrace{\frac{P(\omega_2)}{P(\omega_1)} \frac{\lambda_{12} - \lambda_{22}}{\lambda_{21} - \lambda_{11}}}_{\text{rischio minimo } O(\underline{x})=\alpha_1} \end{aligned} \quad (2.7)$$

Se il rapporto tra le due verosimiglianze è maggiore del prodotto del rapporto inverso delle probabilità delle classi e del rapporto tra le perdite, allora la relazione è soddisfatta e quindi si sceglie l'azione α_1 , altrimenti α_2 . Questo è un **sistema decisionale basato sulla minimizzazione sul rapporto di verosimiglianza**.

Esempio Le classi sono:

$$\omega_1 : \text{apro (A)} \quad P(\omega_1) = 0.7$$

$$\omega_2 : \text{chiudo (C)} \quad P(\omega_2) = 0.3$$

Le azioni sono:

$$\alpha_1 : \text{mando pattuglia (MP)}$$

$$\alpha_2 : \text{non mando pattuglia (NMP)}$$

$$p(\underline{x} \mid \omega_1) = p(\underline{x} \mid A)$$

$$p(\underline{x} \mid \omega_2) = p(\underline{x} \mid C)$$

Scegliamo come feature il numero di veicoli al secondo:

$$x = \text{num veicoli}/s \quad x \in \mathbb{R}$$

Supponiamo di fare delle misurazioni e di ottenere la $p(\underline{x} \mid A)$ e $p(\underline{x} \mid C)$.

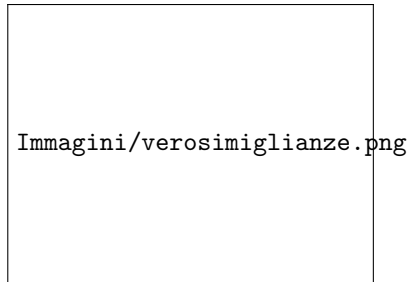


Figura 2.1: Distribuzione delle probabilità condizionate

Analizziamo quindi il **rapporto di verosimiglianza**:

$$\frac{p(\underline{x} \mid \omega_1)}{p(\underline{x} \mid \omega_2)} > \frac{P(\omega_2)}{P(\omega_1)} \frac{\lambda_{12} - \lambda_{22}}{\lambda_{21} - \lambda_{11}} \implies \text{se vera } \alpha_1 \text{ (MP)}$$

Consideriamo le perdite come:

$$\lambda_{11} = \lambda(\text{MP} \mid A) = 0.4$$

$$\lambda_{12} = \lambda(\text{MP} \mid C) = 1$$

$$\lambda_{21} = \lambda(\text{NMP} \mid A) = 0.6$$

$$\lambda_{22} = \lambda(\text{NMP} \mid C) = 0$$

In condizione operativa, considero una determinata x^* . Sostituiamo i valori:

$$\frac{p(\underline{x} \mid \omega_1)}{p(\underline{x} \mid \omega_2)} > \underbrace{\frac{P(\omega_2)}{P(\omega_1)} \frac{\lambda_{12} - \lambda_{22}}{\lambda_{21} - \lambda_{11}}}_{\theta}$$

$$\frac{0.4}{0.1} > \underbrace{\frac{0.3}{0.7} \frac{1 - 0}{0.6 - 0.4}}_{\theta}$$

$$4 > \frac{3}{7} \cdot \frac{1}{0.2}$$

$$4 > \frac{3}{7} \cdot \frac{10}{2}$$

$$4 > \frac{15}{7} ; \quad 4 > 2,1$$

$$\Rightarrow O(\underline{x}) = \alpha_1 \text{ (MP)}$$

Si nota come θ è un numero fisso che non dipende dalla feature $\underline{x}$ osservata. Compiere o meno un'azione significa capire se il rapporto delle verosimiglianze è maggiore o meno di un numero fisso θ



Figura 2.2: Regioni di decisione

θ è una soglia tra le due regioni di decisione, ho quindi identificato una **regione di decisione**, un intervallo dei valori $\underline{x}$ per cui associo un'azione o per cui ne associo un'altra. Nel caso di due features:



Figura 2.3: Regioni di decisione con più features

La forma di divisione della regione prende il nome di **frontiera decisionale**.

Consideriamo ora il caso in cui i campioni siano descritti da un insieme di features numeriche. In uno **scatter plot**, ogni campione è rappresentato come un punto nello spazio delle misure. L'obiettivo della classificazione consiste nel separare i punti appartenenti a classi differenti, in modo che quelli di una classe si trovino da un lato e quelli dell'altra classe dall'altro.

Se, ad esempio, le features disponibili sono tre, i campioni possono essere rappresentati come punti in uno spazio tridimensionale. In questo contesto, qualunque algoritmo di classificazione ha il compito di individuare una **frontiera decisionale** che separi nel modo più efficace possibile le diverse classi. Sebbene ogni algoritmo adotti criteri differenti per costruire tale frontiera, l'obiettivo rimane invariato: discriminare correttamente le classi nello spazio delle feature.

A seconda della forma della frontiera decisionale, si distinguono diversi casi:

- Se la frontiera è una **retta** (o, in dimensioni superiori, un iperpiano), il problema si dice **linearmente separabile**
- Se la separazione richiede curve più complesse (ad esempio una frontiera di tipo **cubico** o comunque non lineare), il problema non è linearmente separabile

L'algoritmo di apprendimento può quindi costruire frontiere decisionali di diversa complessità. Il processo di apprendimento può essere interpretato come la stima dei parametri che definiscono la forma della frontiera stessa. Quanto più la frontiera è complessa, tanto maggiore dev'essere la quantità di dati disponibili per apprendere correttamente tali parametri: una maggiore quantità di campioni conduce, in generale, a una stima più affidabile della separazione tra le classi. Avere troppe features rispetto al numero di campioni disponibili, non permette all'algoritmo di costruire frontiere affidabili (**course of dimensionality**). E' sempre buona norma che:

$$\frac{N}{\text{\#features}} > 10 \quad \text{con } N = \text{numero di campioni}$$

Course of dimensionality E' il fenomeno per cui all'aumentare del numero delle features in una **prima fase** si nota un miglioramento delle prestazioni del modello, ma all'aumentare ancora del numero delle features il modello tende a **degradarsi**.

2.3 Classificatore Bayesiano

Dato un insieme di possibili classi

$$\Omega = \{w_1, w_2, \dots, w_c\},$$

Si può associare ad ogni classe una funzione, detta **funzione discriminante**:

$$g_i(\underline{x}), \quad (2.8)$$

che riceve in ingresso $\underline{x}$ e dice quanto quel classificatore si 'attiva' su quella classe. E' **discriminante** perché è un concetto generale indipendente dal classificatore. Nel classificatore bayesiano la funzione discriminante può essere la **probabilità a posteriori** oppure il concetto di **rischio**, con l'obiettivo di massimizzare/minimizzare $g_i(\underline{x})$.

Supponiamo che $g_i(\underline{x})$ sia da massimizzare, si può quindi assumere una regola di decisione del tipo:

$$\text{se } g_i(\underline{x}) > g_j(\underline{x}) \quad \forall i \neq j \longrightarrow \omega_i,$$

formalmente, prendendo la probabilità più grande. Considerando invece una quantità che analizza il **rischio**, allora $g_i(\underline{x})$ è della forma:

$$g_i(\underline{x}) = -R(\alpha_i \mid \underline{x})$$

Nel caso del classificatore bayesiano, $g_i(\underline{x})$ può essere posta come:

$$g_i(x) = P(w_i \mid x) = \frac{P(x \mid w_i) P(w_i)}{P(x)}$$

dove:

$$P(x) = \sum_c P(x \mid w_c) P(w_c)$$

Si può rimuovere il denominatore poiché è uguale per tutti i termini $g_i(\underline{x})$. I termini a destra sono positivi, non è quindi un errore riscrivere $g_i(\underline{x})$ con il logaritmo:

$$\begin{aligned} \ln g_i(\underline{x}) &= \ln P(w_i \mid \underline{x}) = \ln (P(\underline{x} \mid w_i) P(w_i)) \\ \tilde{g}_i(x) &= \ln p(\underline{x} \mid w_i) + \ln P(w_i) \end{aligned} \quad (2.9)$$

Caso binario Nel caso di due classi $\Omega = \{w_1, w_2\}$ il classificatore bayesiano calcola due funzioni discriminanti $g_1(x)$ e $g_2(x)$, assegnando il campione alla classe che produce il valore maggiore. Definiamo quindi la funzione discriminante differenziale:

$$g(x) = g_1(x) - g_2(x)$$

per cui la regola decisionale diventa $g(x) > 0 \Rightarrow x \in w_1$. Utilizzando l'espressione logaritmica delle probabilità si ottiene:

$$g(x) = \ln p(x | w_1) + \ln P(w_1) - \ln p(x | w_2) - \ln P(w_2),$$

che può essere riscritta in forma compatta come

$$g(x) = \ln \frac{p(x | w_1)}{p(x | w_2)} + \ln \frac{P(w_1)}{P(w_2)}$$

Assumiamo che la verosimiglianza $p(x | w_i)$ sia una distribuzione normale in $\mathbb{R}^d$, dove d rappresenta la dimensione del vettore delle feature. Nel caso particolare $d = 1$, il vettore x contiene una sola feature e la distribuzione è una normale unidimensionale:

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right)$$

Nel caso multidimensionale ($d > 1$):

$$p(x) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\underline{x} - \underline{\mu})^T \Sigma^{-1} (\underline{x} - \underline{\mu})\right),$$

dove Σ è la matrice di covarianza.

Utilizzando la funzione discriminante logaritmica:

$$\tilde{g}_i(x) = \ln p(x | w_i) + \ln P(w_i),$$

si ottiene esplicitamente:

$$\tilde{g}_i(x) = -\frac{d}{2} \ln(2\pi) - \frac{1}{2} \ln |\Sigma_i| - \frac{1}{2} (\underline{x} - \underline{\mu}_i)^T \Sigma_i^{-1} (\underline{x} - \underline{\mu}_i) + \ln P(w_i)$$

Consideriamo il caso in cui la matrice di covarianza della classe w_i sia del tipo

$$\Sigma_i = \sigma^2 I$$

La varianza è la stessa per tutte le feature, le gaussiane differiscono solo per media. I campioni che appartengono a diverse classi hanno tutti la stessa matrice di covarianza e per come è scritta, le features sono tutte indipendenti. Vale quindi:

$$|\Sigma_i| = \sigma^{2d}, \quad \Sigma_i^{-1} = \frac{1}{\sigma^2} I.$$

La distanza quadratica può essere riscritta come:

$$(\underline{x} - \underline{\mu}_i)^\top \Sigma_i^{-1} (\underline{x} - \underline{\mu}_i) = \frac{\|\underline{x} - \underline{\mu}_i\|^2}{\sigma^2}$$

Sostituendo queste espressioni nella forma generale della funzione discriminante,

$$\tilde{g}_i(x) = -\frac{d}{2} \ln(2\pi) - \frac{1}{2} \ln |\Sigma_i| - \frac{1}{2} (\underline{x} - \underline{\mu}_i)^\top \Sigma_i^{-1} (\underline{x} - \underline{\mu}_i) + \ln P(w_i),$$

e trascurando i termini comuni a tutte le $g_i(\underline{x}), g_j(\underline{x})$, si ottiene:

$$\tilde{g}_i(x) = -\frac{1}{2\sigma^2} \|x - \mu_i\|^2 + \ln P(w_i)$$

La quantità al quadrato si espande come:

$$\|\underline{x} - \underline{\mu}_i\|^2 = \underline{x}^\top \underline{x} - 2\underline{\mu}_i^\top \underline{x} + \underline{\mu}_i^\top \underline{\mu}_i$$

La funzione discriminante risulta quindi:

$$\tilde{g}_i(x) = -\frac{\underline{x}^\top \underline{x}}{2\sigma^2} + \frac{\underline{\mu}_i^\top \underline{x}}{\sigma^2} - \frac{\underline{\mu}_i^\top \underline{\mu}_i}{2\sigma^2} + \ln P(w_i)$$

Poiché il termine $-\frac{\underline{x}^\top \underline{x}}{2\sigma^2}$ è identico per tutte le classi, può essere eliminato. Rimane una funzione affine:

$$\tilde{g}_i(x) = \underline{a}_i^\top \underline{x} + \underline{b}_i \quad (2.10)$$

dove:

$$\underline{a}_i = \frac{\underline{\mu}_i^\top}{\sigma^2}, \quad \underline{b}_i = -\frac{\underline{\mu}_i^\top \underline{\mu}_i}{2\sigma^2} + \ln P(w_i) \quad (2.11)$$

Dall'equazione (2.10) si osserva che, nell'ipotesi di distribuzione normale con covarianze identiche tra le classi, varianze uguali per tutte le feature e indipendenza tra le componenti di x , il classificatore bayesiano genera una **frontiera di decisione lineare**.

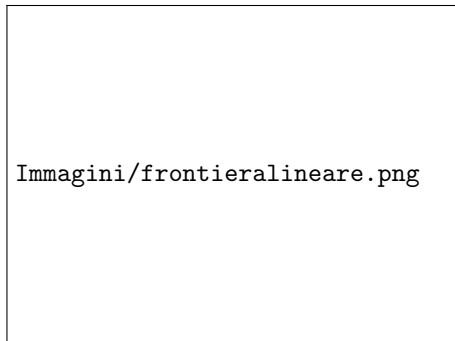


Figura 2.4: Frontiera lineare in pianta del classificatore bayesiano

Un'ulteriore semplificazione si ottiene assumendo che le probabilità a priori siano uguali:

$$\ln P(w_1) = \ln P(w_2)$$

In tal caso i termini costanti si annullano e la regola decisionale si riduce al confronto diretto delle distanze del punto x dai centri delle due classi. Questo porta al metodo del **template matching**: per un nuovo punto x si misura la distanza da ciascun prototipo e si assegna la classe corrispondente al prototipo più vicino:

$$d_1 < d_2 \Rightarrow x \in w_1$$

Consideriamo l'ipotesi in cui tutte le classi condividono la stessa matrice di covarianza,

$$\Sigma_i = \Sigma \quad \forall i$$

Le distribuzioni $p(x | w_i)$ hanno la stessa forma (ellittica), differendo solo per la posizione del loro centro μ_i .

2.4 Alberi di decisione

Gli alberi di decisione sono modelli utilizzati nell'apprendimento supervisionato per affrontare problemi di classificazione e regressione. A differenza del classificatore bayesiano, che richiede features numeriche e ipotesi sulla distribuzione dei dati, gli alberi di decisione possono lavorare anche con features **discrete** e **simboliche**. Risultano quindi particolarmente adatti alla gestione di variabili **nominali**.

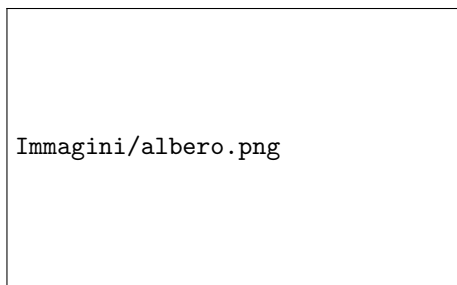


Figura 2.5: Esempio di albero di decisione elementare

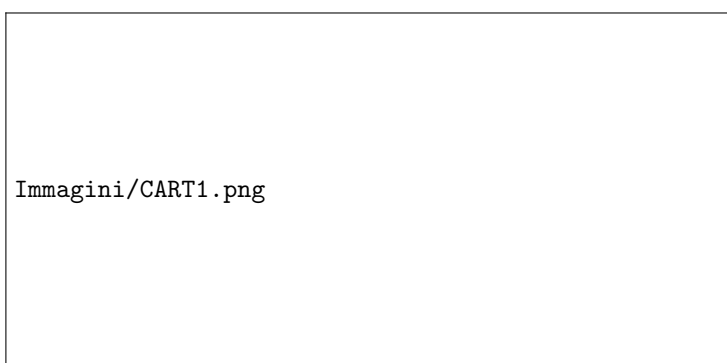
Il nodo iniziale dell'albero, rappresentato in blu in Figura 2.4, prende il nome di **radice**. I nodi terminali, quelli che non presentano ulteriori ramificazioni, sono chiamati nodi **foglia**. La **profondità dell'albero** corrisponde al numero di livelli che lo compongono; qualora le ramificazioni conducano a profondità differenti, l'albero si definisce **sbilanciato**. Un albero è invece **binario** quando, in ogni nodo, sono ammesse esclusivamente due possibili diramazioni.

Quando l'obiettivo è assegnare ciascun elemento a una categoria, si parla di albero di decisione di **classificazione**. Se invece il modello deve prevedere valori

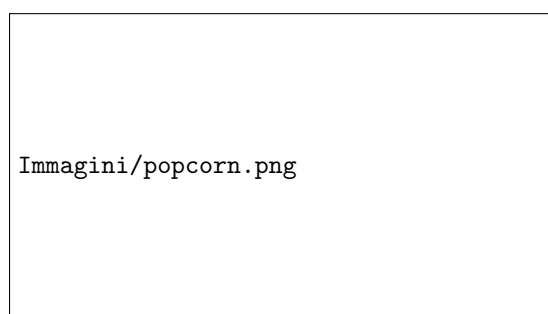
numerici, l'albero è detto di **regressione**. Nel primo caso, le foglie rappresentano una classe; nel secondo caso, le foglie contengono la stima numerica da restituire come output. La stessa feature può essere usata più volte anche con valori differenti, sulle foglie le classi possono essere ripetute.

2.4.1 CART (Classification and Regression Tree)

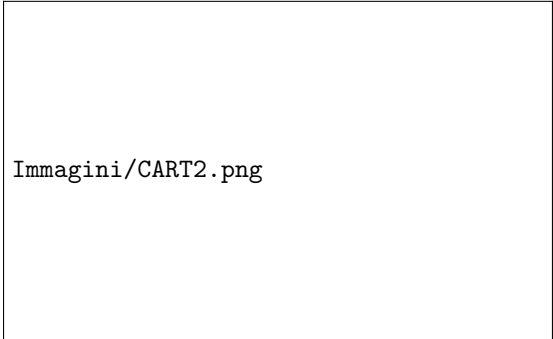
Il modello CART può essere utilizzato sia come classificatore sia come regressore. Per costruzione, ogni nodo effettua sempre una suddivisione in due rami: l'albero è dunque **binario** e, poiché in ciascun nodo viene testata una sola feature, esso è definito **monotetico**. Analizziamo l'esempio riportato in Figura:



Uno degli aspetti centrali nella costruzione di un albero CART è la ricerca di foglie quanto più possibile **pure**, cioè contenenti campioni appartenenti alla stessa classe. Una foglia pura consente infatti di associare senza ambiguità una classe a tutti i punti che vi ricadono. Per comprendere il comportamento del modello, consideriamo inizialmente la prima feature:



Possiamo procedere in modo analogo anche utilizzando la seconda feature:



Immagini/CART2.png

Dall'analisi dei nodi foglia si osserva che, complessivamente, solo uno di essi risulta completamente puro. È quindi naturale domandarsi se esista una misura che quantifichi il grado di **impurità** di una foglia, così da valutare e confrontare la qualità di uno split. Tale misura rappresenta il punto di partenza per stabilire quale suddivisione sia più efficace nel processo di costruzione dell'albero.

2.4.2 Impurità di Gini

L'impurità di Gini su una foglia generica è definita come:

$$i(N) = 1 - \sum_j \hat{P}^2(w_j) \quad (2.12)$$

dove $\hat{P}(w_j) = \hat{P}(x \in w_j \mid N)$ è la probabilità stimata che un campione appartenente al nodo N sia della classe w_j (quindi una probabilità **condizionata al nodo** e non probabilità a priori). Per una foglia che contiene due classi, l'impurità è:

$$i(N) = 1 - (\text{Prob1})^2 - (\text{Prob2})^2$$

Nel dataset considerato, la variabile *età* è numerica e continua. Per poter calcolare l'impurità di Gini rispetto a questa feature, si procede nel seguente modo:

- Si ordinano i valori dell'*età* in modo crescente, dal minimo al massimo
- Si calcola la media tra ogni coppia di valori consecutivi
- Per ciascuna soglia si costruisce idealmente uno split dell'albero, trattando la soglia come potenziale radice, calcolando l'impurità di Gini per entrambe le foglie risultanti

La soglia che produce l'impurità totale più bassa viene selezionata come miglior split. Se due soglie restituiscono lo stesso valore minimo, l'algoritmo ne sceglie una in modo arbitrario.

In generale, un albero non si ferma al primo livello: il processo di suddivisione viene ripetuto ricorsivamente, ma non più sull'intero dataset. Ad ogni passo si

considera soltanto il sottoinsieme di campioni assegnato al nodo corrente. La procedura di crescita dell'albero termina quando il nodo risulta **puro**, quando contiene campioni appartenenti a una sola classe. In un dataset molto esteso è raro che una foglia presenti impurità nulla, per questo motivo si stabilisce una soglia minima accettabile di impurità, oltre la quale il nodo non viene ulteriormente suddiviso. Spingere l'albero a raggiungere impurità zero può generare **overfitting**: il modello si adatta in modo eccessivo alle particolarità del dataset, creando regole troppo specifiche che non generalizzano bene.

Un modo semplice per accorgersi dell'overfitting nei modelli ad albero consiste nell'osservare la profondità dell'albero stesso: maggiore è la profondità, maggiore è il rischio di aver creato un modello troppo complesso. Gli alberi decisionali, essendo **intrinsecamente spiegabili**, consentono un'ispezione immediata della loro struttura. In un dataset con molte colonne, dove ogni colonna rappresenta una feature, l'albero di decisione, per costruzione, non utilizza necessariamente tutte le variabili disponibili. Durante il processo di splitting, il modello seleziona solo le feature che permettono di ridurre maggiormente l'impurità, individuando in modo naturale quelle realmente **discriminanti**, le caratteristiche che contribuiscono in maniera significativa alla decisione. Un albero di decisione svolge simultaneamente due compiti: la **classificazione** (o regressione) e la **selezione delle feature**.

Per evitare fenomeni di overfitting si può adottare il **pruning**. L'albero viene inizialmente costruito fino alla massima profondità, in una forma inevitabilmente overfittata. Successivamente, si valuta la perdita di prestazioni su un validation set e si procede a potare (rimuovere) i rami meno utili, mantenendo solo quelli che realmente contribuiscono al miglioramento delle prestazioni generali del modello.

2.4.3 Entropia

L'entropia è una misura alternativa di impurità. Quantifica il grado di disordine all'interno di una foglia. L'impurità di entropia per un nodo N è definita come:

$$i(N) = - \sum_j P(w_j) \log_2 P(w_j) \quad , \quad (2.13)$$

dove $P(w_j)$ è la probabilità stimata che un campione appartenente al nodo sia della classe w_j . L'entropia assume valore massimo quando le classi sono equiprobabili (massimo disordine) e valore nullo quando la foglia è completamente pura.

2.4.4 Varianza

Nel caso in cui il nodo contenga solamente **due possibili etichette** di classe, un'ulteriore misura di impurità è data dalla **varianza**. Essa valuta quanto le due classi siano bilanciate all'interno della foglia. L'impurità di varianza per un

nodo N è definita come:

$$i(N) = P(w_1) P(w_2) \quad , \quad (2.14)$$

dove $P(w_1)$ e $P(w_2)$ sono le probabilità stimate delle due classi nel nodo. La varianza è nulla quando una delle due classi è assente (foglia pura) e raggiunge il suo massimo quando le due classi sono equamente distribuite.

2.4.5 Misclassificazione

Questa metrica quantifica l'errore che si commetterebbe assegnando al nodo la classe più probabile. E' definita come:

$$i(N) = 1 - \max_j P(w_j) \quad , \quad (2.15)$$

dove $\max_j P(w_j)$ indica la probabilità della classe più rappresentata nella foglia. L'impurità è nulla quando tutte le osservazioni appartengono alla stessa classe, e cresce man mano che le classi diventano più bilanciate.



Figura 2.6: Confronto qualitativo tra misclassification, Gini ed entropia al variare della probabilità P .

Come mostrato nel grafico, la misclassification è una misura meno sensibile rispetto a Gini ed entropia: cresce più lentamente e quindi discrimina con minore finezza gli split meno puri.

Costruendo un nodo successivo, si calcola la **caduta d'impurità** $\Delta i(N)$, pari ad:

$$\begin{aligned} \Delta i(N) &= i(N) - P_L i(N_L) - P_R i(N_R) \\ &= i(N) - P_L i(N_L) - (1 - P_L) i(N_R) \end{aligned} \quad (2.16)$$

2.4.6 Complessità

La complessità computazionale di un albero di decisione distingue due fasi principali: **addestramento** (training) e **predizione** (test). Durante la fase di addestramento, l'algoritmo deve valutare, per ogni nodo, tutte le possibili soglie di split su ciascuna delle d feature e su n campioni. Ordinare i campioni richiede

un costo pari a $\mathcal{O}(n \log n)$ e, poiché la procedura viene ripetuta sui vari livelli dell'albero, la complessità complessiva risulta:

$$\mathcal{O}(dn (\log n)^2) \quad , \quad (2.17)$$

dove:

- d è il numero di feature
- n è il numero di campioni
- $(\log n)^2$ deriva dalla costruzione ricorsiva dell'albero con ripetuti ordinamenti

La fase di test, invece, è **estremamente efficiente**: classificare un nuovo campione significa percorrere un solo ramo dell'albero, visitando un numero di nodi pari alla profondità dell'albero. Se l'albero è bilanciato, tale profondità è dell'ordine di $\log n$. La complessità di predizione risulta quindi:

$$\mathcal{O}(\log n) \quad (2.18)$$

Questo rende gli alberi di decisione molto rapidi in **fase di inferenza**, anche su dataset di grandi dimensioni.

L'albero di decisione **non è resiliente ai dati mancanti**: la presenza di valori assenti può compromettere la capacità del modello di instradare correttamente un campione lungo i rami dell'albero. In assenza di tecniche dedicate, è spesso necessario sostituire i valori mancanti con stime o imputazioni. Per rendere l'albero intrinsecamente in grado di gestire un dato mancante, si può introdurre lo **split surrogato**. Durante l'addestramento, oltre allo split principale, si ricerca per ogni nodo una feature alternativa che produca una separazione il più possibile simile. In fase di test, se il valore della feature principale è mancante, lo split surrogato permette comunque di decidere quale ramo seguire. Questa strategia richiede di valutare molteplici split alternativi e di modificare la struttura dei nodi, risultando quindi **computazionalmente molto onerosa**. Per tale motivo viene adottata solo in situazioni specifiche.

2.4.7 C4.5

C4.5 non utilizza lo split surrogato e viene addestrato nello stesso modo di CART. La differenza principale riguarda la **fase di inferenza**: quando un campione presenta una feature mancante proprio nel nodo in cui tale feature dovrebbe essere testata, l'algoritmo non può determinare direttamente quale ramo seguire. C4.5 risale al livello precedente e considera tutti i rami alternativi che discendono da quel punto. Per ogni percorso alternativo, il campione viene propagato lungo i sottoalberi corrispondenti e si contano le foglie che predicono ciascuna classe. La classe finale è assegnata sulla base di questa distribuzione delle predizioni.

Questo approccio non aumenta il tempo di addestramento, poiché la gestione dei dati mancanti avviene solo in inferenza. Tuttavia, comporta un aumento del tempo di predizione, dovuto alla necessità di esplorare più rami dell'albero per ogni campione incompleto.

2.4.8 Ensemble Learning

L'Ensemble Learning consiste nel **combinare più modelli distinti**, solitamente caratterizzati dalla stessa architettura ma con prestazioni differenti sui medesimi campioni. L'idea alla base è che modelli diversi possano commettere errori diversi: aggregando le loro predizioni si ottiene un sistema complessivamente più robusto, in grado di fornire risposte più accurate e stabili rispetto ai singoli modelli considerati isolatamente.

2.4.9 Albero per regressione

Nel caso della regressione, le foglie dell'albero non contengono una classe ma un valore numerico da predire. Al posto dell'impurità di Gini si utilizza come misura di impurità lo **scarto quadratico medio** (MSE), che quantifica quanto i valori presenti nella foglia si discostano dal valore reale. Una volta modificata la funzione obiettivo, il procedimento di costruzione dell'albero rimane invariato. Minimizzare l'MSE come criterio di impurità significa incentivare l'albero a raggruppare, all'interno della stessa foglia, campioni che presentano valori di y simili tra loro. In fase di inferenza, il valore predetto associato a una foglia è semplicemente la **media** dei valori y dei campioni che ricadono in quella foglia.

3 Dimensionality Reduction (Feature Selection)

L'obiettivo della **dimensionality reduction** (o *feature selection*) consiste nell'identificare il sottoinsieme ottimale di variabili rilevanti per la descrizione di un fenomeno, eliminando quelle ridondanti o affette da rumore. Questo processo è fondamentale per:

- **Ottimizzazione del Modello:** Facilitare la costruzione di modelli di apprendimento più robusti
- **Efficienza Computazionale:** Ridurre il numero di descrittori per abbattere il carico computazionale e i requisiti di memoria durante le fasi di training e inferenza
- **Interpretabilità:** Identificare le correlazioni significative e le misure più informative, rendendo il modello più semplice da spiegare e analizzare

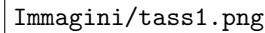
3.1 Tassonomie

Una prima tassonomia, introdotta negli anni 80', divide i metodi di feature selection in tre categorie:

1. **Filtri**: divide o analizza l'insieme delle features indipendentemente dal modello di utilizzo
2. **Wrapper**: identifica le misure più rilevanti, sulla base della relazione con un modello di riferimento
3. **Embedded**: metodi che intrinsecamente al processo di apprendimento, rilevano le misure più rilevanti (es. *alberi decisionali*)

Una migliore tassonomia divide gli approcci in **supervisionato** e **non supervisionato**

3.1.1 Supervised Feature Selection



Immagini/tass1.png

I metodi di **feature ranking** forniscono in uscita il **grado di dipendenza** o meno di una feature dalle altre, tipicamente in modo ordinato. I metodi di **subset selection**, invece, restituiscono dei **sottoinsiemi** di feature rilevanti per l'obiettivo. Il primo non per forza guarda le relazioni reciproche (uno ad uno), i metodi di subset selection invece non forniscono una granularità di accesso all'importanza della feature, ma forniscono l'intorno migliore per uno specifico target. Negli **embedded**, il modello nel suo processo di addestramento restituisce anche un'indicazione di quali sono le features più rilevanti.

3.1.2 Feature Ranking: Pair Wise (*a coppie*)

Analizzano la dipendenza **a coppie** di una feature con le altre, una per una. Nei problemi supervisionati si può studiare anche la relazione tra la **feature** e l'**etichetta**.

Correlazione La correlazione è un'indice che esprime la dipendenza **lineare** tra due variabili, compreso tra -1 ed 1.

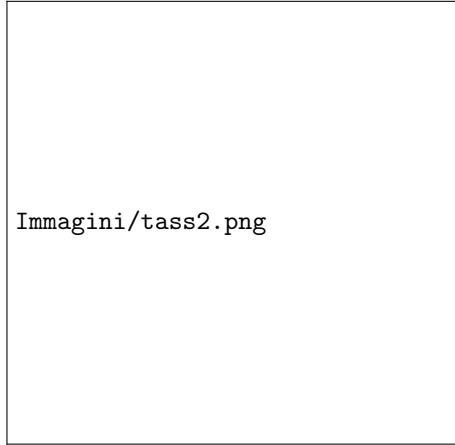


Figura 3.1: Mappa di correlazione

Incertezza Considerano la stima di una misura derivante dalla teoria dell'informazione, nota come **entropia**:

$$H(Y) = - \sum_{k=1}^r t f_j^k \cdot \log_2(t f_j^k) \quad , \quad (3.1)$$

dove:

$$t f_j^k = \frac{n_j^k}{\sum_k n_j^k} \quad , \quad (3.2)$$

è il numero di occorrenze di $t f_j^k$ nella classe c_k . L'entropia fornisce il **guadagno** (*information gain*) di una determinata features e quindi quanto può impattare nell'obiettivo.

$$IG(Y, X) = H(Y) - H(Y|X) \quad (3.3)$$

E' un metodo che si può utilizzare solo per features **discrete**.

Test d'ipotesi Nei test d'ipotesi, l'elemento chiave per la selezione delle feature è il **p-value**, un valore di probabilità compreso nell'intervallo $[0, 1]$. Esso rappresenta la probabilità di osservare un risultato uguale o più estremo di quello ottenuto, assumendo che l'ipotesi nulla (H_0) sia vera. Si interpreta generalmente secondo i seguenti criteri di significatività:

- **p-value $\leq \alpha$ (tipicamente 0.05):** Il risultato è statisticamente significativo. Si rigetta l'ipotesi nulla H_0 , indicando che la feature analizzata ha una relazione rilevante con la variabile target
- **p-value $> \alpha$:** Non vi è evidenza sufficiente per rigettare H_0 . La feature viene considerata non significativa e può essere rimossa

Potere discriminativo I metodi basati sul potere discriminativo valutano la capacità di una feature di massimizzare la separazione tra diverse classi di dati. L'elemento cardine è l'**analisi della varianza**: una variabile è considerata tanto più rilevante quanto più presenta una bassa varianza all'interno della stessa classe (*intra-class variance*) e un'alta varianza tra classi differenti (*inter-class variance*).

3.1.3 Feature ranking: simultaneous

Nei metodi simultanei si analizza la rilevanza di tutte le features, guardando la loro relazione con il target.

Relief Il metodo valuta la qualità delle feature in base a quanto i loro valori siano in grado di distinguere campioni tra loro vicini ma appartenenti a classi diverse. L'algoritmo aggiorna ricorsivamente il peso di ogni attributo analizzando, per un campione estratto, il suo **Near Hit** (il vicino più prossimo della stessa classe) e il suo **Near Miss** (il vicino più prossimo di una classe differente):

$$W_i = W_i - (x_i - \text{nearHit}_i)^2 + (x_i - \text{nearMiss}_i)^2 \quad (3.4)$$

Una feature è rilevante se assume valori simili per istanze della stessa classe e valori diversi per istanze di classi diverse. L'algoritmo è particolarmente **oneroso** perché lo è anche l'**ordinamento**. La relief fa due ordinamenti per ogni feature e per ogni campione estratto.

Principal Component Analysis (PCA) Rientra nell'**unsupervised feature selection**. Si considerino due feature originarie, x ed y , e un insieme di campioni rappresentati dai punti nel piano:

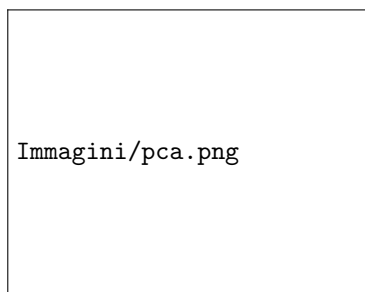


Figura 3.2: Rappresentazione della rotazione degli assi mediante PCA.

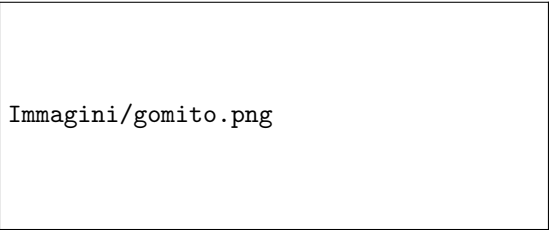
La PCA mira a determinare un nuovo sistema di coordinate composto da **assi ortogonali** (per n feature di partenza si otterranno n componenti principali) attraverso una **rototraslazione** dello spazio originario. L'obiettivo è posizionare i nuovi assi in modo da **massimizzare la varianza spiegata** lungo ciascuno

di essi: il primo asse (PC1) viene orientato lungo la direzione di massima dispersione dei dati, il secondo (PC2) lungo la direzione ortogonale con la rimanente varianza massima, e così via. L'algoritmo alterna i seguenti passaggi:

1. Trovare il centroide della trasformazione come punto medio di tutti i miei punti
2. Si cerca la prima componente principale: la retta che passa per l'origine che massimizza la varianza, ovvero che rappresenta meglio il dato.
3. La seconda componente è la retta perpendicolare alla nuova componente principale

Questo processo genera un numero di **nuove feature** (componenti principali) pari a quelle iniziali, con la proprietà fondamentale di essere tra loro **incorrelate** e ordinate in modo tale che la quota di informazione (varianza) catturata sia **decrescente**. Ciò permette di ridurre la dimensionalità eliminando le componenti che contribuiscono in misura trascurabile alla spiegazione del dataset.

Per **ridurre la dimensionalità** si utilizza il **metodo del gomito/ginocchio**. Si costruisce un grafico che riporta la varianza di ogni feature in ordine decrescente. Dove si forma il gomito tipicamente è dove si può tagliare.



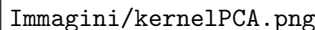
Immagini/gomito.png

Linear Discriminant Analysis (LDA) A differenza della PCA, che è una tecnica non supervisionata, la **LDA** adotta un **approccio supervisionato** con l'obiettivo di trovare una proiezione spaziale che massimizzi la separazione tra le classi. Mentre la PCA si concentra sulla massimizzazione della varianza totale dei dati, la LDA cerca di determinare un insieme di assi (discriminanti lineari) che massimizzino il rapporto tra la varianza tra le classi e la varianza interna alle classi (compattezza dei vari gruppi)

Kernel PCA Nel contesto del machine learning, il **kernel** definisce una trasformazione geometrica dello spazio iniziale con l'obiettivo di proiettare i dati in una dimensione dove le feature risultano linearmente separabili. L'inserimento di una superficie di separazione (iperpiano) consiste nel trovare la posizione ottima per dividere i campioni in regioni distinte.

Mentre la riduzione della dimensionalità tende a compattare lo spazio, l'aumento delle dimensioni (*feature mapping*) espande le regioni in cui può passare una

superficie separatrice. Sebbene in spazi ad alta dimensionalità il problema possa apparire meno determinato, tale espansione facilita il compito del modello nel trovare una soluzione ottima. Il **Kernel Trick** è una strategia che permette di operare in questi spazi trasformati, anche di dimensione infinita, senza calcolare esplicitamente le coordinate dei punti, rendendo il compito di classificazione computazionalmente efficiente.



Il **Kernel PCA** è il metodo che applica preventivamente una funzione kernel per rendere lo spazio delle feature linearmente separabile, procedendo solo successivamente alla proiezione tipica della PCA tradizionale.

3.2 Subset Selection: Configuration

I metodi di subset selection sono quei metodi dove l'outcome è un insieme di feature rilevanti per il target. Nei metodi di **configurazione** si sfruttano le **correlazioni** tra le features e vengono analizzati i legami tra:

feature-target

feature-feature

Max-Min L'idea è **massimizzare** la **rilevanza** rispetto al **target** e **minimizzare** la **ridondanza** tra le **feature**.

Si vogliono delle feature molto informative e poco correlate tra di loro.

1. Si calcola la **rilevanza** delle feature con il target (es. coefficiente di correlazione)
2. Si determina la feature più rilevante e rispetto ad essa ci calcola la correlazione con le altre features
3. Si sceglie la feature X_j meno correlata con lo Score più alto rispetto ad X_i

$$\text{Score}(X_i) = \text{Rilevanza}(X_i, Y_i) - \text{Ridondanza}(X_i, X_2)$$

4. Si effettua una nuova iterazione considerando le feature non scelte, aggiornando il subset S e scartando quindi una feature X_1

3.3 Subset Selection: Evaluation

Nella subset selection supervisionata, gli approcci di valutazione si distinguono in **model-specific** e **model-independent**.

3.3.1 Metodi model-specifier (wrapper)

Questi metodi:

- Utilizzano le prestazioni (tipicamente **accuracy** in cross-validation) di uno specifico modello di apprendimento come criterio di valutazione
- Producono congiuntamente il sottoinsieme di feature e il modello predittivo
- Il sottoinsieme selezionato è generalmente valido solo per il learner usato nella selezione

3.3.2 Metodi model-independent

Questi metodi:

- Si basano su criteri derivati dalle caratteristiche dei dati
- Non costruiscono direttamente un modello predittivo
- Non sono legati ad uno specifico algoritmo di apprendimento
- Il sottoinsieme risultante può essere usato con diversi learner

Considerazioni sulla ricerca I metodi di tipo evaluation richiedono l'esplorazione dello spazio dei sottoinsiemi (di dimensione 2^n con n feature). Le principali strategie di ricerca sono:

- **Complete search** (Ricerca completa): la ricerca esaustiva garantisce la scoperta del sottoinsieme ottimo, ma è computazionalmente molto costosa e spesso inattuabile nella pratica. Se la funzione di valutazione è monotona, è possibile applicare l'algoritmo **branch-and-bound**. Questo approccio permette di evitare l'esplorazione totale di tutte le combinazioni, mantenendo comunque la capacità di trovare il sottoinsieme di feature ottimale rispetto alla funzione di valutazione
- **Heuristic search** (Ricerca euristica): offre un compromesso pratico tra il carico computazionale e l'ottimalità del set di feature trovato. Tra i metodi più noti figurano il *Best first*, la *Sequential Forward Selection* (SFS) e la *Sequential Backward Selection* (SBS). Sebbene SFS e SBS siano algoritmi

generalmente veloci, soffrono dell'effetto di annidamento (*nesting effect*): nella SFS una feature aggiunta non può più essere rimossa nei passaggi successivi, mentre nella SBS una feature scartata non può più essere reinserita. Un limite generale di tutte le strategie euristiche è la loro tendenza a rimanere bloccate nei minimi locali.

- **Non-deterministic search** (Ricerca non deterministica): nasce per superare il problema dei minimi locali tipico delle euristiche. Utilizzando condizioni iniziali o parametri di ricerca randomizzati, queste strategie esplorano lo spazio in modo più ampio. Di conseguenza, esecuzioni diverse dell'algoritmo tenderanno a produrre risultati differenti.

In generale, i metodi basati su configuration risultano più efficienti perché sfruttano correlazioni **feature-target** e **feature-feature** per guidare la ricerca.

3.3.3 Subset Evaluation Complete: Branch and bound

Il metodo **branch-and-bound** permette di trovare il sottoinsieme ottimo di feature senza l'onere computazionale di dover calcolare e valutare in modo esaustivo tutte le combinazioni possibili.

Si definisce $\mathbf{X}_j$ come uno dei possibili sottoinsiemi contenenti j feature, dove $j \in \mathbb{N}$ e $j \in [1, m]$. Si definisce inoltre F come la funzione obiettivo (o di valutazione). Questo approccio può essere applicato solo se è valida la seguente **ipotesi di monotonia**:

$$X_1 \subseteq X_2 \subseteq \dots \subseteq X_m \implies F(X_1) \leq F(X_2) \leq \dots \leq F(X_m) \quad (3.5)$$

Aggiungere feature a un sottoinsieme non fa mai peggiorare (o non fa mai migliorare, a seconda del segno) il valore della funzione di valutazione.

A ogni passaggio di selezione, l'algoritmo valuta le feature rimuovendo quelle meno discriminanti e aggiornando il valore di F :

- Se il nuovo valore migliora (es. $F_{t+1} > F_t$), l'algoritmo continua a esplorare lo stesso percorso (ramo)
- Altrimenti, interrompe l'esplorazione di quel ramo ("bound") sapendo che non potrà portare a una soluzione migliore, e passa a esplorare un percorso alternativo

L'utilizzo logico di "<" o ">" per stabilire la condizione di bound dipende strettamente da come è definita la funzione obiettivo F (se si cerca un massimo o un minimo).

Supponiamo di avere un set iniziale di 4 feature e di voler estrarre le 2 più discriminanti. Una ricerca puramente esaustiva richiederebbe di testare esplicitamente $\binom{4}{2} = 6$ combinazioni. Il branch-and-bound utilizza la regola di monotonia per trovare le 2 feature ottimali esplorando meno di 6 combinazioni, potando i percorsi sub-ottimali.

3.3.4 Subset Evaluation Heuristic: Best First

La **Best First Search** è una strategia di ricerca euristica che mira a trovare un compromesso ottimale tra il costo computazionale e la qualità del sottoinsieme di feature individuato. Non esplora necessariamente l'ultimo nodo costruito, ma utilizza una struttura a coda per muoversi nel grafo delle combinazioni.

- **Approccio globale:** Il metodo esplora il subset più promettente all'interno di una coda di priorità, non limitandosi a seguire un unico ramo
- **Efficienza:** Evita l'esplorazione di tutti i $2^n - 1$ subset tipici della ricerca esaustiva
- **Scalabilità:** Mentre con 30 feature una ricerca esaustiva richiederebbe oltre 1 miliardo di combinazioni, la Best First Search permette di ottenere risultati in tempi ragionevoli

Consideriamo un obiettivo di classificazione per predire se il consumo elettrico di un edificio sarà **Alto (H)** o **Basso (L)**.

- **Feature disponibili:** X_1 (Temp. esterna), X_2 (Prod. fotovoltaica), X_3 (Giorno lavorativo), X_4 (N. occupanti)
- **Valutazione:** Si utilizza un *Decision Tree* con 5-fold cross-validation e lo score basato sull'Accuracy

1. **Inizializzazione:** Si parte dal subset vuoto $S_0 = \emptyset$
2. **Passo 1 (Valutazione singole feature):** Vengono testati i subset composti da una sola feature. I risultati generano una **coda di priorità ordinata**:
 - $\{X_2\} \rightarrow 0.78$ (Migliore)
 - $\{X_4\} \rightarrow 0.74$
 - $\{X_1\} \rightarrow 0.70$
 - $\{X_3\} \rightarrow 0.62$
3. **Passo 2 (Espansione del migliore):** Si espande $\{X_2\}$ aggiungendo una feature alla volta. I nuovi subset $\{X_2, X_4\}$, $\{X_2, X_1\}$ e $\{X_2, X_3\}$ vengono inseriti nella coda. Il subset $\{X_2, X_4\}$ con Accuracy 0.83 diventa il nuovo leader della coda. *Nota: $\{X_4\}$ e $\{X_1\}$ rimangono in lista come potenziali punti di ripartenza*
4. **Passo 4 (Progressione):** L'espansione di $\{X_2, X_4\}$ porta al subset $\{X_2, X_4, X_3\}$ con un'Accuracy di **0.84**.
5. **Passo 5 (Verifica e Stop):** L'algoritmo prova a espandere altri rami promettenti (es. partendo da $\{X_2, X_1\}$). Se dopo k espansioni nessuno dei nuovi subset supera il valore massimo di 0.84, il ramo viene abbandonato e l'algoritmo termina

Il subset ottimale selezionato è $\{X_2, X_4, X_3\}$.

3.3.5 Subset Evaluation Heuristic: SFS e SBS

Entrambi soffrono del **nesting effect** (effetto di annidamento): le decisioni prese ai passi precedenti sono definitive e non possono essere modificate.

Sequential Forward Selection (SFS) La SFS è un approccio di tipo **bottom-up**.

- **Inizializzazione:** Si parte da un sottoinsieme vuoto ($S_0 = \emptyset$)
- **Iterazione:** Ad ogni iterazione, si aggiunge la singola feature che produce il minimo errore di classificazione, o che migliora maggiormente le prestazioni
- **Nesting effect:** In SFS, una volta che una feature è stata aggiunta al sottoinsieme, non può più essere rimossa nei passaggi successivi
- **Costo computazionale:** È generalmente meno costosa della SBS, poiché la funzione obiettivo (ovvero l'addestramento del classificatore) viene valutata inizialmente in uno spazio a dimensionalità molto ridotta

Consideriamo le stesse feature (X_1, X_2, X_3, X_4) e l'Accuracy in cross-validation come metrica.

1. **Passo 1 (Singole feature):** Partendo da $S_0 = \emptyset$, si valutano le singole feature. $\{X_2\}$ ottiene lo score massimo (0.78), quindi aggiorniamo $S_1 = \{X_2\}$.
2. **Passo 2 (Aggiunta 2^a feature):** Aggiungendo una feature a $\{X_2\}$, il subset candidato migliore è $\{X_2, X_4\}$ con 0.83. Aggiorniamo $S_2 = \{X_2, X_4\}$.
3. **Passo 3 (Aggiunta 3^a feature):** Proviamo ad aggiungere a $\{X_2, X_4\}$. Il subset $\{X_2, X_4, X_3\}$ migliora le prestazioni a 0.84. Aggiorniamo $S_3 = \{X_2, X_4, X_3\}$.
4. **Passo 4 (Aggiunta 4^a feature e Stop):** Aggiungendo l'ultima feature, il subset $\{X_2, X_4, X_3, X_1\}$ ottiene 0.83. Poiché le prestazioni peggiorano rispetto al passo precedente, l'algoritmo si ferma.

Subset finale SFS: $\{X_2, X_4, X_3\}$. L'algoritmo ha valutato un solo percorso, senza considerare alternative e senza poter tornare indietro.

Sequential Backward Selection (SBS) La SBS è un approccio di tipo **top-down**.

- **Inizializzazione:** Si parte dal set contenente **tutte** le feature disponibili
- **Iterazione:** Ad ogni iterazione, si rimuove in modo incrementale la singola feature che degrada di meno le prestazioni (o che permette loro di aumentare). Questo permette anche di supervisionare direttamente la perdita di informazione

- **Nesting effect:** In SBS, una volta che una feature è stata scartata, non può più essere riutilizzata in seguito

Esempio Pratico SBS:

1. **Passo 1 (Prima rimozione):** Iniziamo dal subset pieno $S_0 = \{X_1, X_2, X_3, X_4\}$ che ha un'Accuracy di 0.83. Valutiamo i subset ottenuti rimuovendo una sola feature. Togliendo X_1 , il subset $\{X_2, X_3, X_4\}$ raggiunge 0.85 (le prestazioni aumentano). Aggiorniamo $S_1 = \{X_2, X_3, X_4\}$.
2. **Passo 2 (Seconda rimozione):** Proviamo a rimuovere una feature da S_1 . Togliendo X_4 , il subset $\{X_2, X_3\}$ schizza a 0.88. Aggiorniamo $S_2 = \{X_2, X_3\}$.
3. **Passo 3 (Terza rimozione e Stop):** Rimuovendo un'ulteriore feature da $\{X_2, X_3\}$, otteniamo i subset $\{X_3\}$ (0.62) e $\{X_2\}$ (0.78). Poiché entrambi i risultati sono peggiori dello 0.88 precedente, l'algoritmo termina.

Subset finale SBS: $\{X_2, X_3\}$. È interessante notare come l'approccio SFS e l'approccio SBS, partendo dagli stessi dati, abbiano portato a due risultati ottimali locali differenti

Floating Search Per superare il limite del **nesting effect** presente sia in SFS che in SBS, si introduce il **Floating search method**. Questo metodo consente di modificare dinamicamente il set di feature calcolato al passo precedente, permettendo l'aggiunta o la rimozione di feature già processate.

Confronto delle Strategie La scelta dell'algoritmo di ricerca più adatto dipende principalmente dal **numero di feature** a disposizione, dal **costo computazionale** richiesto per l'addestramento del modello e dalla **presenza di interazioni** tra le feature stesse.

La **Best First Search** è generalmente la scelta ideale per identificare il **modello finale**, specialmente quando si hanno risorse computazionali sufficienti. A differenza dei metodi puramente sequenziali, questo approccio riesce a evitare il **nesting effect** mantenendo più percorsi esplorativi aperti e correggendo eventuali decisioni sub-ottimali prese ai passi precedenti.

La **Sequential Forward Selection (SFS)** è lo strumento perfetto per costruire una **baseline** veloce o per effettuare un primo screening. Conviene utilizzarla su dataset poco rumorosi e con un numero limitato di feature, in particolare quando il modello è costoso da addestrare, il tempo a disposizione scarseggia e si ipotizza che le feature siano tra loro abbastanza indipendenti.

La **Sequential Backward Selection (SBS)** è l'approccio d'elezione per snellire e **ridurre modelli** originariamente **sovradimensionati**. Dà il meglio di sé quando si hanno a disposizione molte feature ridondanti e l'addestramento del modello non risulta essere eccessivamente oneroso. Partendo dal set completo, il suo principale vantaggio risiede nella **capacità di valutare fin da subito**

le interazioni globali tra tutte le variabili; è quindi utile se si parte dal presupposto che quasi tutte le feature iniziali siano importanti, limitandosi a scartare in modo progressivo solo l'informazione che si rivela superflua.

3.3.6 Bidirectional Elimination (Stepwise Bidirectional Selection)

L'eliminazione bidirezionale mira a combinare i punti di forza della Sequential Forward Selection (SFS) e della Sequential Backward Selection (SBS), superandone i limiti. L'idea è quella di **aggiungere e rimuovere feature in modo alternato**: questo approccio permette di correggere in itinere eventuali decisioni sub-ottimali o sbagliate prese nelle fasi iniziali dell'algoritmo.

L'algoritmo mantiene un sottoinsieme corrente di feature e , a ogni singola iterazione, esegue in sequenza due operazioni:

1. **Forward step:** Aggiunge al sottoinsieme la singola feature che garantisce il maggior incremento delle performance del modello
2. **Backward step:** Valuta le feature già presenti nel sottoinsieme e rimuove quella meno utile, o che degrada meno le prestazioni

Il processo iterativo continua fino a quando non si osservano più miglioramenti significativi oppure viene raggiunto un criterio di arresto prestabilito.

- **Pro:** Riduce drasticamente il nesting effect
- **Contro:** Essendo più flessibile e valutando più rami, è computazionalmente più oneroso rispetto alle singole strategie SFS o SBS

Esempio Pratico Riprendiamo l'esempio della predizione del consumo elettrico (Alto/Basso) valutata tramite un Decision Tree (5-fold CV, metrica Accuracy). Le feature sono X_1 (Temp. esterna), X_2 (Prod. fotovoltaica), X_3 (Giorno lavorativo), X_4 (N. occupanti).

- **Passo 1 (Forward):** Partendo da $S_0 = \emptyset$, si valutano i sottoinsiemi con una sola feature. $\{X_2\}$ ottiene lo score massimo (0.78), quindi viene selezionato e si aggiorna $S_1 = \{X_2\}$
- **Passo 2 (Backward):** Non essendo possibile scartare feature da un set che ne contiene solo una, questo passo non è applicabile e si procede
- **Passo 3 (Forward):** Si cerca la miglior feature da aggiungere a S_1 . Il subset candidato $\{X_2, X_4\}$ raggiunge uno score di 0.83. Lo selezioniamo: $S_2 = \{X_2, X_4\}$.
- **Passo 4 (Backward):** Proviamo a rimuovere una feature da $\{X_2, X_4\}$. Rimuovendo X_2 otteniamo $\{X_4\}$ (score 0.74); rimuovendo X_4 torniamo a $\{X_2\}$ (score 0.78). In entrambi i casi le prestazioni peggiorano rispetto allo 0.83 corrente, quindi non eliminiamo nulla e lo step si conclude senza alterare S_2

- **Passo 5 (Forward):** Aggiungiamo un'ulteriore feature. Supponiamo che aggiungendo X_1 , il subset $\{X_2, X_4, X_1\}$ raggiunga uno score di 0.85
- **Passo 6 (Backward):** Testiamo la rimozione dal nuovo subset $\{X_2, X_4, X_1\}$. Supponiamo di provare a scartare X_2 : ci ritroveremmo con il sottoinsieme $\{X_4, X_1\}$, che non avevamo mai esplorato in precedenza. Se, valutandolo, scopriremo che $\{X_4, X_1\}$ raggiunge uno score di 0.88 (superiore allo 0.85 di partenza), l'algoritmo confermerebbe l'eliminazione di X_2 , selezionando definitivamente questo nuovo subset per proseguire.

3.4 Embedded

I metodi Embedded incorporano la fase di feature selection direttamente all'interno del processo di addestramento del modello di apprendimento. Questi metodi si articolano in tre strategie principali:

- **Build-in:** Dispongono di un meccanismo nativo e integrato per effettuare la feature selection durante la loro stessa costruzione. (es. CART, ID3 e C4.5)
- **Pruning (Potatura):** L'approccio inizia addestrando un modello completo con tutte le feature a disposizione. Successivamente, tenta di rimuovere alcune feature azzerando (impostando a 0) i coefficienti o i pesi ad esse associati, assicurandosi però di mantenere inalterate le prestazioni del modello
- **Regularization (Regolarizzazione):** Questa tecnica utilizza funzioni obiettivo che svolgono una doppia azione: minimizzano l'errore di adattamento ai dati (fitting error) e, contemporaneamente, forzano i coefficienti del modello ad assumere valori piccoli

3.4.1 Embedded: Regularization-Regression Lasso

I metodi di regolarizzazione sono tecniche di tipo embedded che effettuano la selezione delle feature agendo direttamente sulla funzione di costo del modello durante la fase di training. Per comprendere queste tecniche, è necessario partire dai fondamenti della **Regressione Lineare**, il cui obiettivo è modellare la variabile target come combinazione lineare delle feature di input:

$$y_{pred} = a_0 + a_1x_1 + \dots + a_nx_n$$

I coefficienti a_i vengono solitamente stimati minimizzando la funzione di costo dell'Errore Quadratico Medio (MSE):

$$\sum_{i=1}^{N_{training}} (y_{real}^{(i)} - y_{pred}^{(i)})^2$$

La regressione lineare standard presenta il limite di essere sensibile alla **collinearità**, alla correlazione tra le feature. Per evitare ciò si può integrare alla PCA.

Ridge Regression (Penalità L_2) La Ridge regression modifica la funzione di costo originale introducendo un termine di penalità L_2 basato sul quadrato dei coefficienti:

$$\sum_{i=1}^{N_{\text{training}}} (y_{\text{real}}^{(i)} - y_{\text{pred}}^{(i)})^2 + \alpha \sum_{j=1}^n a_j^2$$

L'obiettivo è "restringere" (shrink) i valori dei coefficienti per disattivare le feature inutili e ridurre il rischio di overfitting.

- **Parametro α :** È un iperparametro che deve essere calibrato; se $\alpha = 0$ si torna alla regressione lineare standard, mentre se è troppo grande i risultati diventano inaffidabili
- **Scaling:** È indispensabile scalare le feature in anticipo affinché la penalità sia equa per tutti i coefficienti
- **Selezione:** Può portare i coefficienti di alcune feature a valori prossimi allo zero, applicando una sorta di selezione automatica

LASSO Regression (Penalità L_1) La LASSO (*Least Absolute Shrinkage and Selection Operator*) utilizza invece una penalità di tipo L_1 basata sul valore assoluto dei coefficienti:

$$\sum_{i=1}^{N_{\text{training}}} (y_{\text{real}}^{(i)} - y_{\text{pred}}^{(i)})^2 + \alpha \sum_{j=1}^n |a_j|$$

Questo approccio esegue una selezione delle feature molto più netta e potente rispetto alla Ridge:

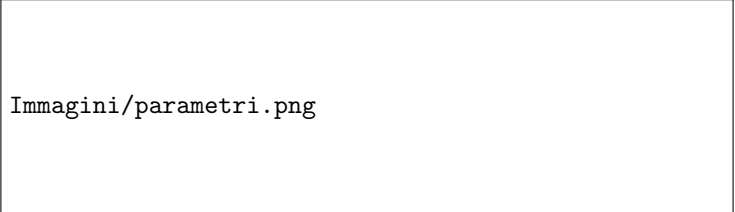
- **Correlazione:** Se due feature sono correlate, la loro presenza simultanea aumenta il costo, spingendo il modello a sceglierne solo una
- **Sparsità:** LASSO tende a ridurre esattamente a zero i coefficienti delle feature meno importanti
- **Eliminazione:** Le feature con coefficiente nullo vengono rimosse automaticamente dal set finale
- **Requisito:** Anche in questo caso è fondamentale scalare preventivamente le feature per renderle numericamente comparabili

4 Hyperparameter Tuning

Nel campo dell'IA e del Machine Learning, non è possibile determinare a priori quale sia il modello ottimale per un determinato set di dati; questa natura empirica porta spesso a critiche sulla mancanza di determinismo della disciplina. La scelta dell'algoritmo più efficace rimane quindi un processo iterativo basato

sulla sperimentazione piuttosto che su una previsione teorica assoluta. Ci si chiede in questo contesto quale sia il modello che **meglio rappresenta il dato noto**.

All'interno del processo di modellazione, assume un ruolo centrale l'individuazione della **configurazione ottimale del sistema**. Nella pipeline di un progetto di Machine Learning, questa fase si identifica con la **ricerca degli iperparametri (Hyperparameter Tuning)**, volta a massimizzare le prestazioni e la capacità di generalizzazione del modello.

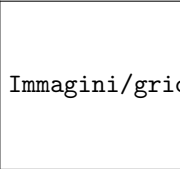


Immagini/parametri.png

Gli **iperparametri** sono i valori noti da scegliere a priori; i **parametri** sono quelle variabili che vengono ottimizzate durante il processo di addestramento.

4.1 Grid Search

La tecnica di *Grid Search* consiste nel definire uno spazio di ricerca discreto per un insieme di iperparametri $\{\theta_1, \theta_2, \dots, \theta_n\}$. Per ogni iperparametro θ_i , si specifica un insieme di valori possibili; l'algoritmo procede validando sistematicamente tutte le combinazioni risultanti dal prodotto cartesiano di tali insiemi.



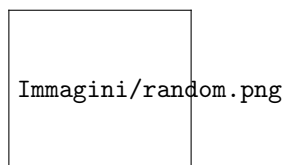
Immagini/grid.png

Sebbene garantisca l'esplorazione dell'intera griglia definita, questa tecnica presenta un costo computazionale estremamente elevato, che cresce esponenzialmente al crescere del numero di iperparametri (**curse of dimensionality**) e non fornisce una griglia di ricerca ottimale in partenza.

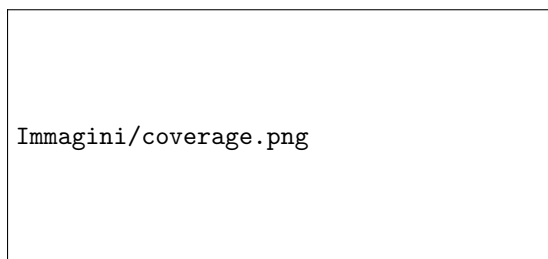
4.2 Random Search

Per ogni iperparametro non si definisce un insieme discreto di valori, bensì un **intervallo (range)** di esplorazione continuo o discreto. Durante il processo, l'algoritmo campiona **casualmente** le configurazioni da una distribuzione di probabilità definita su tale spazio. Questa strategia permette di esplorare un

numero maggiore di combinazioni uniche, risultando spesso più efficiente nel trovare l'ottimo globale con un dispendio computazionale inferiore. E' un algoritmo semplice e parallelizzabile e permette di settare in partenza un costo computazionale di calcolo.



La superiore efficacia della Random Search rispetto alla Grid Search è riconducibile alla diversa densità di campionamento sulle dimensioni rilevanti dello spazio di ricerca. In molti modelli di Machine Learning, la sensibilità rispetto agli iperparametri non è uniforme: solo un sottoinsieme ristretto di essi (**effective dimensionality**) influenza significativamente la funzione obiettivo. Mentre la **Grid Search** alloca risorse computazionali per testare ripetutamente gli stessi valori sulle dimensioni critiche, la **Random Search** esplora valori distinti ad ogni iterazione per ogni iperparametro. Ciò aumenta drasticamente la probabilità di campionare una configurazione prossima all'ottimo globale, specialmente quando quest'ultimo non coincide con i nodi predefiniti della griglia sistematica.



Il **coverage** è quanto viene coperto lo spazio di variazione del modello.

5 Data Cleaning

Spesso i dataset grezzi non sono immediatamente pronti per essere utilizzati da un sistema di apprendimento automatico. Durante la fase di raccolta o integrazione dei dati possono infatti emergere diverse criticità che rendono necessaria una fase di *pre-processing*. Tra i problemi più comuni si individuano:

- **Dati mancanti:** osservazioni o attributi non disponibili durante la raccolta dei dati. Questo problema viene generalmente affrontato tramite tecniche di *imputazione*

- **Dati inconsistenti:** presenza di valori errati, outlier oppure formati non omogenei tra le variabili. La gestione di tali problemi può richiedere metodi *analitici* oppure approcci basati sulla *conoscenza del dominio*
- **Dati non scalati:** feature caratterizzate da ordini di grandezza molto differenti tra loro. In questi casi è necessario applicare tecniche di normalizzazione o standardizzazione, come *Min-Max scaling* o *standardizzazione*

L'obiettivo del processo di *data cleaning* è quindi quello di trasformare il dataset originale in una rappresentazione coerente, completa e strutturata, pronta per essere utilizzata nelle fasi di addestramento dei modelli di apprendimento automatico.

5.1 Gestione dei valori mancanti

La presenza di valori mancanti rappresenta uno dei problemi più frequenti nei dataset reali. A seconda della natura del problema e della quantità di dati disponibili, è possibile adottare diverse strategie:

1. **Eliminazione della feature:** se una variabile presenta una percentuale molto elevata di valori mancanti (ad esempio superiore al 70-75%), essa può essere considerata poco informativa e quindi rimossa dal dataset
2. **Imputazione:** consiste nello stimare i valori mancanti utilizzando le informazioni disponibili nel dataset
3. **Utilizzo di modelli che gestiscono intrinsecamente i valori mancanti:** alcuni algoritmi di apprendimento automatico, come ad esempio gli alberi decisionali, sono in grado di gestire direttamente la presenza di valori mancanti durante la fase di training

5.2 Tecniche di imputazione

Le tecniche di imputazione permettono di stimare i valori mancanti all'interno di un dataset utilizzando informazioni statistiche o relazioni tra le variabili. In generale, tali tecniche possono essere suddivise in tre categorie principali.

- **Imputazione univariata**

In questo approccio ogni variabile viene considerata indipendentemente dalle altre. Quando un campione presenta un valore mancante per una determinata feature, tale valore viene stimato utilizzando esclusivamente le informazioni provenienti dagli altri campioni che possiedono un valore osservato per quella stessa variabile. Le tecniche più comuni includono:

- Sostituzione con una costante
- Imputazione tramite **media**
- Imputazione tramite **mediana**

- Imputazione tramite **moda**

Vantaggi

- Metodo semplice e computazionalmente efficiente
- Particolarmente utile quando il dataset è di piccole dimensioni
- Evita la perdita di dati dovuta alla rimozione di righe o colonne

Svantaggi

- Introduce approssimazioni che possono generare *bias* e aumentare la varianza delle stime
- Non sfrutta le relazioni tra le variabili del dataset
- Generalmente meno efficace rispetto a metodi di imputazione multipla o più complessi

- **Imputazione multivariata** Questo approccio utilizza contemporaneamente più variabili del dataset per stimare i valori mancanti. L'idea di base consiste nel modellare una variabile con valori mancanti come funzione delle altre variabili disponibili. Un metodo comune consiste nell'utilizzare modelli di **regressione**: in questo caso le feature complete vengono utilizzate come variabili esplicative (X), mentre la variabile con valori mancanti rappresenta la variabile da stimare (y). Come vantaggi il metodo:

- Sfrutta le relazioni tra le variabili del dataset
- Può produrre stime più accurate rispetto ai metodi univariati
- Consente di ottenere stime meno distorte dei parametri del modello

Svantaggi

- Richiede maggiore complessità computazionale
- Può introdurre bias se il modello utilizzato non rappresenta correttamente le relazioni tra le variabili
- Le stime ottenute rappresentano comunque una proxy dei valori reali

- **Imputazione basata su modelli predittivi** I valori mancanti vengono stimati tramite modelli di apprendimento automatico addestrati sui dati completi. Il modello viene prima addestrato utilizzando i campioni per cui tutte le feature sono disponibili e successivamente utilizzato per predire i valori mancanti. Tra gli approcci più utilizzati rientrano:

- Regressione
- Alberi decisionali
- Metodi basati su *k-nearest neighbors*
- Tecniche di imputazione multipla

Questo approccio può fornire stime più accurate, ma richiede una maggiore complessità computazionale e una corretta scelta del modello predittivo.

5.2.1 Imputazione tramite k-Nearest Neighbours

Un metodo di imputazione frequentemente utilizzato è basato sull'algoritmo k-Nearest Neighbours (kNN). Questo approccio sfrutta il principio della similarità tra i campioni del dataset, misurata tipicamente tramite una metrica di distanza, come la distanza euclidea. L'idea alla base del metodo consiste nell'individuare i k campioni più simili (i k vicini più prossimi) rispetto all'osservazione che presenta il valore mancante. Il valore della feature mancante viene quindi stimato utilizzando le informazioni provenienti da tali campioni. Per ogni valore mancante:

- Vengono identificati i k campioni più vicini rispetto al campione incompleto
- Tra questi vengono considerati solo quelli che possiedono un valore osservato per la feature da imputare
- Il valore mancante viene stimato come media dei valori dei vicini oppure come media pesata in funzione della distanza

Nel caso in cui un campione presenti più feature mancanti, l'insieme dei vicini più prossimi può variare a seconda della variabile che si sta imputando. Come vantaggi il metodo:

- Sfrutta le relazioni tra i campioni del dataset
- Può fornire stime più accurate rispetto ai metodi univariati
- Non richiede l'assunzione di un modello statistico esplicito

Ma ha un costo pesante in **inferenza**, perché calcola la distanza di un determinato campione con tutto il dataset, per tutti i campioni che hanno un dato mancante. Inoltre richiede la necessità della definizione di iperparametri, quali il k e il metodo per calcolare la distanza.

5.3 Cleaning di valori non corretti

Abbiamo affermato che in questa problematica esistono tecniche analitiche, tecniche basate sulla conoscenza del dominio e tecniche basate sugli outliers. La gestione di record **errati** o **inconsistenti** si articola in tre approcci principali:

- **Inconsistency detection (Rilevamento delle inconsistenze):** Questa tecnica mira a identificare discrepanze strutturali o di formato all'interno dello stesso campo tra record diversi.
 - *Esempio:* In un dataset, il campo "name" non può contenere in un record il nome completo (es. "Mario Rossi") e in un altro solo l'iniziale (es. "M.").
- **Domain knowledge (Conoscenza del dominio):** Sfrutta la conoscenza specifica del contesto per validare i dati. Questo include:

- **Data scrubbing:** La pulizia dei dati attraverso regole predefinite
- **Data auditing:** L’ispezione dei dati per verificare la conformità a regole logiche
- *Esempio:* Se in un dataset il campo “country” è Italy, mentre il campo “city” è London, siamo di fronte a un’inconsistenza logica rilevabile solo tramite conoscenza geografica (di dominio)
- **Data-centric methods (Metodi basati sui dati):** Si utilizzano approcci statistici per identificare anomalie.
 - **Outlier detection:** Identificazione di valori che deviano significativamente dalla distribuzione
 - Gli outliers devono essere ispezionati manualmente prima di essere eventualmente scartati, poiché possono contenere informazioni interessanti o evidenziare fenomeni rari ma reali.

5.4 Scaling e Normalizzazione

Il problema dello scaling sorge quando le diverse feature di un dataset presentano scale di riferimento molto distanti tra loro (ad esempio, l’età rispetto allo stipendio annuo). Se i dati non vengono normalizzati, le feature con valori numerici più grandi domineranno impropriamente il calcolo delle funzioni aggregate, come la *distanza Euclidea*, portando il modello a ignorare implicitamente le variabili con range minori. Esistono due approcci principali per risolvere questo problema:

5.4.1 Standardizzazione (Z-score)

La standardizzazione trasforma i dati in modo che abbiano media nulla ($\mu = 0$) e varianza unitaria ($\sigma^2 = 1$). La formula per calcolare il valore standardizzato z_i^j (per il record i dell’attributo j) è:

$$z_i^j = \frac{x_i^j - \mu_j}{\sigma_j} \quad (5.1)$$

Dove:

- x_i^j : valore del j -esimo attributo nell’ i -esimo record
- μ_j : media del j -esimo attributo
- σ_j : deviazione standard del j -esimo attributo

Sotto l’assunzione di una distribuzione normale, la maggioranza dei valori normalizzati cade tipicamente nell’intervallo $[-3, 3]$.

5.4.2 Min-Max Scaling

Questa tecnica riscalda i dati in un intervallo fisso $[0, 1]$ o $[-1, 1]$. Il valore trasformato y_i^j si ottiene come:

$$y_i^j = \frac{x_i^j - \min_j}{\max_j - \min_j} \quad (5.2)$$

Dove:

- $\min_j$: valore minimo dell'attributo j
- $\max_j$: valore massimo dell'attributo j

Il Min-Max scaling non è efficace in presenza di **outlier** estremi causati da errori nella raccolta dati, poiché questi espandono artificialmente il denominatore, comprimendo tutti gli altri valori in un intervallo piccolissimo.

È fondamentale che le operazioni di *data cleaning* e *scaling* vengano applicate esclusivamente **dopo** la fase di validazione (es. *split holdout* 70/30). Questo approccio garantisce l'integrità del processo di valutazione ed evita il fenomeno del **Data Leakage**.

- **Procedura di Scaling:** I parametri di trasformazione (come il minimo e il massimo nel *Min-Max Scaling*, o media e deviazione standard nella *Standardizzazione*) devono essere calcolati unicamente sul **training set**
- **Applicazione al Test Set:** Il test set deve essere scalato utilizzando i parametri derivati dal training. In questo modo, il modello valuta i dati mantenendo la prospettiva della distribuzione su cui è stato addestrato

L'applicazione dei parametri di training al test set nel *Min-Max Scaling* può portare a valori esterni all'intervallo $[0, 1]$ se il test set contiene valori inferiori al minimo o superiori al massimo osservati nel training.

Per mitigare la rigidità degli intervalli fissi e gestire meglio valori mai visti precedentemente (outlier o nuovi estremi), si predilige spesso la **Standardizzazione**. Essa non vincola i dati a un intervallo limitato, rendendo il modello più robusto rispetto a dati del test set che presentano una varianza leggermente diversa da quella del training.

6 Data Reduction and Transformation

L'obiettivo delle tecniche di data reduction e data transformation è quello di ridurre la complessità del dataset mantenendo, per quanto possibile, le informazioni più rilevanti per il processo di apprendimento. Tali tecniche permettono di ottenere una rappresentazione più compatta dei dati, preservando il potere discriminante delle variabili e migliorando l'efficienza computazionale dei modelli di apprendimento automatico.

Le strategie di riduzione dei dati possono essere applicate principalmente lungo due direzioni:

1. **Riduzione lungo le feature:** diminuzione del numero di variabili del dataset, mantenendo solo le più informative oppure combinandole in rappresentazioni a dimensionalità ridotta
2. **Riduzione lungo i campioni:** riduzione del numero di osservazioni presenti nel dataset, tramite tecniche di campionamento o aggregazione dei dati

Ogni processo di riduzione comporta inevitabilmente una **perdita di informazione**. L'obiettivo è quindi quello di ridurre la dimensionalità del dataset minimizzando tale perdita e preservando il più possibile la struttura informativa dei dati.

Le principali tecniche di *data reduction* possono essere suddivise nelle seguenti categorie:

- **Data sampling:** riduzione del numero di campioni presenti nel dataset selezionando un **sottoinsieme rappresentativo delle osservazioni originali**, mantenendo la distribuzione statistica dei dati riducendo la dimensione del dataset
- **Feature selection:** riduzione del numero di **variabili** mantenendo le più informative
- **Data reduction con rotazione degli assi:** trasformazione dello spazio delle feature originale in uno spazio di dimensione ridotta attraverso una rotazione degli assi
- **Data reduction tramite trasformazione delle variabili:** i dati vengono trasformati attraverso funzioni matematiche con lo scopo di ridurre la complessità o migliorare la rappresentazione delle informazioni

6.1 Data Sampling per Dati Statici

Il campionamento (*sampling*) è il processo di selezione di un sottoinsieme di dati da un dataset più ampio. L'ipotesi di base per i dati statici è che l'intero dataset sia disponibile a priori e che il numero totale di punti n sia noto.

6.1.1 Unbiased Sampling (Campionamento non polarizzato)

Si seleziona una **frazione** predefinita f dei punti totali. Esistono due varianti principali:

- **Sampling without replacement (Senza reinserimento):** Da un dataset D con n record, vengono estratti casualmente $\lceil n \cdot f \rceil$ record senza includere duplicati

- **Sampling with replacement (Con reinserimento):** I record vengono estratti in modo indipendente per un totale di $\lceil n \cdot f \rceil$ volte; in questo caso, lo stesso record può comparire più volte nel campione

6.1.2 Biased Sampling (Campionamento polarizzato)

Alcune parti del dataset vengono enfatizzate intenzionalmente per la loro importanza strategica nell'analisi.

- **Temporal-decay bias:** I record più recenti hanno una probabilità maggiore di essere inclusi rispetto a quelli più vecchi. Si utilizza una funzione di decadimento esponenziale regolata da un parametro λ :

$$\text{Prob(selezione)} \propto e^{-\lambda t} \quad (6.1)$$

- **Stratified sampling (Campionamento stratificato):** Utile quando alcune classi di dati sono rare ma fondamentali. Il dataset è diviso in partizioni e i campioni vengono estratti indipendentemente da ciascuno strato seguendo proporzioni specifiche
- **Density-biased sampling:** I punti nelle regioni ad alta densità ricevono un peso minore, garantendo una rappresentanza adeguata delle regioni più rare o sparse del dataset

6.2 Data Sampling per Data Streams

A differenza dei dati statici, i **data streams** sono flussi continui in cui il numero totale di punti n aumenta costantemente. Il metodo più noto è il **Reservoir Sampling**, che permette di mantenere dinamicamente un campione rappresentativo di dimensione fissa k .

6.2.1 Algoritmo di Reservoir Sampling (*serbatoio*)

L'obiettivo è garantire che, in ogni istante, ogni elemento arrivato fino a quel momento abbia la stessa probabilità di far parte del campione. L'algoritmo opera come segue:

1. **Inizializzazione:** I primi k punti del flusso vengono inseriti direttamente nel "reservoir" (**serbatoio**)
2. **Gestione del flusso:** Per ogni nuovo punto n -esimo in arrivo ($n > k$), vengono applicate due decisioni di controllo:
 - Il nuovo punto viene inserito nel serbatoio con una probabilità pari a $P = k/n$
 - Se il punto viene selezionato per l'inserimento, un elemento già presente nel serbatoio viene scelto casualmente e sostituito dal nuovo arrivato per mantenere la dimensione k costante

Il valore di questo algoritmo risiede nel seguente lemma:

Dopo che sono arrivati n punti dal flusso, la probabilità che un qualsiasi punto (sia esso il primo o l'ultimo arrivato) sia incluso nel serbatoio è identica e pari a:

$$\text{Prob}(\text{inclusione}) = \frac{k}{n} \quad (6.2)$$

6.3 Data reduction with type transformation

La riduzione dei dati può essere combinata con una *type transformation*, il processo di **conversione dei dati da una rappresentazione complessa a una più semplice**, preservando le informazioni più rilevanti.

6.3.1 Haar Wavelet Transform

La trasformata wavelet di Haar è una tecnica utilizzata per la **decomposizione multigranulare** e la rappresentazione compatta di serie temporali. Consente di esprimere una serie come combinazione di **funzioni base (wavelet) pesate da coefficienti**.

Caratteristiche dei coefficienti Ogni coefficiente rappresenta la variazione della serie temporale tra due sotto-intervalli contigui.

- **Vettore di base wavelet:** rappresenta l'intervallo temporale associato alla variazione ed è modellato come una *step function* (funzione a gradino).
- **Ordine dei coefficienti:** i coefficienti sono organizzati gerarchicamente in base alla dimensione dell'intervallo analizzato:
 - **Coefficienti di ordine superiore:** descrivono l'andamento globale della serie (intervalli ampi)
 - **Coefficienti di ordine inferiore:** catturano dettagli locali e variazioni fini (intervalli piccoli)

Questa struttura gerarchica permette di effettuare una **riduzione dei dati** eliminando i coefficienti meno significativi, mantenendo una buona approssimazione del segnale originale.

Esempio Consideriamo una serie temporale di lunghezza $q = 8$:

$$S = (8, 6, 2, 3, 4, 6, 6, 5)$$

L'algoritmo calcola ricorsivamente:

- le **medie** (Φ), che rappresentano l'approssimazione
- i **coefficienti di dettaglio** (ψ), che catturano le differenze locali

1. **Livello** $k = 3$ (massimo dettaglio)

- Medie:

$$\frac{8+6}{2} = 7, \quad \frac{2+3}{2} = 2.5, \quad \frac{4+6}{2} = 5, \quad \frac{6+5}{2} = 5.5$$

- Coefficienti:

$$\psi_{k=3} = (1, -0.5, -1, 0.5)$$

2. **Livello** $k = 2$

- Medie:

$$\frac{7+2.5}{2} = 4.75, \quad \frac{5+5.5}{2} = 5.25$$

- Coefficienti:

$$\psi_{k=2} = (2.25, -0.25)$$

3. **Livello** $k = 1$

- Media globale:

$$\Phi_{\text{global}} = 5$$

- Coefficiente:

$$\psi_{k=1} = -0.25$$

Il vettore finale dei coefficienti è:

$$\{5, -0.25, 2.25, -0.25, 1, -0.5, -1, 0.5\}$$

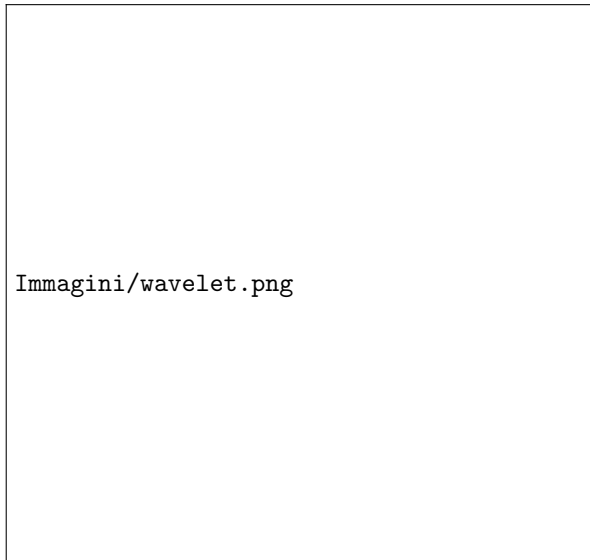


Figura 6.1: Illustrazione della decomposizione

Ricostruzione del segnale Per ricostruire un valore della serie originale, si parte dalla media globale e si attraversa l'albero dei coefficienti:

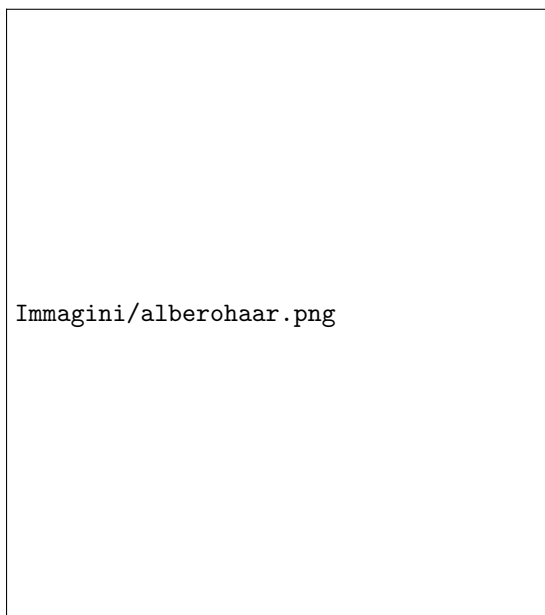
- Si **aggiunge** il coefficiente seguendo il ramo sinistro
- Si **sottrae** il coefficiente seguendo il ramo destro

Esempio: primo elemento ($x_1 = 8$)

$$5 + (-0.25) + 2.25 + 1 = 8$$

Esempio: secondo elemento ($x_2 = 6$)

$$5 + (-0.25) + 2.25 - 1 = 6$$



Compressione Per confrontare l'importanza dei coefficienti, questi vengono **normalizzati** moltiplicandoli per $\sqrt{L}$, con L lunghezza dell'intervallo coperto:

- $\psi_{k=1}$ (copre 8 valori):
 $-0.25 \cdot \sqrt{8} \approx -0.707$
- $\psi_{k=2}$ (copre 4 valori):
 $2.25 \cdot \sqrt{4} = 4.5$
- $\psi_{k=3}$ (copre 2 valori):
 $1 \cdot \sqrt{2} \approx 1.414$

Mantenendo solo i coefficienti con valore normalizzato maggiore (ad esempio 4.5), si ottiene una **buona approssimazione della serie** utilizzando un numero ridotto di dati, minimizzando la perdita di informazione (energia del segnale).

7 Reti Neurali

Le reti neurali artificiali nascono con l'obiettivo di imitare, almeno in parte, il funzionamento del cervello umano e i suoi meccanismi di apprendimento. Uno dei primi modelli di neurone artificiale è quello proposto da McCulloch e Pitts nei primi anni '40, che rappresenta ancora oggi una base teorica fondamentale.

Nel corso del tempo sono stati sviluppati modelli più avanzati, tra cui i neuroni *hebbiani*, il cui sviluppo è legato al contributo dello psicologo canadese Donald Olding Hebb. Il principio di apprendimento hebbiano si basa sull'idea che la connessione tra due neuroni si rafforzi quando essi si attivano simultaneamente.

Un **neurone artificiale** è un'unità di elaborazione dell'informazione che costituisce l'elemento base di una rete neurale. Esso riceve segnali in ingresso, li elabora attraverso una combinazione pesata e produce un'uscita mediante una funzione di attivazione, generalmente non lineare. Il diagramma a blocchi mostrato in Figura 7.1 rappresenta il modello matematico di un neurone non lineare:

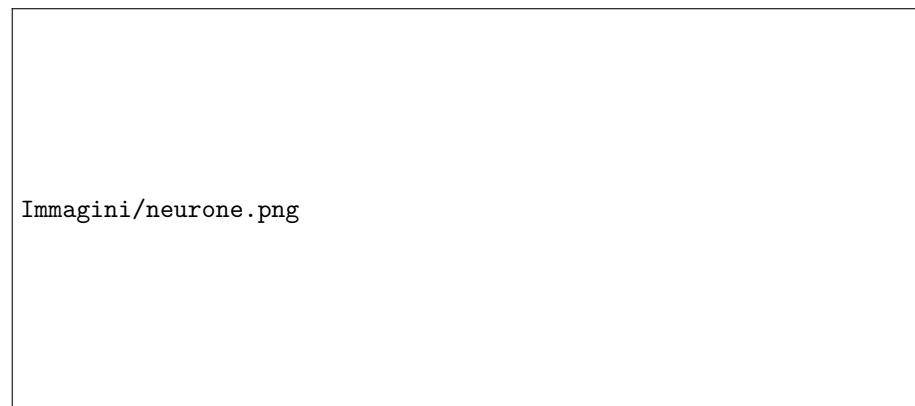


Figura 7.1: Modello non-lineare di un neurone artificiale

Il vettore delle feature x appartiene allo spazio:

$$\underline{x} \in \mathbb{R}^p$$

L'ingresso può essere quindi solo un numero **reale**, la feature non può essere categorica o una stringa. In Figura 7.1 si identificano i seguenti elementi del modello:

- insieme di **sinapsi o connessioni** ognuno caratterizzato da un peso w_{kj} . Si noti che l'input x_j ($j = 1, \dots, p$) alla sinapsi j connessa al k -esimo neurone è moltiplicata per il peso w_{kj} . Se il peso è positivo, la sinapsi ha un comportamento eccitatorio, altrimenti il comportamento è inibitorio.
- un **sommatore**, che somma i segnali in ingresso pesati dai pesi delle sinapsi, effettuando una combinazione lineare. Formalmente:

$$u_k = \sum_{j=1}^p x_j w_{kj} \quad (7.1)$$

- una **soglia** θ_k , che diminuisce l'input della funzione di attivazione. Utilizzando un valore negativo, la soglia è comunemente chiamata *bias* ed ha l'effetto di aumentare il valore in ingresso della funzione di attivazione

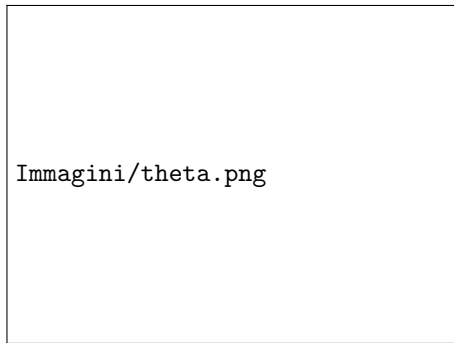


Figura 7.2: Trasformazione affine determinata dai differenti valori di θ_k

Valori di θ_k positivi o negativi hanno lo scopo di aumentare o diminuire l'**eccitazione** del neurone. Della retta in Figura 7.2, i pesi ne cambiano l'inclinazione, θ_k l'intercetta.

- la **funzione di attivazione** $\psi(\cdot)$ ha lo scopo di limitare l'ampiezza della risposta del neurone. Normalmente essa normalizza l'uscita nell'intervallo $[0; 1]$ o $[-1; 1]$. L'**uscita del neurone** si ottiene come segue:

$$y_k = \psi\left(\sum_{j=1}^p x_j w_{kj} - \theta_k\right) \quad (7.2)$$

$$y_k = \psi(u_k - \theta_k)$$

Dove la quantità $(u_k - \theta_k)$ è nota come **attività del neurone** v_k .

I parametri che il modello di apprendimento può imparare sono i **pesi** w_{kj} e la **soglia** θ_k . Essendo quindi θ_k un parametro esterno al neurone, lo si può inserire

concettualmente fra i segnali x_j , ottenendo il modello equivalente in Figura 7.3, dove è stato aggiunto x_0 con peso $w_{k0} = \theta_k$:



Figura 7.3: Modello non lineare di un neurone artificiale avendo integrato la soglia θ fra gli ingressi

Nel modello iniziale valeva:

$$v_k = \sum_{j=1}^p x_j w_{kj} - \theta_k \quad ,$$

mentre ora si può scrivere (supponendo $x_0 = -1$):

$$v_k = \sum_{j=0}^p x_j w_{kj} = -1 \cdot \theta_k + \sum_{j=1}^p x_j w_{kj}$$

7.1 Funzioni di attivazione

Nel caso in cui la funzione di attivazione ψ non sia presente, ovvero sia equivalente a una **funzione identità**, il neurone prende il nome di **neurone ADALINE** (Adaptive Linear Neuron), anziché **percettrone**. In questa configurazione, l'uscita y del neurone non è limitata e può assumere valori nell'intervallo $(-\infty, +\infty)$. La funzione di attivazione $\psi(\cdot)$ svolge un ruolo fondamentale, essa definisce l'uscita del neurone in funzione del potenziale di attivazione indotto v_k . Esistono diverse possibili scelte per la funzione di attivazione ψ , ciascuna con caratteristiche e ambiti di utilizzo specifici:

1. **Gradino**: nota anche come funzione di segno, si ha:

$$\psi(v_k) = \begin{cases} 1 & v_k \geq 0 \\ 0 & v_k < 0 \end{cases} \quad (7.3)$$

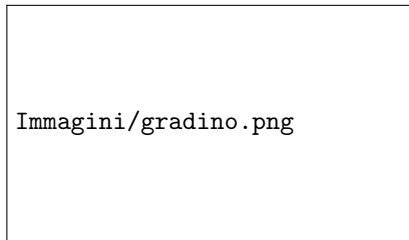


Figura 7.4: Funzione gradino o segno

Ne segue che l'output del neurone è:

$$y_k = \begin{cases} 1 & v_k \geq 0 \\ 0 & v_k < 0 \end{cases} \quad (7.4)$$

Il gradino si può anche esprimere con un'uscita che varia tra $[-1; 1]$:

$$\psi(v_k) \begin{cases} 1 & v_k \geq 0 \\ -1 & v_k < 0 \end{cases} \quad y_k = \begin{cases} 1 & v_k \geq 0 \\ -1 & v_k < 0 \end{cases}$$

2. **Funzione sigmoidea:** è sicuramente la più utilizzata e si caratterizza per essere crescente e sempre derivabile (a differenza del *gradino* e della *funzione a tratti*), limitata inferiormente e superiormente.

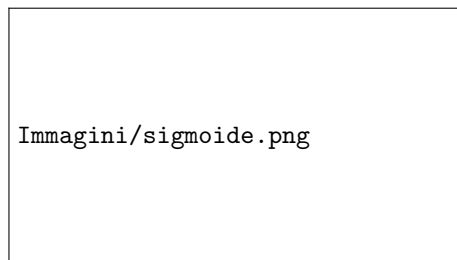


Figura 7.5: Funzione sigmoide

Definita come segue:

$$\psi(v_k) = \frac{1}{1 + e^{-av_k}} \quad (7.5)$$

dove il parametro a determina la pendenza della curva, in quanto si lega al coefficiente angolare della retta tangente nell'origine.

$$\psi'(v_k) = \frac{a}{(1 + e^{-av_k})^2} \quad ,$$

che dipende solo dalla pendenza:

$$\psi'(0) = \frac{a}{4}$$

Dunque per $a \rightarrow \infty$ si ha che $\psi'(0) \rightarrow \infty$ ottenendo la funzione gradino. La sigmoide varia tra $[0; 1]$, ne esiste una variante in cui la variabilità è tra $[-1; 1]$, ottenendo la **tangente iperbolica**:

$$\psi(v_k) = \tanh\left(\frac{v_k}{2}\right) = \frac{1 - e^{-av_k}}{1 + e^{-av_k}} \quad (7.6)$$

Il coefficiente angolare della retta tangente dipende dal parametro a . Infatti:

$$\psi(v_k) = \frac{2ae^{-av_k}}{(1 + e^{-av_k})^2}$$

che nell'origine vale:

$$\psi'(0) = \frac{a}{2}$$

7.2 Processi di apprendimento

Il processo di apprendimento consiste nell'adattamento dei parametri liberi di una rete neurale attraverso un'interazione continua con l'ambiente esterno. La rete modifica il proprio comportamento sulla base degli stimoli ricevuti, con l'obiettivo di migliorare le proprie prestazioni.

La tipologia di apprendimento dipende dalle modalità con cui avviene l'aggiornamento di tali parametri. In generale, il processo può essere descritto attraverso i seguenti passaggi:

1. **Stimolazione:** la rete riceve in ingresso un campione proveniente dal set di addestramento, fornito dall'ambiente esterno
2. **Aggiornamento dei parametri:** i pesi sinaptici e le soglie di attivazione vengono modificati in funzione dello stimolo ricevuto e della risposta prodotta dalla rete
3. **Risposta della rete:** la rete genera una nuova uscita in risposta allo stesso ingresso, sulla base dei parametri aggiornati

Il processo ha inizio assegnando valori iniziali (**tipicamente casuali**) ai pesi della rete. Dato un ingresso x , questo viene propagato attraverso il neurone mediante la combinazione pesata (somma pesata), a cui viene poi applicata la funzione di attivazione ψ , ottenendo così l'uscita y .

A questo punto si calcola l'errore relativo al singolo campione n , definito come:

$$e_k(n) = d_k(n) - y_k(n) \quad , \quad (7.7)$$

dove $d(n)$ rappresenta il valore desiderato (*ground truth*, nel caso di apprendimento supervisionato), mentre $y(n)$ è l'uscita prodotta dalla rete.

L'errore così ottenuto viene utilizzato per aggiornare i pesi della rete. In particolare, il peso k -esimo al passo successivo è dato da:

$$w_{kj}(n+1) = w_{kj}(n) + \Delta w_{kj}(n) \quad (7.8)$$

dove $w_{kj}(n)$ indica il valore del peso all'istante n , mentre $\Delta w_{kj}(n)$ rappresenta la variazione del peso calcolata sulla base dell'errore e dello stimolo ricevuto.

Il processo di apprendimento dipende quindi dal modo in cui viene determinata la quantità $\Delta w_{kj}(n)$: le diverse modalità di calcolo prendono il nome di **algoritmi di apprendimento**, e ne esistono diverse varianti.

7.2.1 Error-correcting learning

La soluzione pervenuta per questo particolare algoritmo di apprendimento mostra che l'aggiustamento $\Delta w_{kj}(n)$ ha la seguente forma:

$$\Delta w_{kj}(n) = \eta e_k(n) x_j(n) \quad (7.9)$$

dove $\eta \in [0; 1]$ è una costante positiva detta **learning rate** o tasso di apprendimento da un passo di apprendimento al successivo.

Nel caso si volesse considerare l'**apprendimento hebbiano**, la correzione ha questa forma:

$$\Delta w_{kj}(n) = \eta y_k(n) x_j(n) \quad (7.10)$$

Nel modello ADALINE, l'uscita del neurone coincide con il potenziale di attivazione, ovvero:

$$y = \psi(v) = v = \sum_{i=0}^p x_i w_i$$

La **frontiera di decisione** è la superficie che, nello spazio delle feature, separa le diverse regioni di decisione associate alle classi. Un neurone di tipo ADALINE realizza una **frontiera lineare**. In particolare, nel caso semplice in cui $x \in \mathbb{R}^2$, la frontiera di decisione è definita imponendo:

$$y = 0$$

Sostituendo l'espressione di y , si ottiene:

$$\sum_{i=0}^p x_i w_i = 0$$

Esplicitando il termine di soglia θ (associato al peso w_0), si può riscrivere:

$$-\theta + w_1 x_1 + w_2 x_2 = 0$$

Tale equazione rappresenta una retta nel piano (x_1, x_2) , che separa lo spazio in due semipiani.

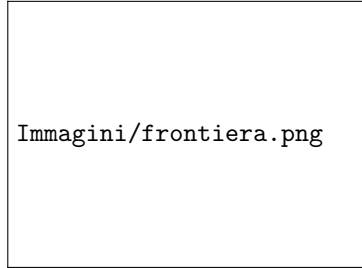


Figura 7.6: Rappresentazione della **frontiera lineare** nel caso $x \in \mathbb{R}^2$

Si definisce **epoca di apprendimento** il processo in cui tutti i campioni del dataset di addestramento vengono presentati una volta alla rete neurale. Durante un'epoca, ciascun campione viene utilizzato per calcolare l'uscita della rete, l'eventuale errore (nel caso di apprendimento *supervisionato*) e l'aggiornamento dei parametri. Al termine dell'epoca, la rete ha quindi effettuato un aggiornamento complessivo basato sull'intero dataset.

In generale, una singola epoca non è sufficiente per ottenere buone prestazioni: per questo motivo, il processo di addestramento viene ripetuto per più epoche, fino al raggiungimento di un criterio di arresto (ad es. un errore sufficientemente piccolo o un numero massimo di epoche).

7.3 La soluzione dell'AND logico

Si considera la tavola di verità della congiunzione logica (AND), dove i due input sono indicati con x_1 ed x_2 rappresentando lo spazio delle features:

$$x = [x_1, x_2]^\top$$

x_1	x_2	AND
0	0	0
0	1	0
1	0	0
1	1	1

Tabella 1: Tabella di verità della funzione AND

Si hanno a disposizione quattro campioni possibili:

$$x_1 = [0; 0]^\top \text{ e } y_1 = 0$$

$$x_2 = [1; 0]^\top \text{ e } y_2 = 0$$

$$x_3 = [0; 1]^\top \text{ e } y_3 = 0$$

$$x_4 = [1; 1]^\top \text{ e } y_4 = 1$$

Ricordando:

$$\sum_{i=0}^p x_i w_{ki} - \theta = 0$$
$$w_1 x_1 + w_2 x_2 - \theta = 0$$

Si può riscrivere:

$$\underline{w}^\top \underline{x} - \theta = 0$$

Scegliendo $\underline{w} = [2; 2]^\top$ si ha:

$$2x_1 + 2x_2 - \theta = 0$$

A questo punto si determina il valore di θ .

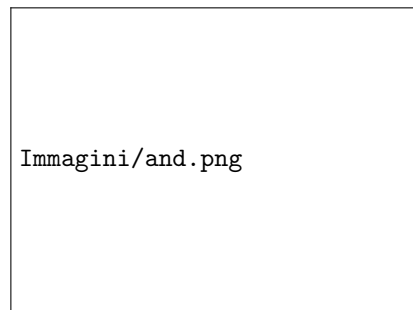


Figura 7.7: Soluzione del problema dell'AND

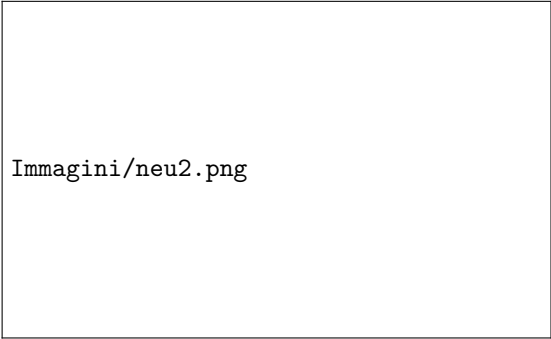
Con riferimento alla Figura 7.7 si prenda in considerazione il punto $A = (0.75, 0.75)$ e si imponga il passaggio della frontiera per questo punto:

$$2\frac{3}{4} + 2\frac{3}{4} - \theta = 0$$
$$\theta = \frac{12}{4} = 3$$

A questo punto la rete è addestrata.

7.4 Teorema di convergenza del percettrone

Questa sezione presenta il teorema di convergenza del percettrone nel caso di classificazione binaria, basato su un meccanismo di correzione iterativa degli errori. A tal fine si fa riferimento allo schema del neurone mostrato in Figura 7.8.



Immagini/neu2.png

Figura 7.8: Rappresentazione di un neurone secondo il modello di McCulloch-Pitts

Il teorema afferma che, se il problema è linearmente separabile, l'algoritmo è in grado di trovare un insieme di pesi che separa correttamente le classi in un numero finito di iterazioni.

Esistono diversi algoritmi per i quali è possibile dimostrare proprietà di convergenza verso una soluzione. Tra i più rilevanti si possono citare:

1. Il percettrone
2. Le *Support Vector Machine* (SVM)
3. L'algoritmo *K-Means*

Tuttavia, tali proprietà non si estendono automaticamente a strutture più complesse. In particolare, considerando due percettroni disposti in cascata (l'uscita del primo costituisce l'ingresso del secondo), non è più garantita la convergenza dell'algoritmo. La combinazione in serie di più unità computazionali introduce non linearità e complessità tali da rendere non più applicabile il teorema di convergenza nella sua forma originale.

Consideriamo un neurone di tipo ADALINE, la cui uscita è pari a:

$$y(n) = \underline{w}^\top(n) \underline{x}(n) \quad ,$$

dove:

$$\underline{x}(n) = [-1, x_1(n), x_2(n), \dots, x_p(n)]^\top \quad ,$$

è il vettore degli ingressi (incluso il termine di soglia), e

$$\underline{w}(n) = [\theta, w_1(n), w_2(n), \dots, w_p(n)]^\top \quad ,$$

è il vettore dei pesi.

La frontiera di decisione è definita dalla relazione:

$$\underline{w}^\top(n) \underline{x}(n) = 0 \quad ,$$

che rappresenta un iperpiano nello spazio delle feature.

Consideriamo ora un problema di classificazione binaria con due classi:

$$\Omega = \{c_1, c_2\}$$

Si assume che le due classi siano **linearmente separabili**. Introduciamo quindi i seguenti insiemi:

$$\Gamma_1 = \{\underline{x}_1(1), \underline{x}_1(2), \dots\}$$

$$\Gamma_2 = \{\underline{x}_2(1), \underline{x}_2(2), \dots\}$$

dove Γ_1 contiene i campioni appartenenti alla classe c_1 e Γ_2 quelli appartenenti alla classe c_2 .

L'obiettivo è determinare un vettore dei pesi $\underline{w}$ tale da separare correttamente le due classi. In particolare, si assume che i campioni della classe c_1 producano un'uscita non negativa, mentre quelli della classe c_2 producano un'uscita negativa:

$$\underline{w}^\top \underline{x} \geq 0 \quad \forall \underline{x} \in c_1$$

$$\underline{w}^\top \underline{x} < 0 \quad \forall \underline{x} \in c_2$$

L'algoritmo per l'adattamento dei pesi del percettrone può essere formulato come segue:

1. Se l' n -esimo membro dell'insieme di addestramento, $\underline{x}(n)$, è classificato correttamente dal vettore dei pesi $\underline{w}(n)$ calcolato alla n -esima iterazione dell'algoritmo, nessuna correzione viene apportata:

$$\underline{w}(n+1) = \underline{w}(n) \quad \text{se } \underline{w}^\top \underline{x} \geq 0 \text{ e } \underline{x}(n) \in c_1$$

$$\underline{w}(n+1) = \underline{w}(n) \quad \text{se } \underline{w}^\top \underline{x} < 0 \text{ e } \underline{x}(n) \in c_2$$

2. Se la classificazione è **errata**, il vettore dei pesi del percettrone è **aggiornato** secondo la seguente regola, dove il parametro η (*learning rate*) controlla l'aggiustamento applicato al vettore di peso all'iterazione n :

$$\underline{w}(n+1) = \underline{w}(n) + \Delta \underline{w}(n) \quad \text{se } \underline{w}^\top \underline{x} \geq 0 \text{ e } \underline{x}(n) \in c_2$$

$$\underline{w}(n+1) = \underline{w}(n) + \Delta \underline{w}(n) \quad \text{se } \underline{w}^\top \underline{x} < 0 \text{ e } \underline{x}(n) \in c_1$$

Si procede assumendo un tasso di apprendimento costante nel tempo, $\eta(n) = \eta > 0$. Si pone $\eta = 1$ e la condizione iniziale $\underline{w}(0) = \underline{0}$. Si suppone inoltre che, all'istante n , si verifichi una condizione di classificazione errata: sia dato un campione $\underline{x}$ tale che $\underline{x} \in c_1$, ma per il quale si ottiene $\underline{w}^\top \underline{x} < 0$. In base alla regola di aggiornamento precedentemente definita e considerando $\eta = 1$, il nuovo vettore dei pesi è:

$$\underline{w}(n+1) = \underline{w}(n) + \underline{x}(n) \tag{7.11}$$

Dato $\underline{w}(0) = \underline{0}$, si ricava la soluzione in via iterativa:

$$\begin{aligned} n = 0 & \quad : \quad \underline{w}(1) = \underline{w}(0) + \underline{x}(0) = \underline{x}(0) \\ n = 1 & \quad : \quad \underline{w}(2) = \underline{w}(1) + \underline{x}(1) = \underline{x}(0) + \underline{x}(1) = \underline{x}(1) \\ n = 2 & \quad : \quad \underline{w}(3) = \underline{w}(2) + \underline{x}(2) = \underline{x}(0) + \underline{x}(1) + \underline{x}(2) = \underline{x}(1) + \underline{x}(2) \end{aligned}$$

e così via, ottenendo:

$$\underline{w}(n+1) = \underline{x}(0) + \underline{x}(1) + \underline{x}(2) + \cdots + \underline{x}(n) \quad (7.12)$$

Se il problema è linearmente separabile, è risolvibile con un perceptrone, deve quindi esistere un **insieme ottimo dei pesi** $\underline{w}_S$ per la quale:

$$\underline{w}_S^\top \underline{x} > 0 \quad ,$$

per i vettori $\underline{x}(1), \dots, \underline{x}(n)$ che appartengono alla classe c_1 . Tra le possibili soluzioni che esistono, si considera una α e si impone sia la più piccola:

$$\alpha = \min_{\underline{x}(n) \in \Gamma_1} \underline{w}_S^\top \underline{x}(n) \quad (7.13)$$

Moltiplicando ambo i membri dell'equazione (7.12) per $\underline{w}_S^\top$:

$$\underline{w}_S^\top \underline{w}(n+1) = \underline{w}_S^\top \underline{x}(0) + \underline{w}_S^\top \underline{x}(1) + \underline{w}_S^\top \underline{x}(2) + \cdots + \underline{w}_S^\top \underline{x}(n)$$

E considerando la (7.13):

$$\underline{w}_S^\top \underline{w}(n+1) \geq n\alpha \quad (7.14)$$

Considerando la disuguaglianza di Cauchy-Schwarz: dati due vettori $\underline{w}_S$ e $\underline{w}(n+1)$ si ha, dopo aver elevato al quadrato:

$$||\underline{w}_S||^2 ||\underline{w}(n+1)||^2 \geq [\underline{w}_S^\top \underline{w}(n+1)]^2$$

Dall'equazione:

$$\underline{w}_S^\top \underline{w}(n+1) \geq n\alpha \quad ,$$

si ha che il termine in parentesi quadrata $\underline{w}_S^\top \underline{w}(n+1)$ è maggiore o uguale di $n^2\alpha^2$, da cui:

$$||\underline{w}_S||^2 ||\underline{w}(n+1)||^2 \geq n^2\alpha^2$$

e quindi:

$$||\underline{w}(n+1)||^2 \geq \frac{n^2\alpha^2}{||\underline{w}_S||^2} \quad (7.15)$$

Ora si segue una strada differente: si parte dalla regola di aggiornamento dei pesi e dalla condizione iniziale $\underline{w}(0) = \underline{0}$:

$$\underline{w}(n+1) = \underline{w}(n) + \underline{x}(n), \quad \text{per } n = 1, \dots, n \text{ e } \underline{x}(n) \in \Gamma_1 \quad (7.16)$$

Prendendo la norma euclidea al quadrato di entrambi i lati di questa equazione, si ottiene:

$$\|\underline{w}(n+1)\|^2 = \|\underline{w}(n)\|^2 + \|\underline{x}(n)\|^2 + 2\underline{w}^\top(n)\underline{x}(n)$$

Si osservi che risulta $\underline{w}^\top(n)\underline{x}(n) < 0$, ovvero l'ultimo termine è negativo per ipotesi che si sia verificata una **classificazione sbagliata**. In conseguenza di ciò, il primo membro dell'equazione precedente è minore o uguale alla somma dei primi due termini del secondo membro:

$$\begin{aligned} \|\underline{w}(n+1)\|^2 &\leq \|\underline{w}(n)\|^2 + \|\underline{x}(n)\|^2 \\ \|\underline{w}(n+1)\|^2 - \|\underline{w}(n)\|^2 &\leq \|\underline{x}(n)\|^2 \end{aligned}$$

La si esplicita per diversi valori di n :

$$\begin{aligned} n=0 & : \quad \|\underline{w}(1)\|^2 - \|\underline{w}(0)\|^2 \leq \|\underline{x}(0)\|^2 \\ n=1 & : \quad \|\underline{w}(2)\|^2 - \|\underline{w}(1)\|^2 \leq \|\underline{x}(1)\|^2 \\ & \vdots \\ n=n & : \quad \|\underline{w}(n+1)\|^2 - \|\underline{w}(n)\|^2 \leq \|\underline{x}(n)\|^2 \end{aligned}$$

Sommo a sinistra della disequazione su tutte le righe ambo i membri:

$$\begin{aligned} \|\underline{w}(1)\|^2 + (\|\underline{w}(2)\|^2 - \|\underline{w}(1)\|^2) + (\|\underline{w}(3)\|^2 - \|\underline{w}(2)\|^2) + \dots + (\|\underline{w}(n+1)\|^2 - \|\underline{w}(n)\|^2) &\leq \\ \leq \|\underline{x}(0)\|^2 + \|\underline{x}(1)\|^2 + \|\underline{x}(2)\|^2 + \dots + \|\underline{x}(n)\|^2 \end{aligned}$$

operando le opportune semplificazioni dei termini:

$$\begin{aligned} \|\underline{w}(n+1)\|^2 &\leq \|\underline{x}(0)\|^2 + \|\underline{x}(1)\|^2 + \dots + \|\underline{x}(n)\|^2 \\ \|\underline{w}(n+1)\|^2 &\leq \sum_{j=1}^n \|\underline{x}(j)\|^2 \end{aligned} \quad (7.17)$$

Si definisce ora $\beta \in \mathbb{R}^+$ il massimo degli ingressi, tale che:

$$\beta = \max_{\underline{x}(j) \in \Gamma_1} \|\underline{x}(j)\|^2 \quad (7.18)$$

così che l'equazione si può riscrivere come:

$$\|\underline{w}(n+1)\|^2 \leq \beta n \quad (7.19)$$

Concentrando l'attenzione sull'equazioni (7.15) e (7.19), il secondo risultato dell'equazione (7.19) è chiaramente in conflitto con il precedente risultato dell'equazione (7.15) per valori sufficientemente grandi di n .

L'unico modo affinché esse siano entrambe valide è affermare che n non può essere maggiore di un certo valore $\bar{n}$, per il quale entrambe le equazioni siano soddisfatte con il segno di uguaglianza. Cioè, $\bar{n}$ è la soluzione dell'equazione:

$$\frac{\bar{n}^2 \alpha^2}{\|\underline{w}_S\|^2} = \beta \bar{n} \quad ,$$

ovvero

$$\bar{n}^2 \alpha^2 - \beta \bar{n} \|\underline{w}_S\|^2 = 0 \quad ,$$

che, risolvendo per $\bar{n}$, porta alle due soluzioni:

$$\bar{n} = 0 \text{ (non accettabile)}$$

$$\bar{n} = \frac{\beta \|\underline{w}_S\|^2}{\alpha^2} \quad (7.20)$$

In conclusione, si è dimostrato che, impostando un tasso di apprendimento costante $\eta(n) = 1$, e partendo da condizioni nulle ($\underline{w}(0) = \underline{0}$, $\underline{x}(0) = \underline{0}$), la convergenza dell'algoritmo è garantita. Se esiste un vettore soluzione $\underline{w}_S$ capace di separare correttamente le classi (ipotesi di separabilità lineare), la procedura di aggiornamento dei pesi sinaptici deve necessariamente interrompersi entro un numero finito di iterazioni, non superiore a $\bar{n}$.

Il vettore dei pesi della rete raggiunge uno stato di stabilità dopo al massimo $\bar{n}$ iterazioni, cessando di subire ulteriori modifiche. Tale risultato, sebbene derivato analiticamente per l'ipotesi Γ_1 , mantiene la sua validità anche per l'ipotesi Γ_2 .

7.5 Error Correcting Learning e LMS

Consideriamo un neurone di tipo ADALINE. L'uscita del neurone coincide con il potenziale interno:

$$y(n) = v(n)$$

Per descrivere il processo di apprendimento, si introduce una funzione di perdita (*loss*), indicata con L (oppure C o J), che rappresenta la quantità che l'algoritmo deve minimizzare al fine di apprendere correttamente dai dati.

In generale:

- Negli **alberi decisionali**, il costo è legato alla riduzione dell'impurità tra nodo padre e nodi figli
- Nel **KNN**, non è presente una vera e propria funzione di costo da ottimizzare, poiché non avviene un apprendimento esplicito dei parametri

Nel caso del neurone ADALINE, la funzione di costo è definita come il valore atteso dell'errore quadratico.

$$L = \mathbb{E}[e(n) v(n)]$$

Poiché non è possibile conoscere tutti i possibili dati della distribuzione, si utilizza una *stima empirica* della loss, indicata con $\hat{L}$, calcolata sul **training set**:

$$\hat{L} = e(n) v(n) = [d(n) - y(n)] v(n) \quad (7.21)$$

dove:

- d_n è l'uscita desiderata (target)
- y_n è l'uscita prodotta dal neurone

L'obiettivo dell'apprendimento è minimizzare questa funzione di costo. A tal fine, si calcola la derivata della loss rispetto ai **pesi** del neurone, i parametri liberi del modello. Questo porta alla determinazione del **gradiente**, utilizzato negli algoritmi di ottimizzazione (ad esempio, discesa del gradiente).

$$\begin{aligned} \frac{\partial L}{\partial \underline{w}(n)} &= \nabla \hat{L} = [d(n) - y(n)] \frac{\partial L}{\partial \underline{w}} v(n) \\ &= \frac{\partial L}{\partial \underline{w}(n)} = \nabla \hat{L} = [d(n) - y(n)] \frac{\partial L}{\partial \underline{w}} \underline{w}^\top \underline{x} \end{aligned}$$

Si considera un problema di **classificazione**. Se l'uscita y_n assume valori discreti, si parla di:

- **Crisp label**: il classificatore assegna direttamente una classe (uscita discreta)

Se invece l'uscita appartiene a un intervallo continuo, si parla di:

- **Soft label**: il classificatore fornisce una misura continua (ad es. una probabilità o un punteggio di appartenenza alla classe)

Quando l'uscita è una crisp label, $y_n \in [-1; +1]$ può essere considerata indipendente dai pesi (w). Il termine desiderato d in equazione 7.21 esce dalla derivata, lasciando solo la derivata dell'attività rispetto ai pesi.

L'attività di un neurone lineare è data dal prodotto scalare tra il vettore dei pesi e il vettore degli ingressi:

$$v = \underline{w}^\top \underline{x}$$

La derivata di v rispetto al vettore dei pesi è quindi:

$$\frac{\partial [w_0 x_0 + w_1 x_1 + w_2 x_2 + \dots]}{\partial \underline{w}} = \underline{x}(n)$$

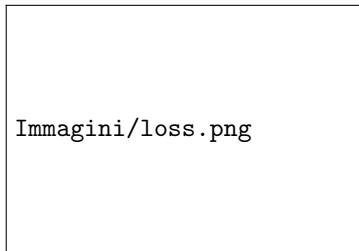
Considerando la funzione di costo basata sull'errore quadratico per un singolo campione e ricordando che per ADALINE vale $y_n = v_n$, si può calcolare il gradiente rispetto ai pesi:

$$\nabla \hat{L} = [d(n) - y(n)] \underline{x}(n) = \underline{e}(n) \underline{x}(n)$$

Un approccio comune consiste nel definire la funzione di costo complessiva come la somma degli errori quadratici su tutti i campioni del training set:

$$L = \frac{1}{2} \sum_{t=1}^m e_t^2 = \frac{1}{2} \sum_{t=1}^m (d_t - y_t)^2 \quad (7.22)$$

Questa funzione può essere interpretata come una superficie nello spazio multi-dimensionale dei pesi. L'obiettivo dell'apprendimento è individuare il punto di minimo di tale superficie, il vettore di pesi $\underline{w}_S$ che minimizza l'errore.



Si può immaginare la funzione di costo (in casi semplici) come una superficie a forma di parabola. Il problema diventa quindi:

Dato un punto qualsiasi sulla superficie di costo, in quale direzione bisogna modificare i pesi per ridurre il valore della funzione di perdita?

La risposta è fornita dal **gradiente**: esso indica la direzione di massima crescita della funzione. Per minimizzare la *loss*, è necessario muoversi nella direzione opposta al gradiente. La variazione dei pesi è quindi definita come:

$$\Delta w = -\eta \nabla \hat{L} = \eta \underline{e}(n) \underline{x}(n)$$

Questa formulazione garantisce che l'aggiornamento avvenga sempre nella direzione di diminuzione della funzione di costo:

- Se il gradiente è positivo, ci si muove in direzione opposta
- Se il gradiente è negativo, l'aggiornamento porta comunque verso il minimo

In questo modo, iterando il processo sui dati di addestramento, i pesi convergono (in condizioni opportune) verso un minimo della funzione di perdita.

Nella pratica, la funzione di costo raramente presenta una forma semplice e convessa (come una parabola). Al contrario, può assumere una struttura complessa, caratterizzata dalla presenza di **minimi locali** oltre al **minimo globale**. Se il processo di ottimizzazione converge verso un minimo locale, la rete neurale potrebbe non raggiungere le prestazioni ottimali.

Immagini/lossvar.png

Un ruolo fondamentale è svolto dal **tasso di apprendimento** η :

- Se η è troppo piccolo, gli aggiornamenti dei pesi risultano molto ridotti, rallentando la convergenza e rendendo difficile uscire da eventuali minimi locali
- Se η è troppo grande, gli aggiornamenti possono risultare eccessivi, causando oscillazioni o addirittura impedendo la convergenza, poiché si rischia di oltrepassare il minimo globale

Per mitigare questi problemi, si introduce spesso un **tasso di apprendimento variabile**, indicato con $\eta(n)$, dove n rappresenta l'iterazione. Una strategia comune consiste nell'iniziare con un valore relativamente elevato (ad esempio $\eta_0 = 10^{-1}$) e ridurlo progressivamente durante l'addestramento. Una possibile legge di decadimento è:

$$\eta(n) = \eta_0 \cdot \frac{1}{1 + \frac{n}{\tau}} \quad , \quad (7.23)$$

con τ costante positiva che controlla la velocità di decadimento del tasso di apprendimento.

Questo approccio consente:

- Nelle fasi iniziali, di effettuare passi più ampi, utili per esplorare lo spazio dei pesi e superare eventuali minimi locali
- Nelle fasi finali, di ridurre progressivamente l'ampiezza degli aggiornamenti, favorendo una convergenza più stabile e precisa verso un minimo

Come si evince dalla (7.23), all'aumentare del numero di iterazioni, il tasso di apprendimento $\eta(n)$ tende a diminuire.

Considerando ora la formulazione della funzione di perdita basata sull'errore quadratico medio, **LMS** (*Least Mean Squares*), è naturale chiedersi a quale paradigma di apprendimento essa conduca.

7.5.1 Paradigma di apprendimento della LMS

La variazione dei pesi è definita dalla regola della discesa del gradiente:

$$\Delta w_i = -\eta \frac{\partial L}{\partial w_i}$$

dove la funzione di costo è:

$$L = \frac{1}{2} \sum_{t=1}^m (d_t - y_t)^2$$

Per derivare la regola di aggiornamento, si considera un singolo campione e si analizza la derivata di:

$$e_t^2 = (d_t - y_t)^2$$

rispetto al peso w_i :

$$\frac{\partial e_t^2}{\partial w_i} = \frac{\partial}{\partial w_i} (d_t - y_t)^2$$

Sviluppando il quadrato si ottiene:

$$(d_t - y_t)^2 = d_t^2 + y_t^2 - 2d_t y_t$$

La derivata può quindi essere calcolata termine per termine:

$$\frac{\partial}{\partial w_i} [d_t^2 + y_t^2 - 2d_t y_t]$$

Il primo termine è nullo, poiché il target d_t non dipende dai pesi. I restanti termini dipendono invece da y_t , per cui è necessario calcolare:

$$\frac{\partial y_t}{\partial w_i}$$

L'uscita del modello è data da:

$$y_t = \sum_{i=0}^p w_i x_{it}$$

Per calcolare il termine:

$$\frac{\partial y_t^2}{\partial w_i},$$

si esplicita:

$$y_t = \sum_{k=0}^p w_k x_{kt},$$

da cui:

$$y_t^2 = \left(\sum_{k=0}^p w_k x_{kt} \right)^2 = \sum_{l=0}^p \sum_{k=0}^p w_l x_{lt} w_k x_{kt}$$

Derivando rispetto a w_i , contribuiscono solo i termini che contengono w_i . In particolare, fissato un indice, si selezionano i termini in cui compare w_i , ottenendo:

$$\frac{\partial y_t^2}{\partial w_i} = 2 \sum_{k=0}^p x_{it} w_k x_{kt}$$

dove il fattore 2 deriva dal fatto che i termini incrociati compaiono due volte nella doppia sommatoria.

Raccogliendo:

$$\frac{\partial y_t^2}{\partial w_i} = 2x_{it} \sum_{k=0}^p w_k x_{kt} ,$$

e riconoscendo l'espressione di y_t , si ottiene:

$$\frac{\partial y_t^2}{\partial w_i} = 2x_{it} y_t$$

Si può ora procedere con il calcolo completo:

$$\frac{\partial e_t^2}{\partial w_i} = 0 + 2x_{it}y_t - 2d_t x_{it} = 2x_{it}(y_t - d_t)$$

Introducendo l'errore $e_t = d_t - y_t$, segue:

$$\frac{\partial e_t^2}{\partial w_i} = -2x_{it}e_t$$

Considerando la funzione di costo L , si ha:

$$\frac{\partial L}{\partial w_i} = -x_{it}e_t \quad ,$$

e quindi la regola di aggiornamento LMS:

$$\Delta w_i = \eta \frac{\partial L}{\partial w_i} = \frac{1}{2} [-2\eta x_{it}e_t]$$

$$\Delta w = -\eta x_{it} e_t \quad , \quad (7.24)$$

che non è altro che l'**error correcting learning**.

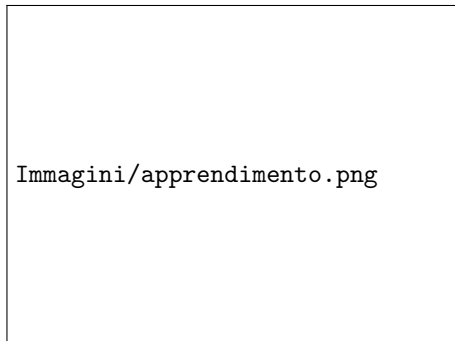
7.5.2 Curva di apprendimento

Il processo di apprendimento di un modello avviene in maniera iterativa: ad ogni passo di addestramento i pesi vengono aggiornati sulla base dell'errore commesso. Per analizzare l'andamento di questo processo è utile introdurre la *curva di apprendimento*.

Tale curva rappresenta l'andamento della funzione di costo L in funzione del numero di passi di addestramento. In particolare:

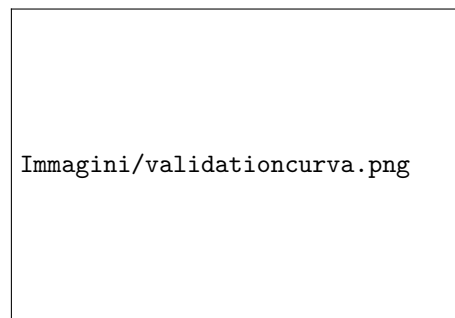
- Sull'asse orizzontale si riporta il numero di campioni (o passi di apprendimento)
- Sull'asse verticale si riporta il valore della loss

Idealmente, ci si aspetta che la curva parta da un valore elevato e decresca progressivamente, indicando che il modello sta imparando a ridurre l'errore.



Il grafico in Figura 7.5.2 rappresenta tipicamente l'andamento della loss sul **training set**. Durante l'addestramento, infatti, i pesi vengono aggiornati proprio per minimizzare questa quantità.

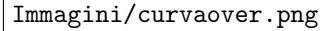
Tuttavia, per valutare la capacità del modello di generalizzare, è fondamentale considerare anche un insieme di dati separato, il **validation set**, sul quale la loss viene calcolata senza aggiornare i pesi.



L'andamento della loss sul validation set può assumere comportamenti differenti:

- Può decrescere in modo simile a quella del training set, indicando una buona capacità di generalizzazione
- Può inizialmente diminuire e successivamente aumentare

Quest'ultimo caso è indice di **overfitting**: il modello continua a migliorare le prestazioni sul training set, ma peggiora su dati mai visti.



Il divario tra le due curve (training e validation) rappresenta la differenza tra le prestazioni sui dati di addestramento e quelle su dati nuovi. Un gap elevato indica che il modello si è eccessivamente adattato al training set, perdendo capacità di generalizzazione.

E' importante chiarire il significato di **passo di addestramento**. In molti casi, l'aggiornamento dei pesi non avviene dopo ogni singolo campione, ma dopo aver presentato alla rete l'intero insieme di dati di training (epoca).

Early stopping Un metodo per stabilire quando interrompere l'addestramento di una rete neurale è l'**early stopping**. In generale, durante le prime fasi dell'addestramento, la perdita (loss) sul training set tende a diminuire. Tuttavia, dopo un certo numero di iterazioni, può accadere che la perdita sul validation set inizi ad aumentare, segnalando un possibile fenomeno di overfitting. L'early stopping consiste nell'interrompere l'addestramento non appena la prestazione sul validation set peggiora in modo sistematico. Si monitora la curva di validation e si arresta l'addestramento quando questa mostra un andamento divergente rispetto a quella di training. A tal fine si introduce un parametro chiamato **patience**, che indica il numero massimo di iterazioni consecutive durante le quali la prestazione sul validation set può non migliorare. Se tale soglia viene superata, l'addestramento viene interrotto. I pesi della rete vengono salvati nel punto in cui si è osservata la migliore prestazione sul validation set, prima dell'inizio del peggioramento.

7.5.3 Gradiente discendente stocastico

Aggiornare i pesi dopo ogni singolo campione può introdurre un'elevata variabilità negli aggiornamenti (alta volatilità), portando a instabilità nel processo di apprendimento. Per questo motivo, nella pratica si preferiscono strategie intermedie (ad esempio **mini-batch**), che permettono un compromesso tra stabilità e velocità di apprendimento, aggiornando ogni k campioni.

In questo contesto, l'aggiornamento dei pesi non avviene né dopo ogni singolo campione né dopo aver processato l'intero dataset, ma dopo aver elaborato un sottoinsieme di k campioni. Ogni mini-batch fornisce quindi una stima della direzione di discesa del gradiente, utilizzata per aggiornare i pesi del modello.

Il processo di addestramento richiede la ripetizione di più **epoche**. Ad ogni nuova epoca, i campioni vengono tipicamente **mescolati in modo casuale** (*shuffling*) prima di essere suddivisi in mini-batch. Questa randomizzazione ha un ruolo fondamentale: evita che il modello apprenda eventuali correlazioni spurie legate all'ordine dei dati e contribuisce a rendere il processo di ottimizzazione più stabile ed efficace.

Il metodo di ottimizzazione descritto prende il nome di **gradiente discendente stocastico** (*Stochastic Gradient Descent, SGD*), in quanto l'aggiornamento dei pesi si basa su una stima stocastica (ovvero approssimata) del gradiente, calcolata su sottoinsiemi **casuali** dei dati piuttosto che sull'intero dataset.

Dimensione del mini-batch In quasi tutti i computer, 32 campioni rappresentati in feature reali rientrano perfettamente dentro una RAM. Tuttavia, il consumo di memoria varia drasticamente in base alla natura dei dati. Considerando ad esempio un video di 5 secondi a colori con risoluzione 3000×1600 :

- **Singolo frame:** $4.8M$ (pixel) $\times$ 3 (canali) $\times$ 8 bit = 115 Mb $\approx$ 14 MB
- **Video intero (30 fps):** 14 MB $\times$ 30 fps $\times$ 5 sec = 2100 MB $\approx$ 2.1 GB
- **Mini-batch** ($m' = 32$): 2.1 GB $\times$ 32 $\approx$ 64 GB

Avere un mini-batch grande su pochi campioni non ha senso. Siccome i tipici dataset sono dell'ordine delle migliaia, i valori di default delle librerie seguono multipli di 2, bilanciando l'efficienza computazionale con i limiti fisici della memoria.

7.6 Tecniche per mitigare l'overfitting

7.6.1 Semplificazione del modello

Una delle principali cause dell'overfitting è l'eccessiva complessità del modello, dovuta alla presenza di un numero elevato di parametri (pesi e bias). Un modello troppo complesso tende infatti ad adattarsi eccessivamente ai dati di training, perdendo capacità di generalizzazione. Per mitigare questo problema è possibile ridurre la complessità del modello, ad esempio diminuendo il numero di neuroni o di layer. Non esistono regole generali per determinare a priori la struttura ottimale: spesso si procede in modo empirico, semplificando progressivamente il modello e valutandone le prestazioni sul validation set.

7.6.2 Early stopping

Un'altra tecnica consiste nell'interrompere l'addestramento quando le prestazioni sul validation set iniziano a peggiorare. In questo modo si evita che il modello continui ad apprendere dettagli specifici del training set, responsabili dell'overfitting. L'addestramento viene quindi arrestato non appena si osserva un comportamento divergente tra la curva di training e quella di validation.

7.6.3 Data Augmentation

La data augmentation rappresenta un approccio complementare alla semplificazione del modello: invece di ridurre la complessità del modello a parità di dati, si aumenta la quantità (o la varietà) dei dati mantenendo invariata la struttura del modello.

Un possibile approccio è l'utilizzo di modelli di *Generative AI*, in grado di produrre nuovi campioni sintetici. Questa strategia non è banale, richiede che i dati generati appartengano alla stessa distribuzione dei dati reali. La definizione di metriche affidabili per valutare questa proprietà rappresenta una delle principali sfide in questo ambito.

La data augmentation può essere realizzata anche senza ricorrere a modelli generativi, ma tramite **trasformazioni statistiche o geometriche**, particolarmente efficaci nel caso di immagini e video. A partire da un campione originale, è possibile generare varianti applicando trasformazioni quali rotazioni, traslazioni, scalature o altre deformazioni. Il processo può essere descritto come segue:

- Per ciascun campione, prima dell'inserimento in una mini-batch, si estrae un numero casuale
- Se tale numero supera una certa soglia, si applica una trasformazione scelta casualmente da un insieme predefinito
- In caso contrario, il campione viene inserito nel batch senza modifiche

In questo approccio non generativo, il **numero di campioni rimane invariato**, ma aumenta la varietà delle rappresentazioni fornite al modello, migliorandone la capacità di generalizzazione.

7.6.4 Regolarizzazione

Quando una rete neurale va in overfitting, si osserva tipicamente che la funzione di loss diminuisce rapidamente sul training set, mentre aumenta sul validation set. Una discesa molto rapida della loss nel training indica che i pesi della rete stanno assumendo valori elevati o variando in modo significativo, adattandosi eccessivamente ai dati di addestramento. La **regolarizzazione** introduce dei vincoli sui pesi, penalizzando valori troppo grandi e favorendo soluzioni più semplici, che tendono a generalizzare meglio. Nel caso della regolarizzazione **L1** (*Lasso*), la funzione di loss viene modificata aggiungendo un termine di penalizzazione proporzionale alla somma dei valori assoluti dei pesi:

$$L = L_0 + \frac{\lambda}{m} \sum_{i=1}^n |\omega_i| \quad (7.25)$$

L'equazione (7.25) mostra che alla loss originale L_0 viene aggiunto un termine che dipende dal valore assoluto dei pesi. Questo contributo aumenta il valore

complessivo della loss, rendendo più difficile la sua minimizzazione e scoraggiando la crescita eccessiva dei pesi. Il parametro λ controlla l'importanza del termine di regolarizzazione rispetto alla loss originale: valori più elevati di λ comportano una penalizzazione più forte.

Un'alternativa è la regolarizzazione **L2**, in cui il termine di penalizzazione è basato sul quadrato dei pesi:

$$L = L_0 + \frac{\lambda}{2m} \sum_{i=1}^n \omega_i^2 \quad (7.26)$$

In questo caso, il contributo della regolarizzazione cresce riquadrando, penalizzando in modo più marcato i pesi di grande ampiezza. Questo porta generalmente a un apprendimento più graduale e a modelli più stabili. Un ulteriore vantaggio della regolarizzazione L2 è di natura analitica: il termine quadratico è continuo e derivabile in ogni punto, rendendolo particolarmente adatto ai metodi di ottimizzazione basati sul gradiente. Il termine L1, basato sul valore assoluto, non è derivabile nell'origine, il che può rendere l'ottimizzazione leggermente più complessa.

7.6.5 Dropout

Si consideri una rete neurale con un elevato numero di neuroni. La tecnica del **dropout** consiste nello spegnere in modo casuale una parte dei neuroni, ponendo temporaneamente a zero le loro attivazioni. In questo modo si ottiene, a ogni iterazione, una sottorete più piccola della rete originale.

Il **procedimento** è il seguente: si estrae un mini-batch di dati e si definisce una configurazione della rete in cui alcuni neuroni vengono disattivati casualmente. Il mini-batch viene fatto passare attraverso la sottorete, si calcola la loss e si aggiornano i pesi. Durante l'aggiornamento, i neuroni disattivati non vengono modificati (*freeze dei neuroni*), impedendo al gradiente di propagarsi attraverso di essi. Successivamente, mantenendo lo stesso mini-batch, si genera una nuova configurazione spegnendo un diverso insieme casuale di neuroni. Si ripete quindi il processo di forward, calcolo della loss e aggiornamento dei pesi. Questa procedura viene iterata più volte.

Al termine dell'elaborazione del mini-batch, si ottengono diverse configurazioni della rete, ciascuna con pesi aggiornati. Per passare al mini-batch successivo, si considerano **i pesi medi risultanti da queste configurazioni**.

Il dropout può essere interpretato come un metodo appartenente alla famiglia dei **metodi multiesperto**: ogni sottorete generata equivale a un modello diverso, e la media dei pesi rappresenta una forma di aggregazione (*ensemble*) di più modelli.

7.6.6 Inizializzazione della rete

Abbiamo affermato precedentemente che i pesi di una rete neurale vengono inizializzati in modo casuale. Fino a circa una decina di anni fa, una scelta comune

era quella di inizializzare i pesi con una distribuzione gaussiana a media nulla e varianza unitaria. Si può interpretare ogni neurone come una combinazione lineare di variabili casuali indipendenti. In particolare, la **combinazione di variabili gaussiane** a media nulla e varianza unitaria è ancora una variabile gaussiana a media nulla, ma con varianza pari a:

$$\sigma^2 = n \quad \text{con } n = \# \text{neuroni in ingresso}$$

Anche se i singoli pesi sono estratti da distribuzioni con varianza unitaria, la loro combinazione porta a una variabile con varianza che cresce linearmente con il numero di neuroni in ingresso. Le attivazioni possono assumere valori molto elevati, causando problemi come **overfitting** e **saturazione** delle funzioni di attivazione. Per evitare questi effetti indesiderati, sono state introdotte strategie di inizializzazione più sofisticate. Una delle più diffuse è l'inizializzazione di **Xavier**, in cui i pesi sono estratti da una distribuzione gaussiana a media nulla e varianza pari a:

$$\sigma^2 = \frac{1}{n}$$

(in alternativa, in una forma più generale, $\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}$).

Questa scelta mantiene la varianza delle attivazioni stabile attraverso i diversi layer della rete, evitando sia l'esplosione che la scomparsa dei segnali durante la propagazione.

7.6.7 Batch Normalization

I dati in ingresso a una rete neurale vengono spesso normalizzati, ad esempio nell'intervallo $[-1, 1]$ oppure imponendo media nulla e varianza unitaria. Tuttavia, quando si lavora con mini-batch estratti casualmente dal dataset, non si ha la certezza che ogni batch mantenga queste proprietà statistiche. Questa variabilità è dovuta al fatto che i campioni sono selezionati in modo randomico e ciò può introdurre instabilità nel processo di apprendimento.

Per mitigare questo problema, durante l'utilizzo della rete si può attivare un'opzione chiamata **Batch Normalization**. Questa tecnica consiste nel normalizzare direttamente i dati all'interno di ciascun mini-batch, imponendo media nulla e varianza unitaria. Empiricamente si osserva che questo approccio rende il processo di addestramento più stabile e migliora la convergenza della rete.

7.7 Multi-Layer Perceptrons (MLP)

Le nozioni introdotte per il singolo neurone si estendono naturalmente ai *Multi-Layer Perceptrons* (MLP), ovvero reti neurali composte da più percettroni organizzati in strati.

Un esempio di architettura MLP è mostrato in Figura 7.9.

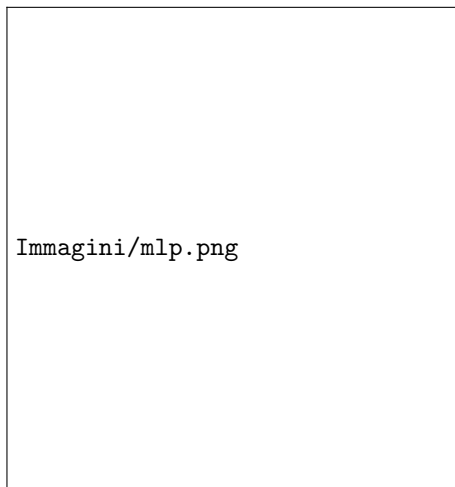


Figura 7.9: Esempio di architettura di una rete neurale *Multi-Layer Perceptron*. In rosso sono mostrati i neuroni di ingresso, in blu quelli degli strati nascosti e in verde quelli di uscita.

Un MLP è una rete **fully connected**, in cui ogni neurone di uno strato è connesso a tutti i neuroni dello strato successivo. Inoltre, è una rete di tipo **feedforward**: l'informazione fluisce dall'ingresso verso l'uscita senza la presenza di cicli o connessioni di ritorno.

Le reti che includono connessioni all'indietro o dipendenze temporali sono invece dette **ricorrenti** (o *recurrent neural networks*) e sono tipicamente utilizzate per l'analisi di sequenze e serie temporali.

All'interno di un MLP si distinguono tre tipologie di strati:

- **Input layer**: riceve i dati in ingresso (le feature $\underline{x}$)
- **Hidden layer**: uno o più strati intermedi che trasformano progressivamente l'informazione
- **Output layer**: produce l'uscita del modello

Gli strati nascosti sono detti tali perché non sono direttamente osservabili: non interagiscono né con l'ingresso grezzo né con l'uscita finale, ma operano come livelli intermedi di rappresentazione.

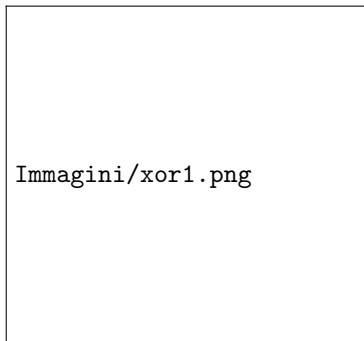
A differenza del perceptrone singolo, in cui l'errore è calcolato direttamente sull'uscita, negli MLP la presenza degli strati nascosti rende necessario propagare l'informazione dell'errore all'indietro nella rete (tramite l'algoritmo di *backpropagation*) per aggiornare tutti i pesi.

Un aspetto fondamentale è che, grazie alla composizione di più strati e all'uso di funzioni di attivazione non lineari, gli MLP sono in grado di modellare relazioni

non lineari e quindi di apprendere frontiere di decisione molto più complesse rispetto al perceptrone lineare.

7.7.1 Il problema dello XOR

Il problema dello XOR è un classico nella ricerca sulle reti neurali artificiali. Esso consiste nel determinare l'uscita della funzione logica XOR dati due ingressi binari. La funzione restituisce valore 1 se gli ingressi sono diversi e 0 se sono uguali.



Dati gli input x_1 e x_2 , lo spazio delle feature è:

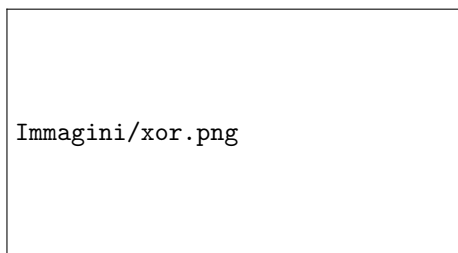
$$\underline{x} = [x_1, x_2]^\top$$

I campioni disponibili sono:

- $[0, 0]^\top \rightarrow 0$
- $[1, 0]^\top \rightarrow 1$
- $[0, 1]^\top \rightarrow 1$
- $[1, 1]^\top \rightarrow 0$

Questi punti non sono linearmente separabili, non esiste una singola retta nel piano (x_1, x_2) che separi correttamente le due classi. Per questo motivo è necessario introdurre uno strato nascosto.

Consideriamo quindi una rete **2-2-1**, composta da due neuroni nello strato hidden e uno nello strato di output.



Ogni neurone è caratterizzato da pesi w_{ij} e bias θ_i , dove j indica il nodo di partenza e i il neurone di arrivo.

L'idea è quella di costruire, tramite i neuroni hidden, delle rette nel piano delle feature. Un neurone hidden calcola infatti:

$$x_1 w_1 + x_2 w_2 + \theta \quad ,$$

e l'equazione:

$$x_1 w_1 + x_2 w_2 + \theta = 0 \quad ,$$

rappresenta una retta.

Si scelgono quindi due rette semplici del tipo:

$$x_1 + x_2 = c \quad ,$$

che corrispondono a rette con pendenza $m = -1$.

Una prima retta è:

$$x_1 + x_2 = \frac{3}{2} \quad ,$$

che in forma standard diventa:

$$x_1 + x_2 - \frac{3}{2} = 0 \quad ,$$

da cui si ricavano i parametri del **primo neurone hidden**:

$$w_{11} = 1, \quad w_{21} = 1, \quad \theta_1 = -\frac{3}{2} \quad (7.27)$$

Una seconda retta è:

$$x_1 + x_2 = \frac{1}{2} \quad ,$$

che in forma standard diventa:

$$x_1 + x_2 - \frac{1}{2} = 0 \quad ,$$

da cui si ottengono i parametri del **secondo neurone hidden**:

$$w_{12} = 1, \quad w_{22} = 1, \quad \theta_2 = -\frac{1}{2} \quad (7.28)$$

Le due rette separano il piano nel modo seguente:

- la retta $x_1 + x_2 = \frac{3}{2}$ separa il punto $(1, 1)$ dagli altri
- la retta $x_1 + x_2 = \frac{1}{2}$ separa il punto $(0, 0)$ dagli altri

In questo modo, i punti appartenenti alla classe 1 risultano isolati in una regione intermedia.

I neuroni hidden trasformano quindi lo spazio originale (x_1, x_2) in una nuova rappresentazione (h_1, h_2) in cui il problema diventa linearmente separabile.

Il **neurone di output** combina queste nuove feature per produrre la classificazione finale.

$$y_1 w_{31} + y_2 w_{32} + \theta_3$$

Si possono scegliere come valori:

$$w_{31} = -2 \quad , \quad w_{32} = 1 \quad , \quad \theta_3 = \frac{1}{2} \quad ,$$

da cui segue:

$$-2y_1 + y_2 - \frac{1}{2}$$

Supponiamo per semplicità di calcolo che i tre neuroni usino il gradino come funzione di trasferimento $\psi(\cdot)$. Quindi:

$$y_1 = \psi\left(x_1 + x_2 - \frac{3}{2}\right) \quad (7.29)$$

$$y_2 = \psi\left(x_1 + x_2 - \frac{1}{2}\right) \quad (7.30)$$

$$y_3 = -2\psi\left(x_1 + x_2 - \frac{3}{2}\right) + \psi\left(x_1 + x_2 - \frac{1}{2}\right) - \frac{1}{2} = 0 \quad (7.31)$$

Queste relazioni permettono di calcolare l'output a partire dall'input (Figura 7.10):

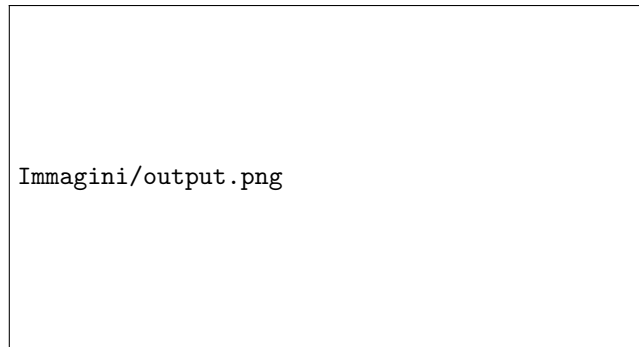
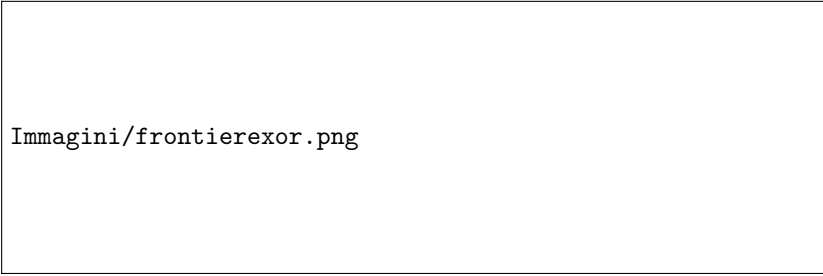


Figura 7.10: Risultati dei tre neuroni nel calcolo dello XOR



Immagini/frontierexor.png

Figura 7.11: Frontiere decisionali costruite dai tre neuroni della rete MLPs utilizzata per risolvere il problema dello XOR

Come visto, l'uscita di un neurone può costituire l'ingresso di un altro: questo è un principio fondamentale nei *Multi-Layer Perceptrons* (MLP), dove gli strati sono concatenati tra loro. Nel caso del problema XOR, la frontiera decisionale risultante non è una singola retta, ma una regione più complessa (una sorta di *striscia*), come mostrato in Figura 7.11. Questo evidenzia come il modello sia in grado di rappresentare **separazioni non lineari**. Per costruire tale frontiera, i neuroni dello strato nascosto trasformano lo spazio originale delle feature (x_1, x_2) in una nuova rappresentazione. In particolare, ciascun neurone hidden calcola una combinazione lineare degli ingressi seguita da una funzione di attivazione, producendo nuove variabili intermedie.

Lo spazio generato da queste nuove variabili prende il nome di **spazio latente** (*latent space*). In questo spazio, le coordinate non sono più le feature originali, ma le uscite dei neuroni dello strato precedente. Nel caso specifico, le uscite dei due neuroni hidden definiscono un nuovo spazio bidimensionale (h_1, h_2) , che costituisce lo spazio di ingresso del neurone di output y_3 .

Un aspetto fondamentale è che, mentre i dati non sono linearmente separabili nello spazio originale, possono diventarlo nello spazio latente. Il neurone di output opera quindi in questo nuovo spazio, dove è sufficiente una separazione lineare per risolvere il problema. In generale, una rete neurale introduce tanti spazi latenti quanti sono gli strati nascosti: ogni strato costruisce una rappresentazione intermedia sempre più astratta dei dati.

7.8 Error backpropagation: le equazioni

In questa sezione si analizza il processo di apprendimento di una rete neurale, ovvero come vengono aggiornati i pesi durante la fase di addestramento.

Consideriamo un'architettura MLP semplificata di tipo 3-4-2:

Immagini/mplex.png

La rete è assunta **fully connected** e composta da tre strati: input, hidden e output. Come visto per il singolo neurone, l'uscita è data da una funzione di attivazione applicata a una combinazione lineare degli ingressi:

$$y = \psi(v) = \psi \left(\sum_{k=1}^p w_k x_k + \theta \right)$$

Nel caso di una rete MLP, l'uscita del neurone j allo strato l è:

$$y_j^l = \psi \left(\underbrace{\sum_k w_{jk}^l y_k^{l-1} + \theta_j^l}_{v_j^l} \right) ,$$

dove:

- y_k^{l-1} sono le uscite dello strato precedente
- w_{jk}^l sono i pesi delle connessioni
- θ_j^l è il bias
- v_j^l è il potenziale di attivazione del neurone

Il processo di addestramento di una rete neurale segue una sequenza di passi iterativi:

1. Si inizializzano i pesi (tipicamente in modo casuale)
2. Si seleziona un insieme di campioni (batch o mini-batch)
3. I dati vengono propagati in avanti attraverso la rete (*forward pass*)
4. Si confronta l'uscita della rete y con la ground truth d
5. Si calcola la funzione di costo (loss)
6. Si calcola il gradiente della loss rispetto ai pesi
7. Si determinano gli aggiornamenti dei pesi Δw

8. Gli aggiornamenti vengono scalati tramite il parametro di apprendimento η
9. I pesi vengono aggiornati
10. Il processo viene ripetuto sul batch successivo

Nel caso di una rete MLP, il paradigma di apprendimento del singolo perceptrone non è più direttamente applicabile. Infatti, mentre nello strato di output è possibile calcolare la loss come scarto tra ground truth d e predizione y , negli strati nascosti tale informazione non è disponibile. Nei layer hidden non si conosce un valore di riferimento con cui confrontare direttamente l'uscita dei neuroni. Di conseguenza, non è possibile aggiornare i pesi utilizzando la stessa regola locale impiegata nel perceptrone.

Il processo di addestramento di una rete neurale può essere visto come composto da due fasi:

- una fase in avanti (*forward pass*), in cui i dati attraversano la rete producendo un'uscita
- una fase all'indietro (*backward pass*), in cui l'errore calcolato all'uscita viene utilizzato per aggiornare i pesi

L'algoritmo di **backpropagation** estende il principio del perceptrone introducendo un meccanismo per propagare all'indietro un segnale di errore, rendendo possibile l'aggiornamento dei pesi anche negli strati nascosti.

L'idea centrale è definire, per ogni neurone della rete, una quantità che misuri quanto esso contribuisce all'errore finale. Nel layer di uscita l'errore è direttamente calcolabile a partire dalla loss:

$$L \approx d - y$$

Negli strati intermedi si introduce invece un **errore locale**:

$$\delta_j^l = \frac{\partial L}{\partial v_j^l} \quad ,$$

dove v_j^l è il potenziale di attivazione del neurone j allo strato l .

Per comprendere il significato di δ_j^l , si considera una piccola variazione dell'attività del neurone:

$$v_j^l \rightarrow v_j^l + \Delta v_j^l$$

Questa variazione produce una modifica dell'uscita del neurone:

$$y_j^l = \psi(v_j^l + \Delta v_j^l)$$

Poiché l'uscita di un neurone costituisce l'ingresso dei neuroni dello strato successivo, tale variazione si propaga lungo la rete fino a influenzare l'uscita finale e quindi il valore della funzione di costo.

La conseguente variazione della loss può essere espressa come:

$$\Delta L = \frac{\partial L}{\partial v_j^l} \cdot \Delta v_j^l \quad ,$$

ovvero:

$$\Delta L = \delta_j^l \cdot \Delta v_j^l$$

Il termine δ_j^l rappresenta quindi il fattore di proporzionalità tra la variazione dell'attività del neurone e la variazione della loss. Esso misura quanto una variazione locale nel neurone influisce sull'errore complessivo.

Il problema dell'apprendimento negli strati nascosti si riduce quindi al calcolo di questi errori locali a partire dall'errore osservato in uscita. L'algoritmo di backpropagation fornisce una procedura sistematica per propagare tali quantità all'indietro nella rete, consentendo l'aggiornamento di tutti i pesi.

7.8.1 1. Errore locale nello strato di uscita ($l = L$)

Calcoliamo l'errore locale δ_j^L per un neurone j appartenente allo strato di uscita. Per definizione:

$$\delta_j^L = \frac{\partial L}{\partial v_j^L}$$

Per esplicitare questa quantità, consideriamo la dipendenza della loss dalle uscite del layer finale:

$$\delta_j^L = \sum_k \frac{\partial L}{\partial y_k^L} \cdot \frac{\partial y_k^L}{\partial v_j^L}$$

Tuttavia, l'uscita y_k^L dipende da v_j^L solo nel caso in cui $k = j$. Per $k \neq j$, la derivata è nulla, e la sommatoria si riduce al solo termine corrispondente:

$$\delta_j^L = \frac{\partial L}{\partial y_j^L} \cdot \frac{\partial y_j^L}{\partial v_j^L}$$

Poiché l'uscita del neurone è data da:

$$y_j^L = \psi(v_j^L) \quad ,$$

si ottiene:

$$\delta_j^L = \frac{\partial L}{\partial y_j^L} \cdot \psi'(v_j^L) \tag{7.32}$$

L'equazione (7.32) rappresenta l'errore locale nello strato di uscita e mostra che esso è il prodotto tra:

- La sensibilità della loss rispetto all'uscita del neurone;
- La derivata della funzione di attivazione

Un aspetto importante riguarda il comportamento della derivata $\psi'(v_j^L)$. Nel caso di funzioni di attivazione come la sigmoide, per valori molto grandi (positivi o negativi) dell'attività v_j^L , la funzione tende a saturare, ovvero si avvicina a un asintoto orizzontale.

In queste regioni, la derivata $\psi'(v_j^L)$ diventa molto piccola (tende a zero), e di conseguenza anche l'errore locale δ_j^L si riduce notevolmente.

Questo fenomeno comporta che l'aggiornamento dei pesi diventa molto piccolo, rallentando o addirittura bloccando il processo di apprendimento. Tale effetto è noto come **problema della saturazione** (*vanishing gradient*). Spesso la sigmoide viene sostituita negli **strati interni** della rete con una funzione di attivazione ψ denominata **RELU**, che risolve il problema del gradiente evanescente:



Figura 7.12: Funzione RELU

Tipicamente si usa nei layer interni e non nel layer di uscita perché non essendo limitata (*bounded*) fornisce un problema nell'interpretazione di y .

7.8.2 2. Errore locale nello strato interno l

Consideriamo l'errore nello strato intermedio (*hidden*) l :

$$\delta_j^l = \frac{\partial L}{\partial v_j^l}$$

Consideriamo l'errore anche per il layer successivo $l + 1$ e sommiamo per tutte le connessioni k in uscita:

$$\delta_j^{l+1} = \frac{\partial L}{\partial v_j^{l+1}} = \sum_k \frac{\partial L}{\partial y_k^{l+1}} \cdot \frac{\partial y_k^{l+1}}{\partial v_j^l}$$

Sommando quelle dopo, il principio dell'annullamento della derivata non vale più. Il termine:

$$\frac{\partial L}{\partial v_k^{l+1}} = \delta_k^{l+1} \quad ,$$

è l'errore locale del neurone k . Inoltre:

$$v_k^{l+1} = \sum_j w_{kj}^{l+1} \cdot y_j^l + \theta_k^{l+1}$$

Considerandone la derivata:

$$\frac{\partial v_k^{l+1}}{\partial v_j^l} = \frac{\partial (w_{kj}^{l+1} \cdot y_j^l + \theta_k^{l+1})}{\partial v_j^l} = w_{kj}^{l+1} \cdot \psi'(v_j^l)$$

Si può quindi affermare che:

$$\delta_j^l = \left[\sum_k \delta_k^{l+1} \cdot w_{kj}^{l+1} \right] \cdot \psi'(v_j^l) \quad (7.33)$$

L'errore locale sul j -esimo neurone nello strato interno è dato dalla derivata della sua funzione di attivazione, dalla somma su tutte le sue connessioni in uscita degli errori locali dei neuroni a lui successivi per i pesi che lo collegano.

Abbiamo affermato precedentemente che nel processo di addestramento di una rete neurale si distinguono due fasi:

- Una fase *forward*, in cui i dati attraversano la rete a pesi fissati (non ancora aggiornati), producendo l'uscita
- Una fase *backward*, in cui l'errore calcolato sull'uscita viene propagato all'indietro per aggiornare i pesi

Durante il *backward pass*, l'aggiornamento dei pesi si basa sugli errori locali dei neuroni. Ogni neurone deve ricevere un'informazione su quanto contribuisce all'errore finale.

Per i neuroni dello strato di uscita, questo errore è direttamente calcolabile. Per i neuroni negli strati nascosti, invece, l'errore deve essere ricavato a partire dagli **strati successivi**. L'idea fondamentale è che l'errore locale di un neurone in uno strato l dipende dagli errori locali dei neuroni nello strato successivo $l + 1$. In altre parole, l'errore viene propagato all'indietro lungo la rete.

Più precisamente, l'errore locale viene ottenuto combinando:

- Gli errori locali dei neuroni dello strato successivo
- I pesi delle connessioni verso tali neuroni
- La derivata della funzione di attivazione del neurone corrente

Questo giustifica l'uso dell'indice $l + 1$: nel calcolo dell'errore locale di uno strato, si utilizzano le informazioni provenienti dallo strato topologicamente successivo, ma già aggiornate nel passo di propagazione all'indietro.

In questo modo, il segnale di errore fluisce dall'uscita verso l'ingresso, permettendo di aggiornare progressivamente tutti i pesi della rete, anche in assenza di una ground truth negli strati intermedi.

7.8.3 3. Variazione della Loss rispetto a θ di apprendimento

Nel gradiente discendente è necessario calcolare la derivata della funzione di costo rispetto ai pesi. Nello schema del perceptrone, i pesi hanno al loro interno anche θ , così come nella trattazione precedente. Abbiamo affermato che:

$$w(n+1) = w(n) + \Delta w(n) \quad \text{con} \quad \Delta w(n) = \eta \cdot \frac{\partial L}{\partial w} = \eta \cdot e \cdot x$$

Distinguo tra l'aggiornamento di θ al passo n e l'aggiornamento del peso $\Delta w_{jk}^l(n)$ al passo n :

$$\Delta \theta_j^l(n) = \eta \cdot \frac{\partial L}{\partial \theta_j^l} \quad (7.34)$$

$$\Delta w_{jk}^l(n) = \eta \cdot \frac{\partial L}{\partial w_{jk}^l} \quad (7.35)$$

Calcoliamo in un primo momento la (7.34). Ricordiamo che:

$$\delta_j^l = \frac{\partial L}{\partial v_j^l}$$

Si può riscrivere come:

$$\delta_j^l = \sum_k \frac{\partial L}{\partial \theta_k^l} \frac{\partial \theta_k^l}{\partial v_j^l} = \frac{\partial L}{\partial \theta_j^l} \cdot \frac{\partial \theta_j^l}{\partial v_j^l} \quad (7.36)$$

Sapendo che:

$$v_j^l = \sum_k w_{jk}^l y_k^{l-1} + \theta_j^l$$

La derivata è pari a:

$$\frac{\partial \theta_j^l}{\partial v_j^l} = \frac{\partial [v_j^l - \sum_k (w_{jk}^l y_k^{l-1} + \theta_j^l)]}{\partial v_j^l} = 1 - 0 = 1$$

Quindi la (7.36) è pari a:

$$\delta_j^l = \frac{\partial L}{\partial \theta_j^l} \quad (7.37)$$

L'aggiornamento del θ sullo strato l è pari ad un rate of learning per quella quantità.

7.8.4 4. Variazione della Loss rispetto ai pesi w_{jk}

Si calcola:

$$\frac{\partial L}{\partial w_{jk}^l}$$

Considerando:

$$\delta_j^l = \frac{\partial L}{\partial v_j^l}$$

Si può riscrivere:

$$\begin{aligned}\partial_j^l &= \sum_k \frac{\partial L}{\partial w_{jk}^l} \frac{\partial w_{jk}^l}{\partial v_j^l} = \\ &= \underbrace{\frac{\partial L}{\partial w_{jk}^l}}_{\text{isolo}} \frac{\partial w_{jk}^l}{\partial v_j^l}\end{aligned}$$

Isolando il termine evidenziato:

$$\frac{\partial L}{\partial w_{jk}^l} = \delta_j^l \cdot \frac{\partial v_j^l}{\partial w_{jk}^l} \quad (7.38)$$

$$\frac{\partial v_j^l}{\partial w_{jk}^l} = \frac{\partial [\sum_k w_{jk}^l \cdot y_k^{l-1} + \theta_j^l]}{\partial w_{jk}^l} = y_k^{l-1}$$

La (7.38) è quindi pari a:

$$\frac{\partial L}{\partial w_{jk}^l} = \delta_j^l \cdot y_k^{l-1} \quad (7.39)$$

Per calcolare quindi la variazione della loss rispetto ai pesi si considera l'uscita del layer precedente e la si moltiplica per l'errore locale, noto dall'equazione (7.33).

7.9 Due funzioni di perdita: Cross entropy e Softmax

Dopo aver visto la funzione di perdita basata sull'errore quadratico, questa sezione presenta due funzioni di perdita che permettono di migliorare il processo di apprendimento della rete neurale.

7.9.1 Cross-Entropy

Abbiamo visto che nelle dimostrazioni precedenti nel calcolo dell'errore locale compare il termine di ψ' che può essere un problema nell'ipotesi di funzioni non derivabili o con asintoto orizzontale per $v \rightarrow \infty$ limitando l'apprendimento della rete neurale. Per ovviare a questo problema si usa una funzione di apprendimento che non è più l'MSE, ma la **cross-entropy**. Si usa in problemi di **classificazione** ed è definita come la sommatoria su tutti i campioni m :

$$L = -\frac{1}{m} \sum_{t=1}^m [d_t \ln y_t + (1 - d_t) \ln(1 - y_t)] \quad (7.40)$$

Si può dimostrare che la derivata della loss rispetto al peso j -esimo non dipende da ψ' eliminando alla radice il problema della **saturazione**.

7.9.2 Softmax

Nel caso di problemi di classificazione multiclasse, è utile interpretare l'uscita della rete come una distribuzione di probabilità sulle classi. Per ottenere questa proprietà, si utilizza la funzione **softmax**.

L'attività interna dei neuroni viene calcolata come di consueto:

$$v_j^L = \sum_k w_{jk}^L y_k^{L-1} + \theta_j^L$$

L'uscita del neurone j è quindi definita come:

$$y_j^L = \frac{e^{v_j^L}}{\sum_k e^{v_k^L}} \quad (7.41)$$

1. Proprietà di normalizzazione

La softmax produce automaticamente una distribuzione di probabilità. Infatti:

$$\sum_{j=1}^p y_j^L = \sum_{j=1}^p \frac{e^{v_j^L}}{\sum_{k=1}^p e^{v_k^L}} = \frac{\sum_{j=1}^p e^{v_j^L}}{\sum_{k=1}^p e^{v_k^L}} = 1$$

Inoltre, essendo l'esponenziale sempre positivo:

$$0 \leq y_j^L \leq 1$$

2. Monotonia

La funzione softmax è monotona rispetto alle attivazioni v_j^L : se v_j^L aumenta, anche y_j^L aumenta. In particolare, l'esponenziale amplifica le differenze tra le attivazioni, rendendo più marcata la classe con valore massimo. Se aumenta l'uscita di un neurone, diminuisce l'uscita degli altri.

In un neurone softmax la funzione *loss* che si usa è detta **verosimiglianza logaritmica** ed è definita come:

$$L = -\ln y_j^L \quad (7.42)$$

Si può dimostrare che non dipende da ψ' .

8 Autoencoder

Nell'analisi di dati complessi, l'estrazione di feature robuste è un aspetto fondamentale. In molti casi, le relazioni tra le variabili non sono lineari e quindi una semplice proiezione lineare non è sufficiente a rappresentare adeguatamente la struttura dei dati. I dati reali presentano spesso dipendenze complesse, ed è quindi necessario disporre di modelli in grado di apprendere **trasformazioni non lineari** dello spazio delle feature.

In questo contesto, le reti neurali risultano particolarmente efficaci, in quanto sono in grado di modellare relazioni non lineari tra i dati. La non linearità viene introdotta attraverso le **funzioni di attivazione** ψ , applicate all'uscita di ciascun neurone.

Consideriamo una rete neurale composta da N layer. Se non si applicassero funzioni di attivazione, ogni neurone eseguirebbe una combinazione lineare degli ingressi. Di conseguenza, l'intera rete si ridurrebbe a una composizione di trasformazioni lineari. Tuttavia, la composizione di più trasformazioni lineari è ancora una trasformazione lineare. Pertanto, una rete neurale priva di funzioni di attivazione non lineari sarebbe equivalente a un singolo modello lineare, indipendentemente dal numero di layer.

La capacità delle reti neurali di apprendere relazioni complesse deriva quindi esclusivamente dalla **presenza delle funzioni di attivazione non lineari**, che permettono di costruire rappresentazioni sempre più articolate dei dati attraverso gli strati della rete.

Nota questa proprietà, è naturale porsi la domanda se sia possibile sfruttare una struttura a rete neurale per effettuare *dimensionality reduction* in dataset che presentano correlazioni non lineari tra le feature.

In questo contesto entrano in gioco gli **autoencoder**, un algoritmo di feature selection il cui obiettivo è ridurre la dimensionalità del dato, massimizzando l'informazione contenuta in una rappresentazione a dimensionalità più bassa. Si considera un vettore di input x , contenente le feature, che viene fornito in ingresso a un autoencoder, il quale produce in uscita una stima $\hat{x}$:

$$x \rightarrow \boxed{\text{AE}} \rightarrow \hat{x}$$

Non viene utilizzata una variabile target y per effettuare la dimensionality reduction.

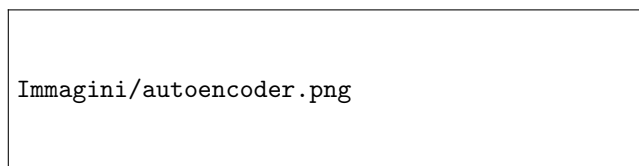


Figura 8.1: Struttura di un autoencoder

Il **modulo encoder** è una rete neurale che, dato l'input x , produce un vettore:

$$h = f(x) \tag{8.1}$$

detto **spazio latente**, caratterizzato da una dimensionalità più piccola rispetto all'input. L'**encoder** ha quindi il compito di mappare il dato originale in una rappresentazione più compatta.

Successivamente, una seconda rete neurale, il **decoder**, riceve in ingresso il vettore h e cerca di ricostruire un output $\hat{x}$ il più possibile simile all'input originale. La *loss function* è:

$$L = \hat{x} - x \quad (8.2)$$



Figura 8.2: Schema di funzionamento di un autoencoder: l'input viene codificato in uno spazio latente a dimensionalità ridotta (encoding) e successivamente ricostruito (decoding)

E' un algoritmo **self-supervised**, la ground through da minimizzare è proprio l'input x e non l'uscita y .

8.1 Overcomplete hidden layers

Un autoencoder con una dimensione dello strato nascosto superiore a quella dell'input, come mostrato in Figura 8.3, è definito **overcomplete**.



Figura 8.3: Architettura di un autoencoder overcomplete

Nel processo di addestramento, l'obiettivo principale della rete è minimizzare l'errore di ricostruzione, facendo in modo che l'output $\hat{x}$ sia il più fedele possibile all'input x . Tuttavia, avendo a disposizione uno **spazio latente** di dimensioni maggiori rispetto allo spazio di partenza, la rete rischia di risolvere il problema semplicemente imparando la funzione identità, ovvero *copiando* l'informazione x in $\hat{x}$ in modo banale.

A differenza delle architetture **undercomplete**, che forzano una riduzione della dimensionalità per estrarre le caratteristiche salienti, un autoencoder overcomplete non garantisce la scoperta di feature utili solo minimizzando l'errore.

8.2 Sparse Autoencoder

Durante la fase di addestramento, un determinato numero di neuroni nello strato nascosto viene disattivato in modo casuale, con una configurazione che varia a ogni epoca.

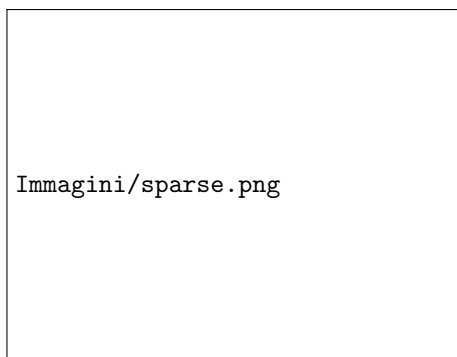
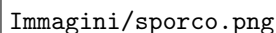


Figura 8.4: Rappresentazione di un autoencoder con disattivazione casuale dei neuroni nell'hidden layer durante una singola epoca di addestramento.

Grazie a questo meccanismo, i neuroni non possono fare affidamento su specifiche connessioni per la ricostruzione del dato. Di conseguenza, si impedisce alla rete di limitarsi a memorizzare o "copiare" passivamente l'informazione di input (evitando l'apprendimento della funzione identità). Il modello è quindi costretto a estrarre rappresentazioni latenti più robuste, distribuite e intrinsecamente più informative.

8.3 Denoising Autoencoders

Il paradigma degli autoencoder può essere esteso e applicato con successo alla rimozione del rumore dai dati (*denoising*). L'obiettivo, in questo caso, è addestrare un modello neurale che, ricevendo in input un'immagine corrotta da rumore, sia in grado di ricostruirne la versione originale intatta.



Immagini/sporco.png

Figura 8.5: Processo di pulizia di un'immagine tramite un Denoising Autoencoder.

La preparazione del dataset avviene in modo controllato. A partire da un set di immagini originali integre, viene applicato artificialmente un rumore stocastico (utilizzando funzioni di alterazione specifiche, come ad esempio il rumore *salt-and-pepper* o il rumore gaussiano) per generare le corrispondenti versioni corrotte, $\tilde{x}$.

Durante il processo di apprendimento, l'autoencoder riceve in input il dato alterato $\tilde{x}$, ma l'obiettivo della rete (la minimizzazione dell'errore) viene calcolato confrontando l'output generato $\hat{x}$ con l'immagine originale "pulita" x . Poiché la rete deve annullare l'errore di ricostruzione rispetto al target integro, il modello è costretto a non copiare semplicemente l'input, ma ad apprendere la struttura fondamentale e le dipendenze semantiche del dato, filtrando in modo efficace le perturbazioni casuali.

8.4 Stacked Autoencoders

Ci possono essere più layer di encoding o decoding e strutture non simmetriche. Questi paradigmi prendono il nome di **stacked encoder**:

Immagini/stacked.png

9 Support Vector Machine (SVM)

La Support Vector Machine richiede che l'uscita sia:

$$y = \{-1; 1\} \quad , \quad (9.1)$$

quindi può trattare solamente problemi **binari**. Inoltre:

$$x \in \mathbb{R}^D \quad , \quad (9.2)$$

ovvero le feature possono essere solonumeriche. La **frontiera** che essa costruisce è un'equazione del tipo:

$$\underline{\omega}^\top \underline{x} + \underline{b} = 0 \quad , \quad (9.3)$$

e quindi l'SVM costruisce solo **frontiere lineari**, come il perceptrone e il bayesiano. L'uscita stimata $\hat{y}$ è della forma:

$$\hat{y} = g(z) \quad \text{con} \quad z = \underline{\omega}^\top \underline{x} + \underline{b}$$

Gli $\underline{\omega}$ sono i parametri che devono essere appresi. La support vector machine impone quindi che:

$$g(z) = \begin{cases} +1 & z \geq 0 \\ -1 & z < 0 \end{cases}$$

Consideriamo uno spazio in $\mathbb{R}^2$ supponendo di costruire un dataset come in Figura

Il problema è linearmente separabile, può essere tracciata una delle infinite rette che esistono che separa il problema in due porzioni. Ci si chiede quindi come inserire la frontiera lineare. Preso un valore di ω e b , ovvero i parametri incogniti, considerando:

$$\omega \cdot x + b \quad ,$$

se l'uscita è positiva, allora sono nell'iperpiano dei positivi (crocette marroni), altrimenti nel piano dei *cerchi rosa* (Figura). Il dataset è formato dalle coppie:

$$(\underline{x}_i, \underline{y}_i) \quad \text{feature - ground truth}$$

10 Clustering

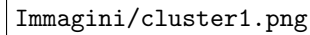
Il **clustering** è una tecnica fondamentale dell'apprendimento non supervisionato il cui scopo è l'analisi e il raggruppamento di un insieme di dati non etichettati. A differenza dei paradigmi supervisionati, il clustering non si basa su una relazione predefinita tra variabili di input x e target y , ma opera esclusivamente sulla struttura intrinseca e sulla distribuzione spaziale dei campioni x . Per comprendere meglio il ruolo del clustering, è utile confrontarlo con le altre categorie principali di algoritmi:

- **Classificazione (Supervisionata):** L'obiettivo è identificare una *frontiera di decisione* ottimale che permetta di separare i dati in classi discrete note a priori, minimizzando l'errore di assegnazione
- **Regressione (Supervisionata):** L'algoritmo modella la relazione tra le variabili per predire un valore numerico continuo relativo a un nuovo campione, basandosi sulla distribuzione della popolazione nota
- **Clustering (Non Supervisionato):** L'obiettivo è la scoperta di pattern latenti. L'algoritmo aggrega i campioni in sotto-insiemi (denominati *cluster*) in modo tale che gli elementi appartenenti allo stesso gruppo presentino un'elevata somiglianza tra loro, mentre elementi appartenenti a gruppi diversi risultino il più possibile dissimili

Mentre nella regressione e nella classificazione cerchiamo una funzione $f(x) = y$, nel clustering l'attenzione è rivolta esclusivamente all'analisi delle caratteristiche dei dati per far emergere una struttura organizzativa coerente senza una guida esterna.

In questo contesto, l'interpretazione dei cluster non è nota a priori, ma emerge come analisi *ex-post* una volta concluso il processo di raggruppamento. L'algoritmo opera infatti in modo agnostico rispetto al significato semantico dei dati, aggregando i campioni esclusivamente sulla base della loro similarità intrinseca e della loro distribuzione nello spazio delle feature, senza alcuna conoscenza preliminare delle loro caratteristiche distintive. Il valore in sé al cluster di appartenenza non ha alcun significato, non è necessario che sia una variabile numerica, deve essere effettuato il *one hot encoding* per mostrare se esiste o no l'appartenenza a singolo gruppo.

In base a come sono distribuite le x , l'output potrebbe cambiare e quindi i raggruppamenti, che dipendono dalla distribuzione delle x .

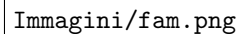


Immagini/cluster1.png

Aumentando la numerosità dei campioni, il modo con cui vengono raggruppati potrebbe cambiare completamente.

10.1 Nomenclatura dei metodi di clustering

In base al paradigma dell'algoritmo, i metodi di clustering vengono divisi in famiglie differenti:



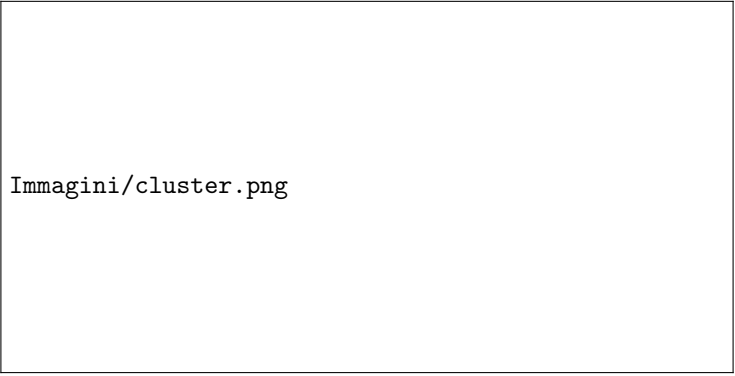
Immagini/fam.png

Figura 10.1: Famiglie di algoritmi di clustering

In base al principio con cui il paradigma di raggruppamento viene utilizzato, l'algoritmo appartiene ad un metodo diverso.

- **Gerarchico:** ordina le informazioni note
- **Partizione:** si basa sulla distribuzione di sottogruppi di dati
- **Grid based:** usa una griglia noti i range di valori delle feature
- **Nets:** sfruttano il deep learning
- **Big Data:** utilizzate per dati che hanno un numero di istanze nell'ordine delle migliaia, i quali vengono semplificati.

In particolare saranno di nostro interesse i metodi segnati in rosso in Figura:



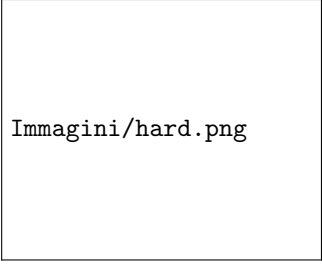
Immagini/cluster.png

Per ogni algoritmo di clustering è necessario individuare una specifica **funzione obiettivo** (*goal*) che ne guidi il comportamento. In base alla modalità con cui vengono considerate le variabili, gli algoritmi si distinguono in:

- **Monotetici:** il raggruppamento avviene isolando una singola o poche caratteristiche, richiedendo che tutti gli elementi di un cluster condividano esattamente tali attributi
- **Politetici:** il processo di raggruppamento sfrutta l'intera multi dimensionalità del campione, tutte le sue feature, calcolando la somiglianza globale tra gli elementi

Un algoritmo di clustering può essere classificato in base alla **granularità dell'appartenenza** (o *durezza*):

- **Hard Clustering:** i confini tra i gruppi sono netti e non sovrapponibili; ogni campione viene assegnato univocamente a uno e un solo cluster e i cluster non si sovrappongono mai:



Immagini/hard.png

Figura 10.2: Esempio di Hard Clustering (partizione netta)

- **Soft Clustering** (o *Fuzzy*): l'appartenenza non è univoca; a ogni punto viene associato un grado di probabilità o un coefficiente di *membership* per ogni cluster individuato. I cluster si sovrappongono reciprocamente:

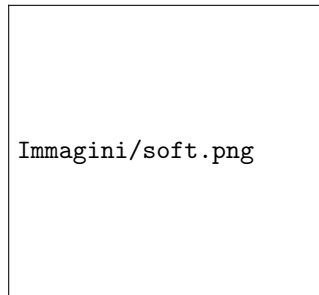


Figura 10.3: Esempio di Soft Clustering (appartenenza probabilistica)

Un'ulteriore distinzione riguarda la **struttura topologica** (o *connettività*) dei raggruppamenti:

- **Flat Clustering:** l'algoritmo restituisce una singola partizione dei dati in un numero prefissato di cluster, senza alcuna relazione gerarchica tra di essi.

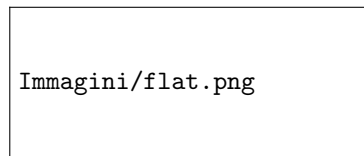


Figura 10.4: Esempio di Flat Clustering (singolo livello).

- **Hierarchical Clustering:** i raggruppamenti sono organizzati in una struttura ad albero (*dendrogramma*); ogni cluster può contenere singoli campioni o sottogruppi di livello inferiore.

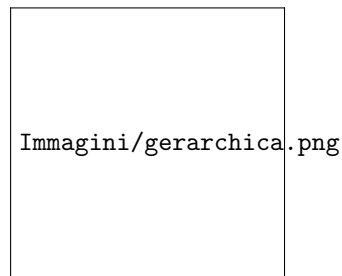
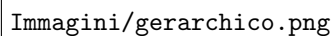


Figura 10.5: Esempio di Clustering Gerarchico (struttura a nido).

10.2 Clustering gerarchico (hierarchical)

Consideriamo un metodo di clustering con una **connettività** di tipo gerarchica. L'obiettivo è, data in input una distribuzione di punti, effettuare un raggruppamento.



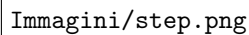
Immagini/gerarchico.png

Il clustering gerarchico può essere di tipo agglomerativo o divisivo.

10.2.1 Clustering gerarchico agglomerativo

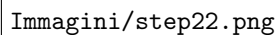
Consideriamo una distribuzione di N punti. L'algoritmo di clustering gerarchico agglomerativo procede secondo i seguenti passi:

1. Ogni punto è inizialmente considerato come un cluster distinto. Si parte quindi da N cluster:



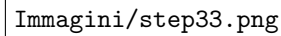
Immagini/step.png

2. Si individuano i due punti (cluster) più vicini e li si unisce in un unico cluster. Il numero totale di cluster diventa $N - 1$.



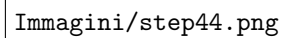
Immagini/step22.png

3. Si ripete il processo unendo, a ogni iterazione, i due cluster più vicini, ottenendo progressivamente $N - 2$, $N - 3$, ... cluster.



Immagini/step33.png

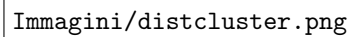
4. Il procedimento continua fino a ottenere un unico cluster che contiene tutti i punti.



Immagini/step44.png

Nel passo di unione tra due cluster è necessario definire una misura di distanza. Una possibile scelta è la **distanza tra centroidi**, in cui ciascun cluster è rappresentato dal proprio centroide e la distanza è calcolata tra tali punti medi. Esistono tuttavia diverse modalità per definire la distanza tra cluster:

- **Distanza minima:** distanza minima tra tutte le coppie di punti appartenenti ai due cluster



Immagini/distcluster.png

Figura 10.6: Distanza minima tra campioni di cluster differenti

- **Distanza massima:** distanza massima tra tutte le coppie di punti appartenenti ai due cluster

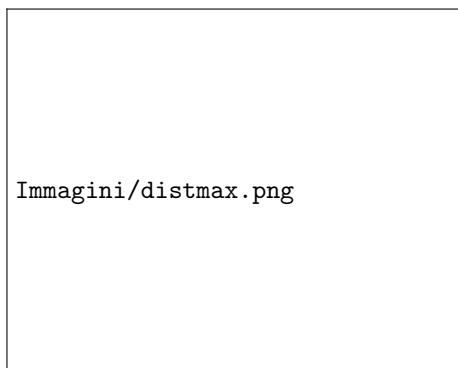


Figura 10.7: Distanza massima tra campioni di cluster differenti

- **Distanza media:** media delle distanze tra tutte le coppie di punti dei due cluster

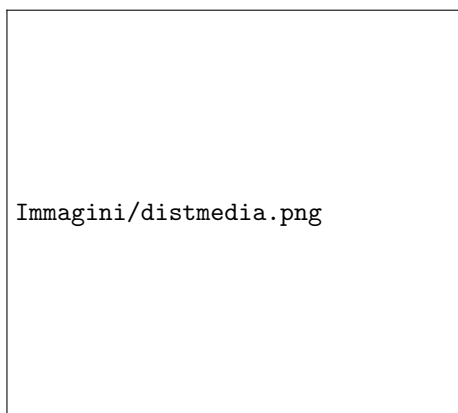


Figura 10.8: Distanza media tra tutte le coppie di campioni di cluster differenti

- **Distanza tra centroidi:** distanza tra i centroidi dei due cluster

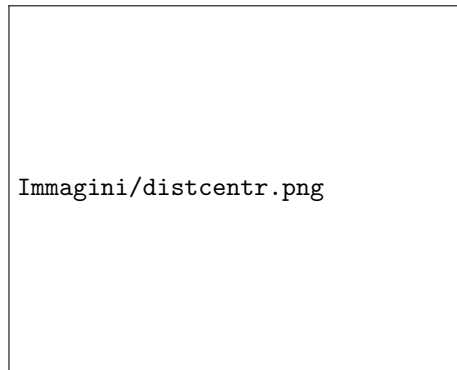
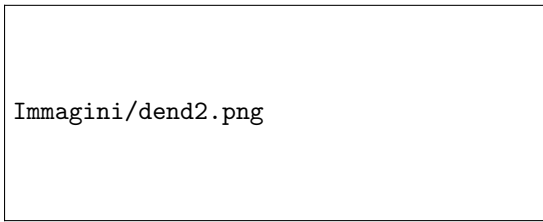
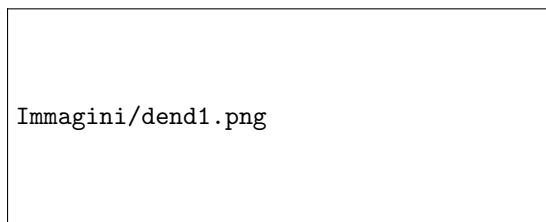


Figura 10.9: Distanza tra i centroidi dei cluster

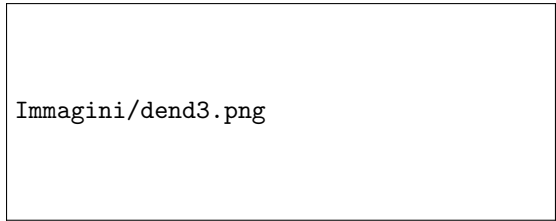
Si osserva che, nel caso di distanza minima, massima e media, è necessario calcolare tutte le distanze tra le coppie di punti appartenenti ai due cluster, con un onere computazionale elevato. Al contrario, nel caso della distanza tra centroidi, è sufficiente calcolare la distanza tra due punti (i centroidi), rendendo il metodo molto più efficiente. La scelta della metrica di distanza rappresenta un **iperparametro** dell'algoritmo: diverse scelte possono portare a differenti strutture di clustering.

L'obiettivo dell'algoritmo di clustering gerarchico è costruire un grafico, detto **dendrogramma**, che consente di interpretare il processo di aggregazione dei dati. Riprendendo l'esempio precedente, si considerano 6 campioni, quindi inizialmente 6 cluster distinti, che vengono rappresentati sull'asse x del grafico:



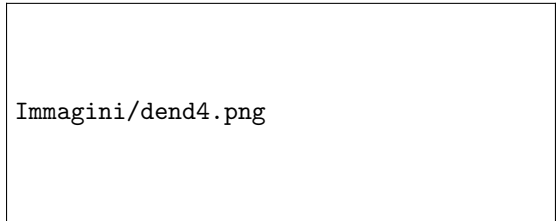
La prima aggregazione, tra P_2 e P_3 , viene rappresentata con un "gradino" la cui altezza corrisponde alla distanza tra i due cluster:

In modo analogo, si rappresenta l'aggregazione tra P_5 e P_6 :



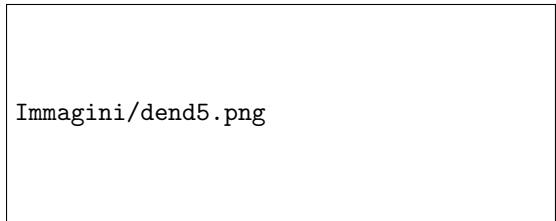
Immagini/dend3.png

Successivamente, il punto P_1 viene aggregato al cluster formato da P_2 e P_3 . Anche in questo caso si introduce un gradino, la cui altezza rappresenta la distanza tra i due cluster:

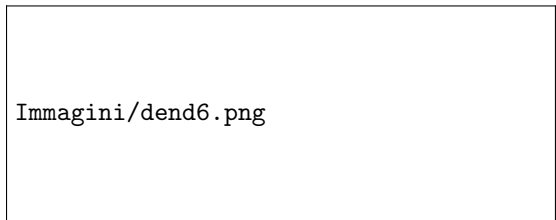


Immagini/dend4.png

Proseguendo con il processo iterativo di aggregazione si ottengono le seguenti configurazioni intermedie:



Immagini/dend5.png



Immagini/dend6.png

Il risultato finale del processo è il dendrogramma mostrato in Figura 10.10:

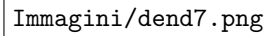
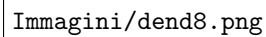
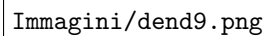
The image is a placeholder for a dendrogram, which is a tree diagram illustrating the hierarchical clustering of data points. The text 'Immagini/dend7.png' is centered within the image box.

Figura 10.10: Dendrogramma dell'algoritmo di clustering agglomerativo dell'esempio

Il dendrogramma consente di analizzare la struttura dei dati. Tracciando una retta orizzontale a una certa altezza, il numero di intersezioni con i rami del dendrogramma corrisponde al numero di cluster individuati:

The image is a placeholder for a dendrogram, which is a tree diagram illustrating the hierarchical clustering of data points. The text 'Immagini/dend8.png' is centered within the image box.

La scelta dell'altezza della retta è in generale arbitraria; una scelta comune consiste nel considerare una quota intermedia rispetto alla distanza massima osservata nel dendrogramma.

The image is a placeholder for a dendrogram, which is a tree diagram illustrating the hierarchical clustering of data points. The text 'Immagini/dend9.png' is centered within the image box.

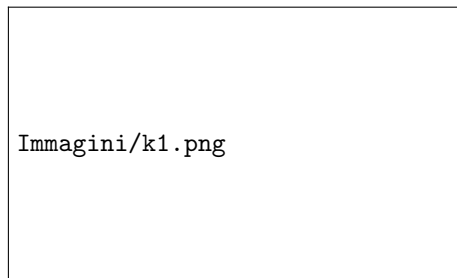
Il metodo è detto **agglomerativo** perché parte da una configurazione iniziale in cui ogni punto rappresenta un cluster distinto e procede unendo progressivamente i cluster più vicini, fino ad arrivare a un **unico cluster** che contiene tutti i dati.

Questo approccio si contrappone al metodo **divisivo**, che segue una logica opposta: si parte da un unico cluster contenente tutti i punti e si procede suddividendolo iterativamente in sottocluster sempre più piccoli.

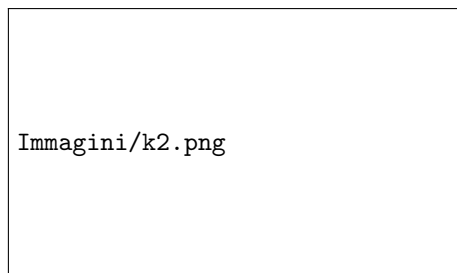
10.3 K-Means Clustering

A differenza del clustering gerarchico dove il numero di cluster lo scelgo a posteriori, il numero di cluster che ottengo nel K-means lo scelgo a priori ed è il mio **iperparametro**. Consideriamo un esempio dove vogliamo capire come la popolazione si raggruppa in due cluster. Seguendo l'esempio, l'algoritmo procede con i seguenti step:

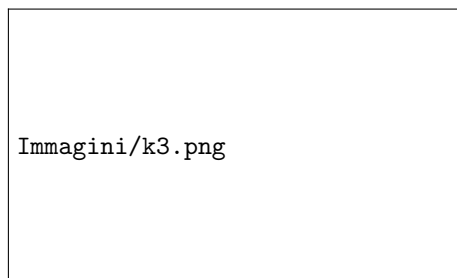
- Si scelgono N punti casuali, nel nostro esempio due, che saranno i centroidi del primo e secondo cluster.



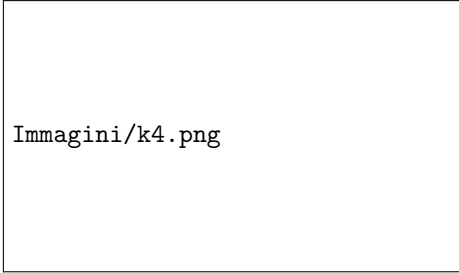
- Assegno ai cluster i punti più vicini ad esso, come se stessi tracciando la retta di equidistanza che divide la retta di congiunzione tra i due centroidi e ne stessi assegnando i campioni al centroide blu e rosso



- Ricalcolo il centroide in base all'assegnazione:

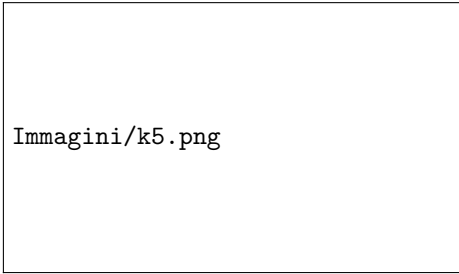


Il centroide si sposta da una posizione iniziale randomica ad una posizione centrale ai punti:



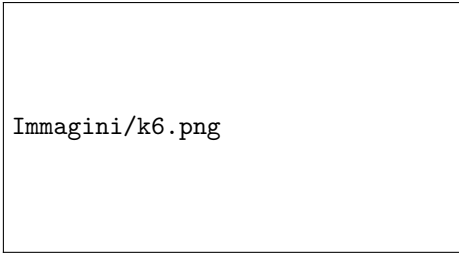
Immagini/k4.png

- Noti i nuovi centroidi, l'algoritmo viene continuato, ricalcolando la retta di equidistanza e riassegnando i punti ai centroidi.

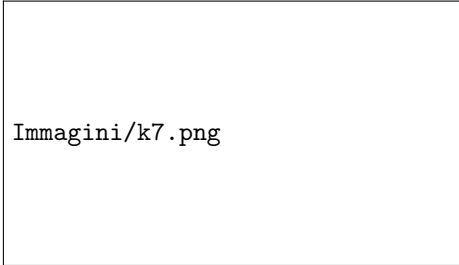


Immagini/k5.png

- Si ricalcola il centroide a partire dalla nuova assegnazione:



Immagini/k6.png



Immagini/k7.png

- Il processo iterativo continua fino a **convergenza**, fino a quando non si ha riassegnazione di cluster. La soluzione dell'esempio è riportata in Figura

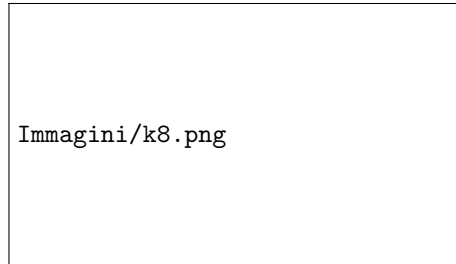
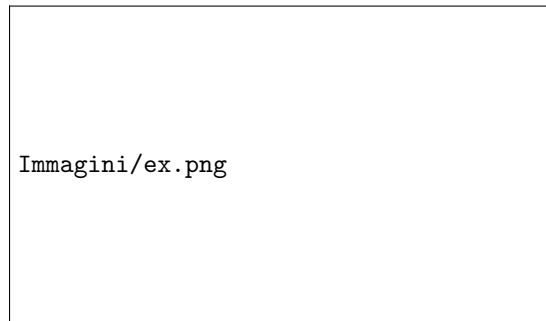


Figura 10.11: Soluzione dell'esempio del K-means, specificando 2 cluster

L'algoritmo per come è costruito, ha il limite di avere un punto di partenza casuale che può portare a convergenza per soluzioni differenti ed inoltre, scegliendo il numero di cluster a priori, non si conosce il numero di cluster ottimale, per questo esiste l'elbow method.

10.3.1 Elbow method

Si consideri una distribuzione di dati di esempio:

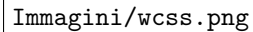


In generale, il numero ottimale di cluster non è noto a priori. Per stimarlo si utilizza la WCSS (*Within Cluster Sum of Squares*):

$$\text{WCSS} = \sum_{P_i \in \text{Cluster 1}} \text{distance}(P_i, C_1)^2 + \sum_{P_i \in \text{Cluster 2}} \text{distance}(P_i, C_2)^2 + \dots \quad (10.1)$$

La (10.1) rappresenta la somma delle distanze quadratiche tra ciascun punto e il centroide del cluster a cui appartiene, estesa a tutti i cluster. L'obiettivo è ottenere valori di WCSS bassi, in modo che i punti appartenenti a uno stesso cluster risultino il più possibile vicini al proprio centroide, indicando una buona compattezza dei cluster.

Per applicare il metodo, si esegue l'algoritmo K -means per diversi valori di K e si calcola, per ciascun caso, il corrispondente valore di WCSS. All'aumentare di K , il valore della WCSS tende a diminuire, poiché i cluster diventano più piccoli e meglio adattati ai dati. I valori ottenuti vengono quindi rappresentati in un grafico in funzione di K . Il criterio dell'**elbow method** consiste nell'individuare il punto in cui la riduzione della WCSS cambia significativamente andamento, formando un "gomito" nella curva.

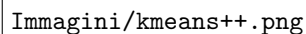


Immagini/wcss.png

Il valore di K corrispondente a tale punto rappresenta un buon compromesso: la WCSS è sufficientemente bassa, ma ulteriori aumenti di K porterebbero a miglioramenti marginali.

10.4 K-Means ++

L'algoritmo K-means presenta una criticità legata all'inizializzazione randomica dei centroidi. Questa casualità può portare a risultati differenti a parità di dati, generando cluster anche molto diversi tra loro.



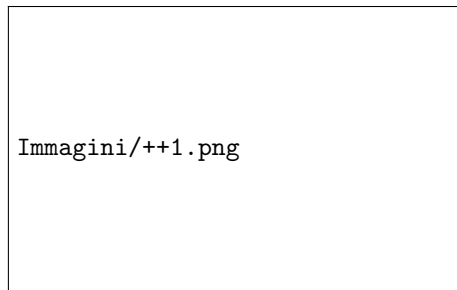
Immagini/kmeans++.png

L'obiettivo è ridurre questa variabilità e ottenere soluzioni più stabili. Il **K-Means ++** rappresenta un'evoluzione del K-means classico, introducendo una

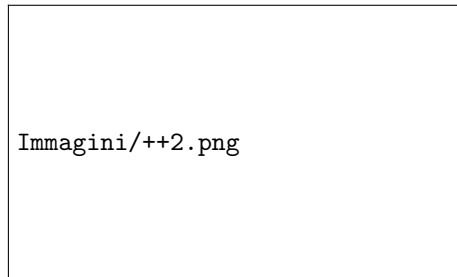
strategia di inizializzazione più strutturata. Pur non garantendo sempre la stessa soluzione, riduce significativamente la variabilità dei risultati.

Si consideri, ad esempio, il caso con $K = 3$. L'algoritmo procede secondo i seguenti passi:

1. Il primo centroide viene scelto in maniera completamente casuale tra i campioni:

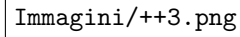


2. Il secondo centroide viene selezionato in modo probabilistico, dando maggiore peso ai punti più distanti dal primo centroide. Per ogni punto si calcola la distanza D dal centroide già scelto e si utilizza D^2 come peso di selezione:



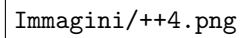
La distanza al quadrato consente di enfatizzare i punti più lontani, aumentando la probabilità di selezionare centroidi ben separati. Questo può comportare al rischio di selezionare un outlier come centroide, generando un cluster isolato che non attrae altri punti e che quindi risulta poco significativo.

3. Il terzo centroide (e in generale i successivi) viene scelto con lo stesso criterio, considerando la distanza quadratica rispetto ai centroidi già selezionati:



Immagini/++3.png

Una volta inizializzati tutti i centroidi, si applica il classico algoritmo **K-means**:



Immagini/++4.png

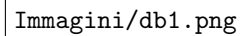
L'**onere computazionale** del K-Means ++ è maggiore rispetto al K-means, proprio a causa del processo di inizializzazione più complesso.

10.5 DBSCAN

DBSCAN è un algoritmo di clustering basato sulla densità, in grado di individuare cluster di forma arbitraria e di riconoscere gli outlier. Si definiscono due iperparametri:

- Un raggio r (epsilon), che definisce il vicinato di un punto
- Un numero minimo di punti l (*minPts*), necessario per definire una regione densa

Consideriamo un esempio:

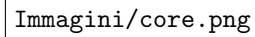


Immagini/db1.png

L'algoritmo procede come segue:

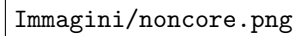
1. Per ogni punto del dataset si verifica se è:

- **Core:** se il numero di punti nel suo vicinato (di raggio r) è maggiore o uguale a l :



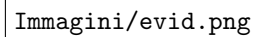
Immagini/core.png

- **Non core:** se il numero di punti nel vicinato è inferiore a l :



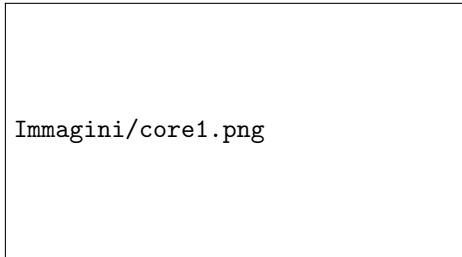
Immagini/noncore.png

2. Si identificano quindi tutti i punti core e non core:



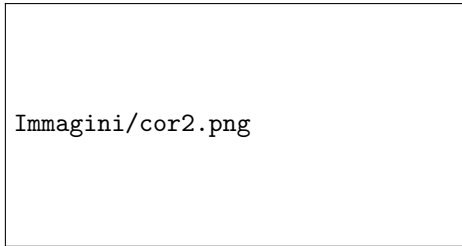
Immagini/evid.png

3. Si seleziona un punto core non ancora assegnato e si avvia la costruzione di un nuovo cluster:



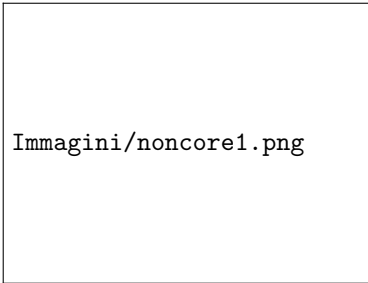
Immagini/core1.png

4. Tutti i punti core nel suo vicinato (raggio r) vengono assegnati allo stesso cluster:



Immagini/cor2.png

5. L'esplorazione continua in modo iterativo: si considerano i punti core appena aggiunti e si verifica se nel loro vicinato sono presenti altri punti core da includere nel cluster.
6. Se un punto **non core** ha nel proprio vicinato almeno un punto core appartenente al cluster, viene assegnato a quel cluster:



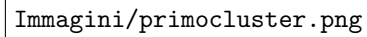
Immagini/noncore1.png

Tuttavia, essendo un punto non core, non genera ulteriore espansione del cluster:



Immagini/stopcore.png

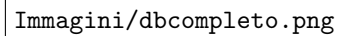
7. Una volta esaurita l'espansione, il cluster è completamente identificato:



Immagini/primocluster.png

8. Si ripete il processo sui punti core non ancora assegnati per individuare eventuali nuovi cluster.

I punti non core che non risultano assegnati ad alcun cluster vengono identificati come **outlier**:



Immagini/dbcompleto.png

Per:

- $r \rightarrow \infty$: tutti i punti appartengono ad un unico cluster
- $r \rightarrow 0$: tutti i punti sono non core e quindi sono tutti outliers

10.6 Indici per valutare i modelli di clustering

Esistono due principali tipologie di indici per valutare la qualità di un risultato di clustering:

- **Indici interni:** misure che valutano la bontà del clustering senza informazioni a priori, basandosi su proprietà intrinseche dei gruppi ottenuti (ad es. compattezza e separazione)
- **Indici esterni:** misure che confrontano la partizione ottenuta dall'algoritmo con una partizione di riferimento (ground truth), quantificando il grado di concordanza tra le due

10.6.1 Silhouette index (interno)

Il *Silhouette Index* è un valore compreso nell'intervallo $[-1, 1]$ e misura quanto un punto è ben assegnato al proprio cluster rispetto agli altri cluster. Valori elevati indicano un buon clustering.

Immagini/silhouette.png

Per ogni punto i si definiscono:

- $a(i)$: distanza media tra il punto i e tutti i punti appartenenti allo stesso cluster (compattezza)
- $b(i)$: minima distanza media tra il punto i e i punti appartenenti a un altro cluster. In particolare, si calcola la distanza media tra i e ciascun cluster $c \neq C_i$, e si prende il valore minimo:

$$b(i) = \min_{c \neq C_i} D(i, c)$$

dove $D(i, c)$ è la distanza media tra i e i punti del cluster c

Il Silhouette Index è definito come:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (10.2)$$

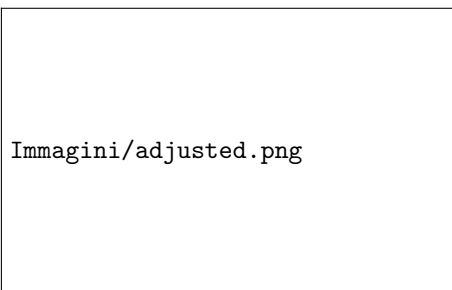
Interpretazione dei valori:

- $s(i) \approx 1$: il punto è ben assegnato al proprio cluster (alta compattezza e buona separazione)
- $s(i) \approx 0$: il punto si trova al confine tra due cluster
- $s(i) \approx -1$: il punto è probabilmente assegnato al cluster sbagliato

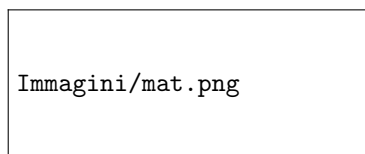
Il procedimento viene effettuato per tutti i punti i appartenenti al cluster.

10.6.2 Adjusted Rand Index (esterno)

L'*Adjusted Rand Index* (ARI) è una misura di similarità tra due partizioni dello stesso dataset, corretta rispetto al caso casuale. In altre parole, valuta quanto due clustering (ad es. quello ottenuto da un algoritmo e quello reale) siano concordi, tenendo conto della similarità attesa per puro caso.



Si considerano due diverse partizioni sugli stessi dati: una derivante dall'algoritmo di clustering e una rappresentante la verità a priori (etichette reali). A partire da queste, si costruisce una matrice di contingenza:



In questa matrice, le righe rappresentano i cluster ottenuti, mentre le colonne rappresentano le classi reali. Ogni elemento indica il numero di punti condivisi tra un cluster e una classe.

Una corrispondenza perfetta tra le due partizioni si riflette in una matrice con valori elevati concentrati lungo una diagonale (non necessariamente quella principale, le etichette dei cluster possono essere permutate).

L'Adjusted Rand Index assume valori nell'intervallo $[-1, 1]$:

- $ARI = 1$: perfetta concordanza tra le due partizioni
- $ARI \approx 0$: concordanza simile a quella ottenuta casualmente
- $ARI < 0$: concordanza peggiore del caso casuale

11 Regressione

Per regressione si intende la predizione di un valore continuo all'interno di un intervallo. Il metodo più semplice per affrontare questo problema è la regressione

lineare, che si basa sulla minimizzazione dell'errore quadratico medio tra i valori osservati e quelli predetti.

La qualità della regressione può essere valutata attraverso diverse metriche. Una di queste è la RSE (*Residual Standard Error*), che misura la dispersione degli scarti tra i valori reali e quelli stimati:

$$\text{RSE} = \sqrt{\frac{1}{n-d-1} \sum_{i=1}^k (y_i - \hat{y}_i)^2} \quad , \quad (11.1)$$

dove n è il numero totale dei campioni e d il numero di variabili esplicative. Un'altra metrica ampiamente utilizzata è il coefficiente di determinazione R^2 , che assume valori compresi tra 0 e 1:

$$R^2 = \frac{\text{TSS} - \text{RSS}}{\text{TSS}} = 1 - \frac{\text{RSS}}{\text{TSS}} \quad , \quad (11.2)$$

dove:

$$\text{TSS} = \sum (y_i - \bar{y})^2 \quad (11.3)$$

$$\text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (11.4)$$

Il coefficiente R^2 esprime la frazione di variabilità dei dati spiegata dal modello. Essendo strettamente legato alla RSS, questa metrica è particolarmente significativa nel caso di regressori lineari.

11.1 Regressione multipla

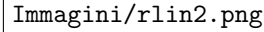
Nel caso di regressione multipla si considerano più variabili esplicative ($d > 1$):

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \epsilon \quad (11.5)$$

dove ϵ rappresenta un **errore non riducibile**, dovuto a fattori non modellati o al rumore presente nei dati. Anche in questo caso la bontà del modello può essere valutata tramite RSE o R^2 , confrontando i valori predetti con quelli reali.

Oltre alla regressione lineare esistono modelli più complessi che introducono relazioni non lineari tra le variabili. Ad esempio, una regressione di secondo grado può essere scritta come:

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_1^2 + \epsilon$$



Tuttavia, anche modelli di grado superiore possono essere ricondotti a una forma lineare attraverso un opportuno cambio di variabile, ad esempio:

$$x_2 = x_1^2$$

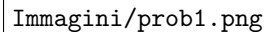
In questo modo, la regressione rimane lineare nei parametri, pur descrivendo relazioni non lineari nelle variabili originali.

11.2 Potenziali problemi di regressione

Esistono diversi problemi che si possono affrontare in condizioni di regressione

11.2.1 Problema della relazione non lineare

Si consideri un grafico in cui sull'asse delle x sono riportati i *fit values*, ovvero le $\hat{y}$ predette dal modello, mentre sull'asse delle y sono riportati i residui, cioè lo scarto tra il valore reale e quello predetto.



In condizioni ideali, i punti dovrebbero distribuirsi attorno alla retta $y = 0$, senza mostrare particolari strutture. Questo indicherebbe che il modello descrive correttamente la relazione tra le variabili. Nel caso della regressione lineare si assume implicitamente che la relazione tra ingressi e uscita sia lineare. Tuttavia, i due grafici mostrati evidenziano comportamenti differenti:

1. Nel primo modello (a sinistra) si osserva che, per valori piccoli delle predizioni, l'errore è elevato e positivo, mentre per valori grandi tende a divergere. La presenza di questa struttura indica che la relazione non è

lineare: il modello fornisce buone predizioni solo in una zona centrale, ma fallisce agli estremi

2. Nel secondo modello (a destra), invece, i residui risultano distribuiti in modo più uniforme attorno allo zero, suggerendo una migliore approssimazione del fenomeno e una relazione più coerente con l'ipotesi lineare.

11.2.2 Errore non riducibile a varianza non costante

L' ϵ di una regressione rappresenta l'errore non riducibile ed è generalmente assunto a varianza costante, ipotesi nota come *omoschedasticità*, ovvero indipendente dal campione considerato.

Tuttavia, questa ipotesi può non essere verificata nella pratica. Un esempio è mostrato in Figura 11.1:

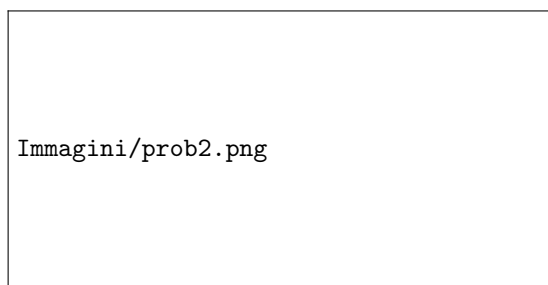


Figura 11.1: Modello a varianza non costante

Nel caso di dispersione uniforme, con una forma simile a un “tubo”, i dati sono compatibili con un errore a varianza costante. Al contrario, una distribuzione a forma di imbuto evidenzia una varianza che dipende dal valore delle predizioni, indicando la presenza di **eteroschedasticità**, cioè un errore non riducibile non costante $\epsilon(x)$.

11.2.3 Outlier

Un **outlier** è un punto i cui valori osservati risultano significativamente lontani rispetto al comportamento medio dei dati, ad esempio rispetto alla media $\bar{y}$ o al modello stimato.

La presenza di outlier può avere diverse cause:

- errori di misura o di acquisizione
- fenomeni rari ma reali
- cambiamenti strutturali nel sistema osservato

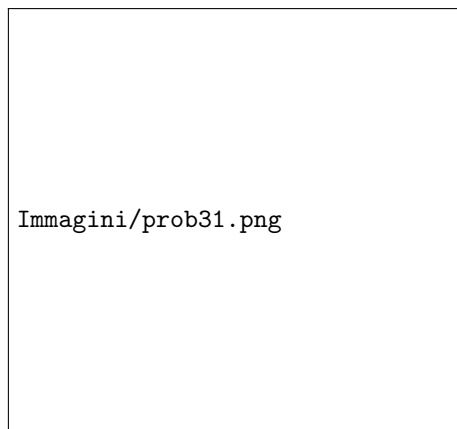
Gli outlier sono particolarmente critici nei modelli di regressione, poiché possono influenzare fortemente la stima dei parametri, alterando la retta (o curva) di regressione.

Un modo efficace per individuare outlier è analizzare il grafico dei **residui**:

$$r_i = y_i - \hat{y}_i$$

costruendo un grafico con:

- asse x : valori predetti $\hat{y}$
- asse y : residui r_i



In questo spazio, un outlier si manifesta come un punto con residuo molto elevato in valore assoluto rispetto agli altri.

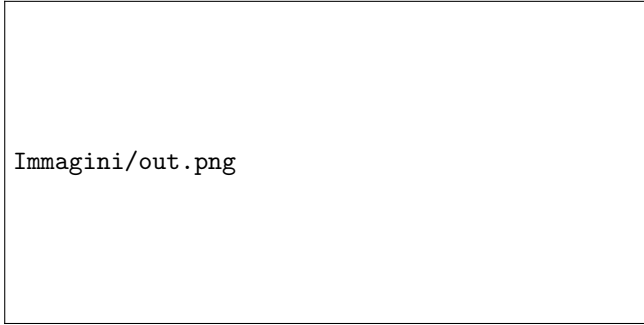
Per una valutazione quantitativa, si considerano spesso i **residui standardizzati**:

$$r_i^* = \frac{r_i}{\sigma_r}$$

dove σ_r è la deviazione standard dei residui. Assumendo una distribuzione approssimativamente gaussiana, si può applicare la regola empirica:

- $|r_i^*| > 2 \Rightarrow$ possibile outlier
- $|r_i^*| > 3 \Rightarrow$ outlier significativo

In particolare, i punti oltre $\pm 3\sigma$ sono considerati outlier, poiché estremamente improbabili sotto ipotesi di normalità dei residui.



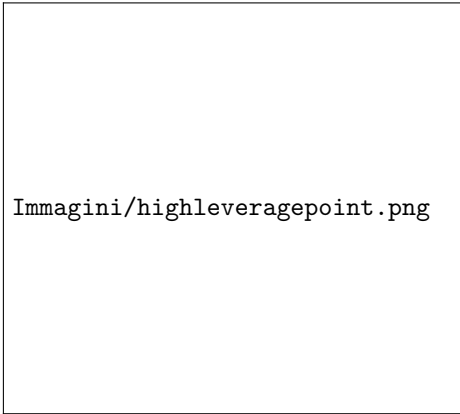
Immagini/out.png

Una volta individuati, è necessario valutare caso per caso se:

- rimuoverli (se dovuti a errori)
- mantenerli (se rappresentano fenomeni reali)
- utilizzare modelli robusti (es. regressione robusta)

11.2.4 High-leverage points

Gli **high-leverage points** sono punti che hanno un forte impatto sulla posizione della funzione di regressione. A differenza degli outlier, che presentano tipicamente un valore di x all'interno del range della distribuzione ma un valore di y fuori scala, un punto di leva è caratterizzato da un valore di x fuori dal range dei dati e spesso anche da un valore di y anomalo. Questo comportamento è illustrato nella seguente figura:



Immagini/highleveragepoint.png

- Il punto 20 è un outlier perché rientra nel range delle x ma non in quello delle y
- Il punto 41 è un high-leverage point perché non rientra né nel range delle x né in quello delle y

Per identificare quantitativamente un high-leverage point si utilizza la **leverage statistic**, definita come:

$$h_i = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum_{i'=1}^n (x_{i'} - \bar{x})^2} \quad (11.6)$$

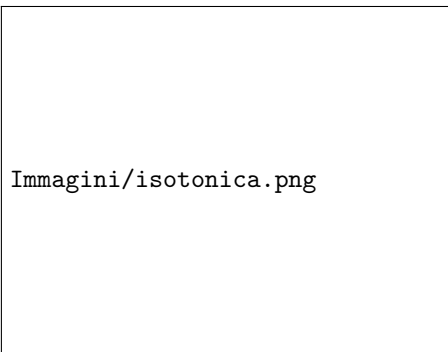
Un punto viene considerato ad alto leverage quando il suo valore h_i è significativamente maggiore del valore medio del leverage.

11.2.5 Collinearità

La collinearità si verifica quando due o più variabili esplicative (regressori) risultano fortemente correlate tra loro. In queste condizioni, diventa difficile distinguere il contributo individuale di ciascuna variabile alla predizione della risposta, con conseguente instabilità nella stima dei parametri e perdita di interpretabilità del modello.

11.3 Regressione isotonica

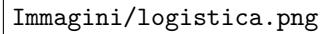
La regressione lineare assume che la relazione tra variabili sia di tipo lineare. La **regressione isotonica** rilassa questa ipotesi, imponendo come unico vincolo che la funzione di regressione sia **monotona crescente**.



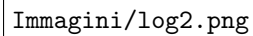
Il principale vantaggio di questo approccio è la rimozione dell'ipotesi sulla forma funzionale del modello: il regressore è libero di adattarsi all'andamento dei dati, purché rispetti la monotonicità. Questo consente una migliore rappresentazione di fenomeni non lineari, mantenendo al contempo una struttura ordinata e coerente con il problema.

11.4 Regressione logistica

Si consideri il caso in cui la variabile y da predire assuma valori compresi tra 0 e 1. Quando le predizioni $\hat{y}$ tendono a concentrarsi in prossimità di 0 o di 1, è possibile interpretare l'uscita del modello in termini probabilistici, riconducendo il problema a una classificazione binaria.

A rectangular box containing the text "Immagini/logistica.png", which serves as a placeholder for an image.

La **regressione logistica** è un modello che utilizza una funzione sigmoide per descrivere la relazione tra le variabili, permettendo di trasformare un problema di classificazione in un problema di regressione. In particolare, il modello produce un output continuo compreso tra 0 e 1, interpretabile come probabilità di appartenenza a una delle due classi.

A rectangular box containing the text "Immagini/log2.png", which serves as a placeholder for an image.

12 Metodi di decomposizione

Abbiamo affermato che la SVM è un classificatore binario, in grado di distinguere tra due classi, tipicamente etichettate come $+1$ e -1 . In questa sezione si affronta il problema di estendere tale approccio a contesti multiclasse. A questo scopo si introducono i **metodi di decomposizione**: un problema di classificazione con più classi viene suddiviso in più problemi binari, che vengono risolti separatamente e successivamente combinati per ottenere la classificazione finale.

Data una feature x , essa viene fornita in ingresso a diversi modelli binari. Il primo modello restituisce un'uscita $M_1(x)$, il secondo un'uscita $M_2(x)$ e così via, ciascuno addestrato su una diversa coppia (o combinazione) di classi.

Immagini/decomp.png

Le diverse uscite dei modelli binari vengono quindi inviate a un modulo di ricostruzione, il cui compito è aggregare i risultati e fornire la classificazione finale nel problema multiclasse. I metodi di decomposizione sono principalmente tre:

1. **Opc** (One per class): noto anche come *one-against-all*
2. **PairWise Coupling (PWC)**: uno contro uno
3. **Error Correcting Output Codes (ECOC)**

12.1 Matrice di decomposizione

La modalità con cui si costruiscono problemi binari a partire da un problema multiclasse prende il nome di **matrice di decomposizione**. Si tratta di una matrice rettangolare caratterizzata da:

- un numero di **righe** pari al numero di classificatori binari che si vogliono costruire
- un numero di **colonne** pari al numero di classi del problema multiclasse

Ogni elemento della matrice può assumere tre valori distinti:

- 0: la classe non viene considerata in quel classificatore
- +1: la classe è assegnata all'etichetta positiva
- -1: la classe è assegnata all'etichetta negativa

Si consideri, ad esempio, la seguente matrice di decomposizione:

$$\begin{pmatrix} 1 & 1 & 1 & 1 & 1 & -1 & -1 \\ -1 & 0 & 0 & 0 & -1 & 1 & 1 \\ 0 & 0 & -1 & 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & -1 & 0 & -1 & 0 \end{pmatrix}$$

Il problema multiclasse originale presenta 7 classi ed è affrontato mediante 4 classificatori binari, corrispondenti alle 4 righe della matrice. Nel passaggio dal

problema multiclasse ai problemi binari, è necessario **ri-etichettare il dataset**. La matrice di decomposizione fornisce esattamente le regole per effettuare questa trasformazione.

Ad esempio, considerando il terzo classificatore (terza riga della matrice), si costruisce un nuovo dataset in cui:

- i campioni appartenenti alle classi 1, 2, 5 e 6 vengono scartati (valore 0)
- i campioni appartenenti alle classi 3 e 7 vengono etichettati come -1
- i campioni appartenenti alla classe 4 vengono etichettati come $+1$

In questo modo, ciascun classificatore binario viene addestrato su una specifica suddivisione delle classi, contribuendo alla costruzione della soluzione finale del problema multiclasse.

12.2 One-per-class (OpC) (One-Against-All)

La matrice di decomposizione di un problema OpC è sempre quadrata: questo metodo costruisce un numero di classificatori pari al numero di classi. In ogni riga della matrice compare un solo valore $+1$, mentre tutti gli altri elementi sono -1 . Ciascun classificatore viene addestrato per distinguere una singola classe rispetto a tutte le altre (approccio one-vs-all).

Assumendo che ogni modello binario produca un'uscita compresa tra -1 e $+1$, si raccolgono tutte le uscite in un vettore e si assegna l'istanza alla classe corrispondente al valore più alto. In questo modo si seleziona il classificatore per cui il modello è più sicuro.

Questo metodo presenta alcuni svantaggi: in presenza di un elevato numero di classi, il numero di classificatori da addestrare cresce notevolmente. Inoltre, anche se il dataset multiclasse è inizialmente bilanciato, i singoli problemi binari generati possono risultare sbilanciati.

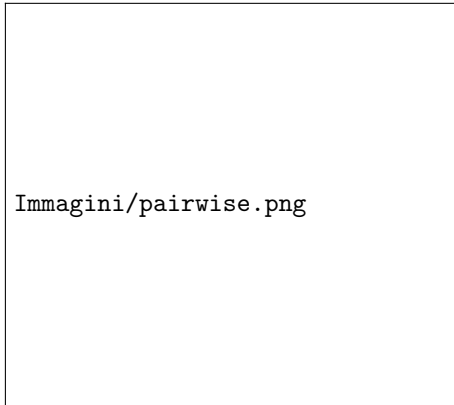
12.3 PairWise Coupling (PWC) (One vs One)

Sia c il numero di classi: il numero totale di coppie possibili è dato dal coefficiente binomiale

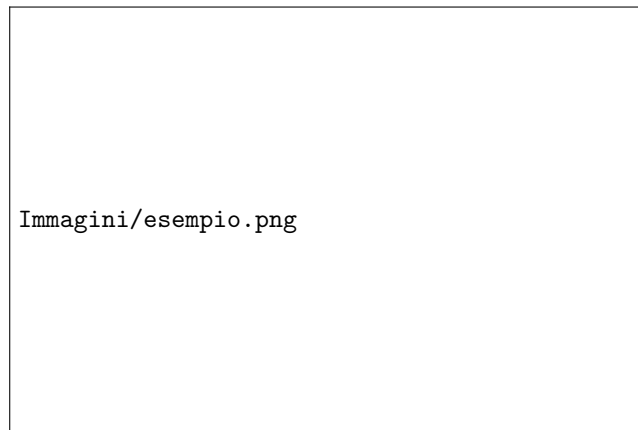
$$\binom{c}{2} = \frac{c(c-1)}{2}$$

Il metodo PWC costruisce quindi un classificatore per ogni coppia di classi, addestrato a distinguere una classe dall'altra.

Nel caso di tre classi, è possibile fornire la seguente interpretazione geometrica:



Un esempio pratico è mostrato nella figura seguente:



Si osserva che, all'aumentare del numero di classi, il numero di classificatori cresce quadraticamente, rendendo il metodo più oneroso dal punto di vista computazionale.

12.3.1 Distributed Output Code (ECOC)

Per mitigare il costo computazionale dei metodi come One-vs-One, è stato introdotto il *Distributed Output Code* (ECOC). Infatti, all'aumentare del numero di classi, il numero di classificatori cresce rapidamente; l'idea è quindi costruire matrici di decomposizione che rendano più semplice ed efficiente la fase di ricostruzione.

In questo approccio, ogni colonna della matrice rappresenta una *codeword* associata a una classe. La distanza tra due stringhe binarie è detta **distanza**

di Hamming ed è definita come il numero di posizioni in cui le due stringhe differiscono. Si consideri, ad esempio, la seguente matrice:

$$\begin{pmatrix} +1 & +1 & +1 & -1 \\ +1 & +1 & -1 & +1 \\ +1 & +1 & -1 & -1 \\ +1 & -1 & +1 & +1 \\ +1 & -1 & +1 & -1 \\ +1 & -1 & -1 & +1 \\ +1 & -1 & -1 & -1 \end{pmatrix}$$

Osservando, ad esempio, la prima e la seconda colonna (così come la prima e la terza), la distanza di Hamming è pari a 4. L'obiettivo è costruire codeword tra loro il più possibile distanti (massima distanza di Hamming).

Ogni colonna descrive come una classe viene codificata rispetto ai diversi problemi binari. Poiché la fase di ricostruzione deve identificare la classe a partire da una stringa di output, avere codeword molto distanti tra loro riduce la probabilità di errore.

La regola di ricostruzione consiste nell'assegnare l'istanza alla classe la cui codeword ha distanza di Hamming minima rispetto alla stringa ottenuta in output dai classificatori. Se le codeword sono ben separate (alta distanza di Hamming), è meno probabile confondere due classi.

Consideriamo l'esempio in figura:



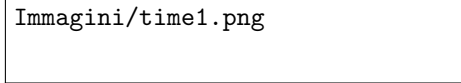
Si ottiene una stringa di output (quella *viola*), che viene confrontata con le codeword delle classi calcolando la distanza di Hamming:

$$0 \quad 2 \quad 2$$

La classe assegnata è quella con distanza minima, in questo caso ω_1 . In sintesi, dato un numero di classi e di classificatori, si progettano codeword con distanza di Hamming elevata, si addestrano i modelli binari e infine si ricostruisce la classe scegliendo quella a distanza di Hamming minima. Questo approccio è particolarmente efficace in problemi multiclasse con molte classi.

13 Time Series Mining

Una *time series* è una sequenza di osservazioni acquisite nel tempo in modo ordinato. Può rappresentare diversi tipi di segnali, come ad esempio un segnale ECG, una sequenza video o una serie di valori numerici.



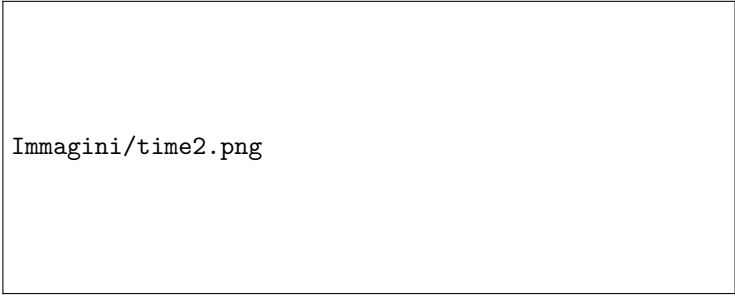
Immagini/time1.png

Le time series possono essere:

- **Unidimensionali:** una singola variabile nel tempo
- **Multidimensionali:** più variabili osservate simultaneamente (ad esempio dati sensoriali o segnali multicanale)
- **Spazio-temporali:** come i video, che aggiungono dimensioni spaziali alla componente temporale
- **Sequenziali simboliche:** come una sequenza di parole o caratteri nel tempo

Su una time series è possibile svolgere diversi task:

- **Forecasting:** previsione dei valori futuri della serie
- **Clustering:** raggruppamento di serie simili
- **Classificazione e regressione:** assegnazione di etichette o previsione di valori continui
- **Motif discovery:** identificazione di pattern ricorrenti all'interno del segnale, individuando le primitive di andamento
- **Outlier detection:** individuazione di anomalie o comportamenti anomali nella serie



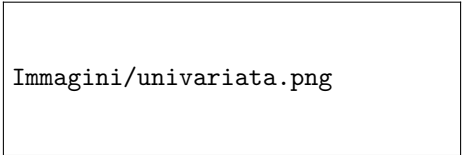
Immagini/time2.png

L'analisi delle time series presenta diverse sfide. Una delle principali riguarda l'elevata **dimensionalità**: ogni campione è descritto da molti istanti temporali, soprattutto quando i dati sono acquisiti ad alta frequenza. Un ulteriore aspetto critico è l'**ordinamento temporale**: i dati non sono indipendenti e identicamente distribuiti, ma ordinati nel tempo. Questo rende più complessa, ad esempio, la fase di validazione dei modelli, poiché non è sempre possibile mescolare casualmente i campioni. Inoltre, le serie temporali possono presentare rumore, valori mancanti, diverse frequenze di campionamento e problemi di scalatura tra serie differenti, rendendo necessarie opportune tecniche di preprocessing.

13.1 Definizioni

Le time series possono essere:

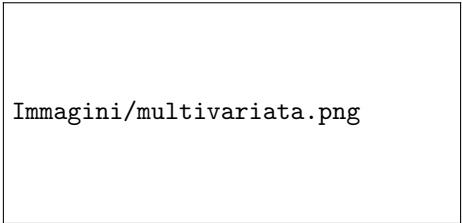
- **Univariate Times Series (UTS)**: una collezione di osservazioni $x_1, \dots, x_N$ misurate nel tempo, così che al tempo t_i si ottiene il valore x_i



Immagini/univariata.png

- **Multivariate Times Series (MTS)**: una MTS di lunghezza N e dimensionalità Q contiene Q UTS, note come **attributi**. Ogni timestamp contiene una componente per ognuna delle Q serie. L'osservazione al tempo t_i è quindi un vettore:

$$\bar{x}_i = (x_1^1, \dots, x_i^Q)$$



Immagini/multivariata.png

13.2 Scale diverse dei campioni

Un problema comune nelle time series è la presenza di scale diverse tra i campioni. Questo può essere risolto tramite tecniche di **normalizzazione** o **standardizzazione**. La **normalizzazione** trasforma i dati in un intervallo fissato, tipicamente $[0, 1]$:

$$x'_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}} \quad (13.1)$$

La **standardizzazione**, invece, centra i dati rispetto alla media e li scala in base alla deviazione standard:

$$x'_i = \frac{x_i - \mu}{\sigma} \quad (13.2)$$

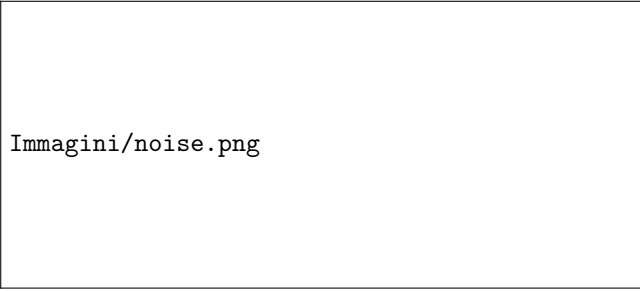
Nelle time series è fondamentale mantenere la coerenza temporale: non è possibile mescolare i dati arbitrariamente. Per questo motivo si utilizza spesso un approccio a **finestra mobile** (*sliding window*). Per ogni intervallo temporale si definiscono porzioni di training, validation e test. Successivamente, la finestra viene spostata in avanti nel tempo per generare nuovi sottoinsiemi di dati. Lo spostamento della finestra è detto **stride** s e rappresenta il numero di istanti temporali di cui la finestra viene traslata a ogni passo.

13.3 Missing Values

Nelle time series è frequente la presenza di valori mancanti, dovuti ad errori di acquisizione, perdita di segnale o problemi nei sensori. La gestione dei *missing values* è fondamentale, poiché può influenzare significativamente le prestazioni dei modelli.

13.4 Rimozione del rumore

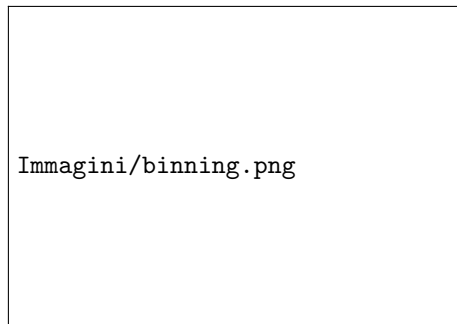
Mediante tecniche di *signal processing*. Le fluttuazioni possono essere rapide o veloci, le tipiche tecniche analizzano filtri basate sulle medie per rimuovere il rumore o di **binning** e **smoothing**.



Immagini/noise.png

13.4.1 Binning

Il *binning* è una tecnica di discretizzazione e smoothing che suddivide un segnale in intervalli equi-spaziati (bin). Per ciascun intervallo si calcola la media dei punti contenuti al suo interno.



Il segnale risultante appare più regolare e meno soggetto a fluttuazioni, poiché le variazioni locali vengono smussate. La scelta della dimensione dell'intervallo (ampiezza del bin) è cruciale:

- Intervalli più grandi producono un segnale più liscio ma con perdita di dettaglio
- Intervalli più piccoli preservano meglio le fluttuazioni originali, ma riducono l'effetto di smoothing



13.4.2 Smoothing

Ci sono vari approcci di smoothing. I più semplici sono quelli a:

- **Media mobile**
- **Media esponenziale**

Smoothing a media mobile La *media mobile* è una tecnica di smoothing che può essere vista come un'estensione del binning basata sul concetto di finestra mobile (*sliding window*).



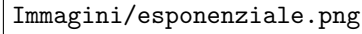
Dato un segnale, esso viene suddiviso in finestre di ampiezza fissata che scorrono lungo il tempo con una certa sovrapposizione:



Per ogni finestra si calcola la media dei valori contenuti e tale valore viene assegnato al centro della finestra, generando un nuovo segnale più regolare. Il binning rappresenta un caso particolare della media mobile in cui le finestre non si sovrappongono (lo stride è uguale all'ampiezza della finestra). Al contrario, permettendo la sovrapposizione tra finestre, la media mobile consente di ottenere una rappresentazione più fluida del segnale.

Grazie alla sovrapposizione, è possibile calcolare più valori medi lungo il segnale rispetto al binning, preservando maggiormente l'andamento locale pur riducendo il rumore.

Smoothing a media esponenziale Nella *media esponenziale* si introduce un coefficiente di peso che assegna maggiore importanza ai campioni più recenti rispetto a quelli passati.



Il valore smussato viene calcolato ricorsivamente come:

$$x'_i = \alpha \cdot x_i + (1 - \alpha) \cdot x'_{i-1} \quad , \quad (13.3)$$

dove $\alpha \in (0, 1)$ è il parametro di smoothing.

Iterando la formula, si ottiene una combinazione pesata esponenziale dei valori passati:

$$x'_i = (1 - \alpha)^i \cdot x_0 + \alpha \cdot \sum_{j=1}^i x_j \cdot (1 - \alpha)^{i-j} \quad (13.4)$$

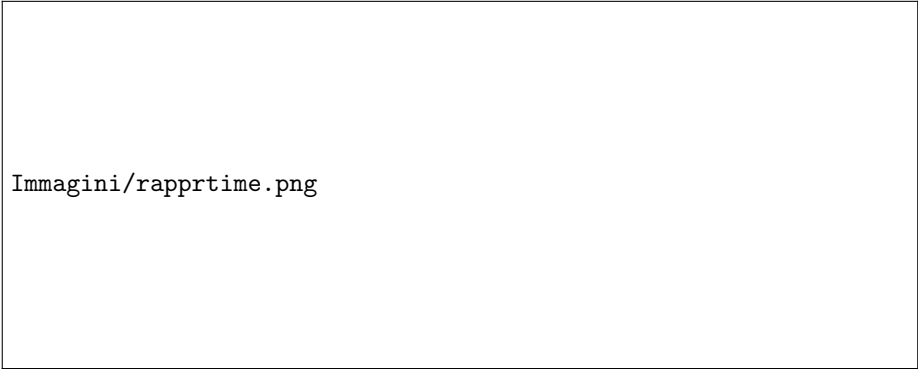
Il termine *esponenziale* deriva dal fatto che i pesi decrescono esponenzialmente nel tempo.

La scelta di α influenza il comportamento del filtro:

- per α grande, il segnale smussato segue più rapidamente le variazioni del segnale originale
- per α piccolo, il segnale risulta più stabile ma meno reattivo ai cambiamenti

13.5 Rappresentazione delle time series

In questo contesto per rappresentazione significa estrarre delle feature che vengono fornite in un determinato modello. Tra le varie tassonomie, una è quella in Figura:

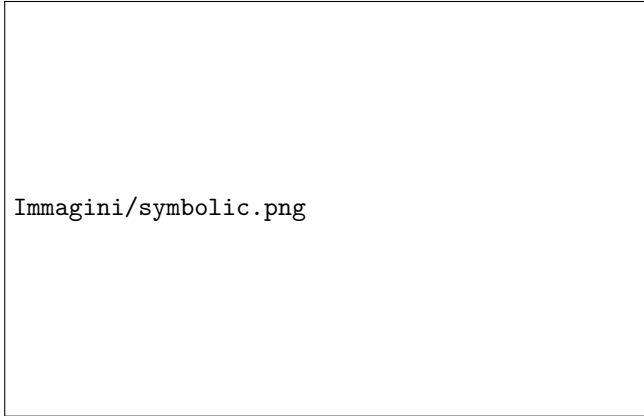


Immagini/rapprtime.png

Rappresentare una time series significa prendere un segnale nel tempo e trasformarlo in un vettore di feature. Un altro modo è il symbolic aggregate approximation.

13.6 Symbolic Aggregate Approximation

Si considera un segnale, lo si divide in finestre e per ciascuna finestra si calcola il valore medio, ottenendo una sequenza di valori medi (*Piecewise Aggregate Approximation*). A partire da questi valori si costruisce una distribuzione.



Immagini/symbolic.png

Successivamente, l'intervallo dei valori viene suddiviso in livelli (tipicamente equiprobabili, assumendo una distribuzione gaussiana normalizzata). A ogni intervallo viene associato un simbolo, ottenendo così una rappresentazione simbolica della serie.

La sequenza di valori medi viene trasformata in una sequenza di simboli (stringa), riducendo la dimensionalità e facilitando il confronto tra serie temporali.

13.7 Forecasting

Dato un andamento del tempo, ci si chiede il valore predetto futuro del prossimo campione al tempo $t + \Delta t$.

Immagini/forecasting.png

Si può effettuare in grandi linee in tre approcci:

1. Approcci generativi: modelli autoregressivi o modelli di Markov
2. Reti neurali: reti neurali ricorrenti (RNN), LSTM
3. Modelli basati sulle rappresentazioni

13.7.1 Modello autoregressivo

Un modello autoregressivo (AR) di ordine p predice il valore corrente di una time series come combinazione lineare dei suoi p valori passati:

$$x(t_i) = \sum_{j=1}^p a_j x(t_{i-j}) + c + \epsilon_t \quad , \quad (13.5)$$

dove a_j sono i coefficienti del modello, c è una costante e ϵ_t rappresenta un termine di rumore bianco. All'aumentare di p , il modello tiene conto di una porzione più ampia del passato.

L'ipotesi alla base è che esista una correlazione tra campioni temporalmente vicini. Tale correlazione può essere misurata con l'**autocorrelazione** al lag p :

$$\rho_p = \frac{\text{Cov}[x(t_i), x(t_{i-p})]}{\text{Var}[x(t_i)]} \quad (13.6)$$

Il modello è efficace quando la serie presenta una significativa dipendenza temporale.

Esistono estensioni più generali del modello autoregressivo:

1. **ARMA (Autoregressive Moving Average)**: combina una componente autoregressiva (AR) e una componente a media mobile (MA)
2. **ARIMA (Autoregressive Integrated Moving Average)**: estende ARMA introducendo una fase di differenziazione (integrazione) per rendere la serie stazionaria

Il modello a **media mobile** (MA) esprime il valore corrente come combinazione lineare dei termini di rumore:

$$y_t = \sum_{i=1}^q b_i \epsilon_{t-i} + c + \epsilon_t \quad (13.7)$$

In questo caso, la difficoltà principale è stimare il rumore bianco ϵ_t .

Il modello **ARMA** combina le due componenti:

$$y_t = \sum_{i=1}^p a_i y_{t-i} + \sum_{i=1}^q b_i \epsilon_{t-i} + c + \epsilon_t \quad (13.8)$$

Il modello **ARIMA** introduce una trasformazione sulla serie originale: invece di modellare direttamente y_t , si considera la serie differenziata. Nel caso discreto, la derivata prima è approssimata come:

$$y'_t = y_t - y_{t-1}$$

Il modello viene quindi applicato alla serie trasformata:

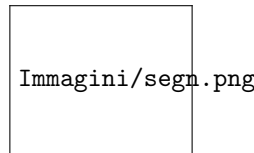
$$y'_t = \sum_{i=1}^p a_i y'_{t-i} + \sum_{i=1}^q b_i \epsilon_{t-i} + c + \epsilon_t \quad (13.9)$$

In questo modo, ARIMA modella le variazioni tra campioni consecutivi anziché i valori assoluti, risultando più adatto a serie non stazionarie.

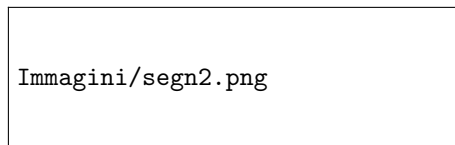
13.8 Classificazione

Nel contesto delle time series, un problema fondamentale è misurare la similarità tra segnali, ad esempio per task di classificazione o **motif discovery**.

Consideriamo i due segnali in figura:



Visivamente risultano simili: uno dei due appare semplicemente più rumoroso. Tuttavia, una macchina può avere difficoltà a riconoscere questa similarità in modo diretto. Consideriamo ora un secondo esempio:



In questo caso, i segnali sono sostanzialmente identici ma uno è specchiato rispetto all'altro. Anche questa trasformazione è facilmente riconoscibile dall'uomo, ma non necessariamente da un algoritmo standard.

Per questo motivo, sono stati sviluppati metodi per confrontare segnali basati su misure di distanza tra rappresentazioni, progettate per essere robuste rispetto a:

- Piccole fluttuazioni (rumore)
- Cambiamenti di scala
- Variazioni di segno
- Disallineamenti temporali

Una prima idea è quella di confrontare due segnali punto per punto (distanza verticale a tempo fissato). Tuttavia, questo approccio è spesso inadeguato.



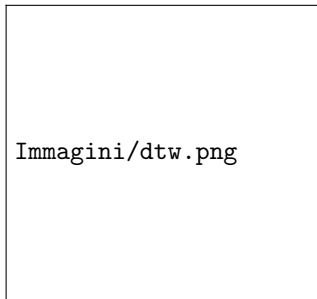
Immagini/segn3.png

Anche segnali molto simili possono risultare fortemente diversi se non sono perfettamente allineati nel tempo. Questo introduce il problema della **traslazione relativa**: due segnali simili possono essere sfasati temporalmente.

Una buona misura di distanza tra time series deve tenere conto di possibili disallineamenti, evitando confronti rigidi punto a punto.

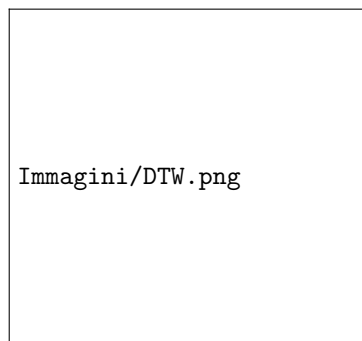
13.8.1 Dynamic Time Warping

Il *Dynamic Time Warping* (DTW) è una delle misure di similarità più utilizzate per confrontare time series. A differenza della distanza euclidea, è una **misura di forma**, in quanto consente di confrontare segnali anche in presenza di disallineamenti temporali.



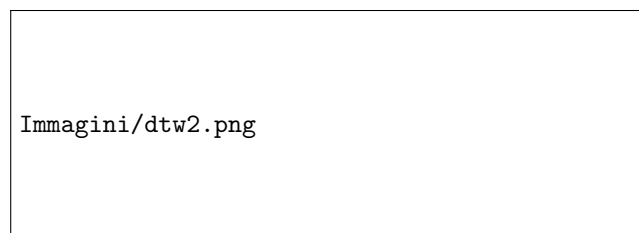
Mentre la distanza euclidea impone un allineamento rigido punto a punto (mapping 1:1), il DTW rilassa questo vincolo permettendo un mapping *uno-a-molti*. In questo modo, porzioni di un segnale possono essere allineate con più punti dell'altro segnale.

Operativamente, i due segnali vengono disposti lungo le righe e le colonne di una matrice. Ogni cella contiene la distanza tra i punti corrispondenti dei due segnali, costruendo così una **matrice di costo**.

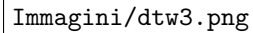


L'obiettivo è trovare un percorso dalla cella iniziale a quella finale che minimizzi il costo totale, ovvero la somma delle distanze lungo il percorso.

La diagonale della matrice rappresenta l'allineamento perfetto (1:1), ma il percorso ottimale può deviare dalla diagonale per adattarsi a variazioni locali nella velocità dei segnali.



Questo rende il DTW particolarmente efficace nel confrontare segnali simili ma non perfettamente allineati nel tempo. Tuttavia, il costo computazionale è elevato (tipicamente quadratico rispetto alla lunghezza dei segnali). Per ridurlo, si introducono dei vincoli detti **warping constraints**, che limitano la ricerca del percorso ottimale a una banda attorno alla diagonale.

A rectangular box containing the text "Immagini/dtw3.png", which serves as a placeholder for an image.

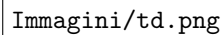
13.9 Reti neurali per Time Series

Nelle reti neurali feed-forward si possono individuare alcune caratteristiche fondamentali:

1. Il segnale fluisce in una sola direzione (in avanti in fase di inferenza, all'indietro durante il backpropagation)
2. A ogni ingresso corrisponde un'unica uscita
3. Non esiste memoria dello stato passato: l'uscita y dipende solo dall'ingresso corrente e dai pesi della rete

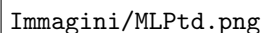
Queste proprietà rendono le reti feed-forward poco adatte a modellare dati temporali, in cui il contesto passato è rilevante.

Una delle prime estensioni introdotte per gestire la dimensione temporale è rappresentata dalle **Time Delay Neural Network** (TDNN). Si trattano essenzialmente di reti MLP che incorporano ritardi temporali sugli ingressi.

A rectangular box containing the text "Immagini/td.png", which serves as a placeholder for an image.

Consideriamo, ad esempio, un percettrore con ritardo temporale: il segnale in ingresso $x(t)$ viene utilizzato insieme ai suoi valori passati $x(t - \Delta)$, $x(t - 2\Delta)$, $\dots$, ciascuno pesato da coefficienti diversi $(w_1, w_2, w_3, \dots)$.

La rete non utilizza soltanto il valore corrente del segnale, ma anche una finestra di m istanti precedenti. Tali valori vengono trattati come un vettore di ingresso, analogamente a quanto avviene nelle MLP tradizionali. Questo approccio può essere esteso a reti più profonde (*multi-layer perceptron*), mantenendo lo stesso schema di ingresso:



Immagini/MLPtd.png

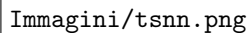
La dimensione temporale viene modellata esclusivamente nello strato di ingresso, mentre gli strati nascosti non possiedono una vera memoria dinamica del passato.

14 Reti neurali ricorrenti (RNN)

I modelli neurali possono essere classificati in base alla relazione tra input e output:

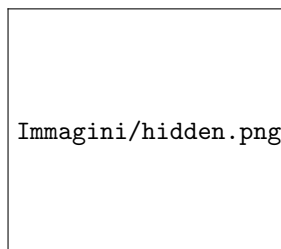
- **One-to-one:** un ingresso e un'uscita (tipico delle reti feed-forward)
- **One-to-many:** un singolo ingresso genera una sequenza in uscita (es. *image captioning*)
- **Many-to-one:** una sequenza in ingresso produce un'unica uscita (es. *sentiment classification*)
- **Many-to-many:** una sequenza in ingresso produce una sequenza in uscita (es. *machine translation*)

Come discusso in precedenza, le reti feed-forward (MLP), pur potendo gestire input e output multipli, non modellano esplicitamente la dimensione temporale o sequenziale.



Immagini/tsnn.png

Per semplificare la rappresentazione, si può descrivere una MLP comprimendo gli strati nascosti in un unico blocco:



Per introdurre la dipendenza temporale, si aggiunge una **connessione ricorrente**, che permette di propagare informazione tra istanti temporali successivi:

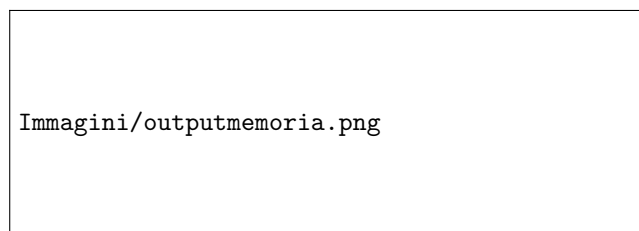


Questa connessione introduce uno stato nascosto h_t , che funge da memoria del sistema. L'output al tempo t non dipende più solo dall'ingresso corrente:

$$\hat{y}_t = f(x_t) \quad ,$$

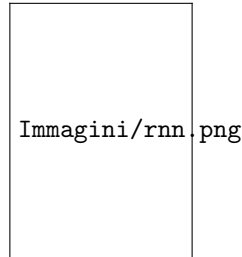
ma anche dallo stato precedente:

$$\hat{y}_t = f(x_t, h_{t-1}) \tag{14.1}$$



Il modulo ricorrente è quindi caratterizzato da:

- Un ingresso x_t
- Uno stato nascosto h_t
- Un'uscita $\hat{y}_t$



Lo stato nascosto viene aggiornato ricorsivamente combinando l'informazione corrente e quella passata:

$$h_t = f_W(h_{t-1}, x_t) \quad (14.2)$$

Ogni istante temporale mantiene una dipendenza dal passato tramite la memoria h_t , consentendo alla rete di modellare sequenze e dipendenze temporali.

14.1 Implementazione Vanilla

La forma più semplice di rete neurale ricorrente (RNN) è detta *vanilla RNN* ed è caratterizzata da un singolo stato nascosto h_t :

$$h_t = f_W(h_{t-1}, x_t)$$

In forma esplicita, l'aggiornamento dello stato nascosto è dato da:

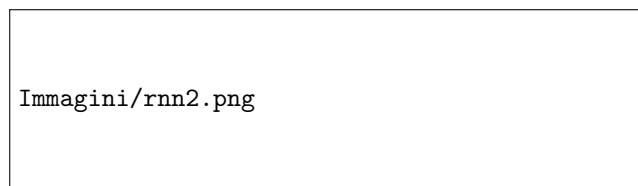
$$h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b) \quad ,$$

dove W_{hh} è la matrice dei pesi ricorrenti, W_{xh} quella degli ingressi e b è un termine di bias. L'output al tempo t è invece:

$$y_t = g(W_{hy}h_t + c) \quad (14.3)$$

dove W_{hy} è la matrice dei pesi verso l'output e c è un bias.

Una RNN può essere rappresentata come un grafo computazionale nel tempo:



Durante il **forward**, per ogni istante temporale viene calcolata una loss L_t . La loss complessiva è la somma delle loss sui vari timestep:

$$L = \sum_{t=0}^T L_t \quad (14.4)$$

Immagini/rnnforw.png

Il passo di backward prende il nome di **Backpropagation Through Time (BPTT)**. In questa fase, il gradiente viene propagato:

- lungo la rete (come nelle reti feed-forward)
- lungo la dimensione temporale, da destra verso sinistra

Immagini/rnnback.png

La derivata della loss rispetto ai pesi ricorrenti può essere espressa come:

$$\frac{\partial L_t}{\partial W_{hh}} = \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_t} \cdot \underbrace{\left(\sum_{k=0}^t \frac{\partial h_t}{\partial h_k} \cdot \frac{\partial h_k}{\partial W_{hh}} \right)}_{\text{propagazione nel tempo}}$$

Il termine di propagazione nel tempo tiene conto del contributo di tutti gli stati precedenti. Applicando la regola della catena, si può scrivere:

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}}$$

Sostituendo, si ottiene:

$$\frac{\partial L_t}{\partial W_{hh}} = \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_t} \cdot \left(\sum_{k=0}^t \prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}} \cdot \frac{\partial h_k}{\partial W_{hh}} \right) \quad (14.5)$$

Questo termine evidenzia come il gradiente dipenda da una produttoria di derivate nel tempo. La moltiplicazione ripetuta di termini può portare a problemi numerici, come il **vanishing** o **exploding gradient**, che rendono difficile l'apprendimento su sequenze lunghe.

14.2 Gradiente evanescente nelle RNN

Il problema del **gradiente evanescente** si manifesta anche nelle reti neurali ricorrenti. In questo caso, il fenomeno è amplificato dalla propagazione del gradiente lungo due dimensioni:

- Lungo la profondità della rete (come nelle MLP)
- Lungo la dimensione temporale (attraverso gli stati ricorrenti)

In particolare, durante la *Backpropagation Through Time*, il gradiente viene moltiplicato ripetutamente per le derivate degli stati nascosti. Questo porta, per sequenze lunghe, a valori che tendono rapidamente a zero (vanishing) oppure a crescere in modo incontrollato (exploding).

L'utilizzo di funzioni di attivazione come la ReLU può mitigare il problema nelle catene verticali (come nelle MLP), ma non è sufficiente per risolverlo lungo la dimensione temporale. Per affrontare questo problema, si adottano diverse strategie:

- **Architetture con gate:** meccanismi che controllano il flusso dell'informazione nel tempo, permettendo di mantenere o dimenticare selettivamente lo stato passato
- **Inizializzazione e funzioni di attivazione:** una scelta appropriata dei pesi iniziali e delle attivazioni può migliorare la stabilità del training
- **Truncated Backpropagation Through Time:** limita la propagazione del gradiente a un numero finito di passi temporali
- **Gradient clipping:** impone un limite al valore del gradiente per evitare instabilità numeriche

Queste tecniche rendono più stabile ed efficace l'addestramento delle RNN, soprattutto in presenza di sequenze lunghe.

15 Ensemble Learning

Per *ensemble learning* si intende l'approccio che combina più modelli (detti anche *multi-esperto*) per risolvere lo stesso problema, aggregando le loro decisioni al fine di ottenere prestazioni migliori rispetto al singolo modello. I sistemi multi-esperto possono essere costruiti manualmente oppure tramite tecniche algoritmiche.



Immagini/multiesperto1.png

Nei sistemi costruiti manualmente, si considerano più modelli (ad es. classificatori), ciascuno dei quali produce un output, come una distribuzione di probabilità (ad es. tramite softmax). Il problema principale diventa quindi come combinare queste predizioni per ottenere una decisione finale.

Affinché l'ensemble sia efficace, i modelli devono essere **diversi**, ovvero devono commettere errori su campioni differenti. Due aspetti fondamentali sono quindi:

- **Diversità:** i modelli devono essere eterogenei e non commettere gli stessi errori
- **Aggregazione:** definire come combinare le decisioni dei singoli modelli

La diversità tra modelli può essere ottenuta in diversi modi:

- **Diversità nelle feature:** utilizzare differenti rappresentazioni del vettore di input $\underline{x}$
- **Diversità nei modelli:** adottare algoritmi di apprendimento differenti (ad esempio alberi, reti neurali, SVM)
- **Diversità nei dati:** addestrare modelli sullo stesso spazio di feature ma su sottoinsiemi diversi del dataset

15.1 Matrice profilo di decisione

Un modo semplice per combinare le predizioni di più modelli è utilizzare la **matrice profilo di decisione** (*decision profile matrix*). In questa matrice:

- Ogni **riga** rappresenta l'output di un classificatore
- Ogni **colonna** rappresenta una classe
- Ogni elemento contiene la probabilità (o score) assegnata da un classificatore a una determinata classe

Una volta costruita la matrice, è possibile combinare le decisioni dei modelli in diversi modi:

- **Media (average pooling):** si calcola la media delle probabilità per ciascuna classe e si seleziona quella con valore più alto
- **Somma:** si sommano i contributi dei classificatori per ogni classe
- **Voto di maggioranza:** ogni classificatore vota per una classe e si sceglie quella più votata

Un approccio più avanzato consiste nel costruire un **meta-classificatore**, che utilizza come input le predizioni dei modelli base per apprendere come combinarle in modo ottimale (approccio noto come *stacking*).

15.2 Metriche per misurare la diversità

La **diversità** è una misura fondamentale negli ensemble, in quanto quantifica quanto due classificatori differiscono nei loro errori. In particolare, si valuta quanto spesso due modelli sono in accordo o in disaccordo nelle loro predizioni. Viene tipicamente misurata *pairwise*, considerando coppie di classificatori. Per ogni coppia, si costruisce una matrice di contingenza 2×2 del tipo:

	D_k corretto (1)	D_k errato (0)
D_i corretto (1)	N^{11}	N^{10}
D_i errato (0)	N^{01}	N^{00}

dove:

- N^{11} : numero di campioni classificati correttamente da entrambi i classificatori
- N^{00} : numero di campioni classificati erroneamente da entrambi
- N^{10} : campioni corretti da D_i ma errati da D_k
- N^{01} : campioni errati da D_i ma corretti da D_k

Questa matrice permette di definire diverse metriche di diversità, basate sul grado di accordo/disaccordo tra i classificatori. In generale, maggiore è il disaccordo (valori elevati di N^{10} e N^{01}), maggiore è la diversità tra i modelli. Nota questa matrice ci sono diverse misure associate.

15.2.1 Q-statistic

Data una coppia di classificatori D_i e D_k , la **Q-statistic** è definita come:

$$Q_{i,k} = \frac{N^{11}N^{00} - N^{01}N^{10}}{N^{11}N^{00} + N^{01}N^{10}} \quad (15.1)$$

Questa misura assume valori nell'intervallo $[-1, 1]$ e consente di valutare il grado di dipendenza tra i due classificatori:

- $Q_{i,k} = 0$: i classificatori sono statisticamente indipendenti
- $Q_{i,k} > 0$: i classificatori tendono a fare previsioni corrette sugli stessi campioni
- $Q_{i,k} < 0$: i classificatori commettono errori su campioni diversi (alta diversità)

Per un insieme di L classificatori, si può calcolare la media della Q-statistic su tutte le coppie:

$$Q_{av} = \frac{2}{L(L-1)} \sum_{i=1}^L \sum_{k=i+1}^L Q_{i,k} \quad (15.2)$$

Valori bassi (o negativi) di Q indicano maggiore diversità.

15.2.2 Coefficiente di correlazione

Un'altra misura di diversità è il **coefficiente di correlazione** tra due classificatori D_i e D_k , definito come:

$$\rho_{i,k} = \frac{N^{11}N^{00} - N^{01}N^{10}}{\sqrt{(N^{11} + N^{10})(N^{01} + N^{00})(N^{11} + N^{01})(N^{10} + N^{00})}} \quad (15.3)$$

Anche questo coefficiente assume valori nell'intervallo $[-1, 1]$:

- valori positivi indicano correlazione tra i classificatori
- valori negativi indicano comportamento complementare (maggiore diversità)
- valori prossimi a zero indicano indipendenza

Analogamente alla Q-statistic, è possibile calcolare il valore medio del coefficiente di correlazione su tutte le coppie di classificatori. Si può dimostrare che, per ogni coppia di classificatori, vale la relazione:

$$|\rho_{i,k}| \leq |Q_{i,k}|$$

15.2.3 Disaccordo

Data una coppia di classificatori D_i e D_k , la **disagreement measure** è definita come:

$$\text{Dis}_{i,k} = \frac{N^{01} + N^{10}}{N^{11} + N^{10} + N^{01} + N^{00}} \quad (15.4)$$

Questa misura rappresenta la frazione di campioni su cui i due classificatori sono in disaccordo, ovvero quando uno classifica correttamente e l'altro no.

Il valore è compreso tra 0 e 1:

- 0: i classificatori sono sempre in accordo
- 1: i classificatori sono sempre in disaccordo

15.2.4 Double fault

La **double fault measure** quantifica la proporzione di campioni che vengono classificati erroneamente da entrambi i classificatori:

$$DF_{i,k} = \frac{N^{00}}{N^{11} + N^{10} + N^{01} + N^{00}} \quad (15.5)$$

Questa misura assume valori nell'intervallo $[0, 1]$:

- 0: i classificatori non commettono errori sugli stessi campioni (caso ideale)
- 1: tutti i campioni sono errati per entrambi i classificatori (caso pessimo)

A differenza delle altre metriche, in questo caso valori **bassi** sono desiderabili, poiché indicano che i modelli non sbagliano sugli stessi esempi.

15.3 Random Forest

Il **Random Forest** è un metodo di ensemble basato su un insieme di alberi decisionali, progettato per aumentare la diversità tra i modelli e migliorare le prestazioni complessive.

La diversità tra gli alberi non viene misurata esplicitamente, ma introdotta attraverso due meccanismi principali:

- **Campionamento dei dati (bagging)**: ogni albero viene addestrato su un sottoinsieme del dataset ottenuto tramite campionamento con reintroduzione
- **Sottoinsieme casuale di feature**: ad ogni nodo dell'albero, invece di considerare tutte le p feature disponibili, si seleziona un sottoinsieme casuale di dimensione tipicamente pari a $\sqrt{p}$ (per problemi di classificazione)

Questo approccio permette di costruire alberi tra loro differenti, riducendo la correlazione tra i modelli e migliorando la capacità di generalizzazione.

Poiché ogni albero restituisce una *crisp label* (una classe), la combinazione delle predizioni avviene tramite:

- **Voto di maggioranza** nel caso di **classificazione**
- **Media** nel caso di **regressione**

Il numero di alberi nella foresta è un iperparametro del modello. In pratica, si utilizzano spesso valori dell'ordine di 100 o più, a seconda della complessità del problema.

15.4 Bagging (Bootstrap Aggregating)

Il **Bagging** (Bootstrap Aggregating) è una tecnica di ensemble che introduce diversità tra i modelli utilizzando diversi sottoinsiemi del dataset di addestramento. L'idea di base è la seguente:

- Si generano più dataset di training tramite campionamento con reintroduzione (**bootstrap**)
- Su ciascun dataset viene addestrato lo stesso modello
- Le predizioni dei modelli vengono poi combinate

Poiché ogni dataset bootstrap contiene campioni duplicati e ne esclude altri, i modelli risultanti sono tra loro diversi, introducendo così diversità nell'ensemble. La combinazione delle predizioni avviene tipicamente tramite:

- **Voto di maggioranza** per problemi di classificazione
- **Media** per problemi di regressione

Il metodo di per sé ha un problema intrinseco perché non garantisce che vengano utilizzati tutti i campioni.

15.5 AdaBoost

L'**AdaBoost** è un algoritmo di ensemble progettato per problemi di classificazione binaria. Ogni campione è rappresentato da un vettore di feature $\underline{x}$ e l'idea fondamentale è associare a ciascun campione un peso che ne rappresenta l'importanza.

Inizialmente, tutti i campioni hanno lo stesso peso:

$$D_1(i) = \frac{1}{m}$$

Nelle iterazioni successive, i pesi vengono aggiornati:

- Aumentano per i campioni difficili da classificare
- Diminuiscono per i campioni facili

In questo modo, il classificatore alle iterazioni successive presta maggiore attenzione ai campioni più difficili. Anche se il dataset rimane lo stesso, la diversa pesatura lo rende *effettivamente* diverso ad ogni iterazione.

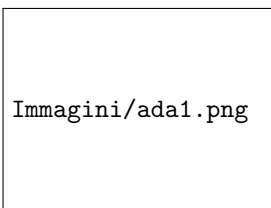
Il meccanismo si basa sull'utilizzo di **weak learner**, ovvero classificatori che performato leggermente meglio del caso casuale. Ad esempio:

- Alberi poco profondi (potati precocemente)
- MLP con pochi neuroni

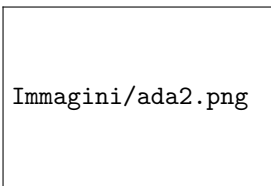
Se il modello è troppo accurato (non più weak), non ha senso continuare ad aggiungere classificatori e l'algoritmo può terminare.

Esempio: si considerino 4 campioni. Alla prima iterazione:

$$D_1(i) = \frac{1}{4}$$



Si addestra il primo classificatore h_1 , che genera una frontiera decisionale:



Supponendo che venga classificato erroneamente un solo campione, l'errore è:

$$\epsilon_1 = D_1(2) = \frac{1}{4}$$

Si calcola quindi il peso del classificatore:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right) \quad (15.6)$$

Nell'esempio:

$$\alpha_1 = \frac{1}{2} \ln 3 = \ln \sqrt{3}$$

I pesi dei campioni vengono aggiornati secondo la regola:

$$\begin{aligned} D_{t+1}(i) &= \frac{D_t(i)}{Z_t} \begin{cases} e^{-\alpha_t} & \text{se } h_t(x_i) = y_i \\ e^{\alpha_t} & \text{se } h_t(x_i) \neq y_i \end{cases} \\ &= \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t} \end{aligned} \quad (15.7)$$

dove Z_t è una costante di normalizzazione.

Quindi:

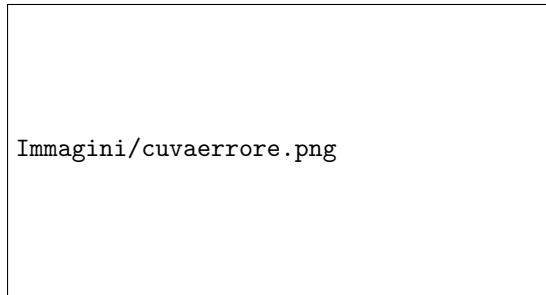
- I campioni classificati correttamente vedono diminuire il proprio peso
- I campioni classificati erroneamente vedono aumentare il proprio peso

Il classificatore successivo si concentrerà quindi maggiormente sui campioni difficili.

L'aggregazione finale dei classificatori è:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right) \quad , \quad (15.8)$$

una combinazione pesata dei classificatori, dove α_t misura l'importanza di ciascun modello. Considerando l'andamento dell'errore al variare del numero di classificatori:



All'aumentare delle iterazioni, il compito dei singoli weak learner diventa più difficile, quindi il loro errore tende ad aumentare e può avvicinarsi a 1/2 (comportamento casuale). Tuttavia, l'ensemble complessivo continua a migliorare le prestazioni.

AdaBoost è uno dei pochi algoritmi per cui è possibile dimostrare un **upper bound** sull'errore dell'ensemble:

$$\prod_t \sqrt{1 - 4\gamma_t^2} \quad , \quad (15.9)$$

dove:

$$\gamma_t = \frac{1}{2} - \epsilon_t \quad ,$$

rappresenta il margine rispetto al classificatore casuale.

16 Reinforcement Learning

I task tipici dell'apprendimento per rinforzo sono:

- **Ottimizzazione** di un determinato task per supporto alle decisioni

- **Controllo** di robot

In uscita si ha una **sequenza di azioni che permettono di ottimizzare un task**. La sfida fondamentale è apprendere come prendere buone decisioni in situazioni di incertezza. Si vuole addestrare un agente che impari come comportarsi in un ambiente inesplorato e dinamico. Lo schema a blocchi tipico è quello presentato in Figura 16.1:

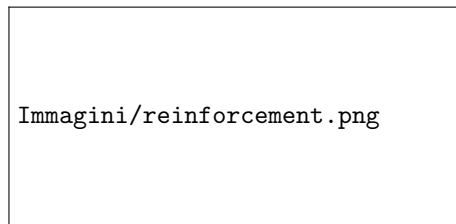


Figura 16.1: Schema a blocchi dell'apprendimento per rinforzo

Si hanno:

- un **agente**
- un **environment** (ambiente)

L'environment può essere un ambiente reale oppure simulato. L'obiettivo dell'agente è imparare a comportarsi correttamente, cioè imparare a risolvere un determinato task.

Ad ogni istante temporale:

- L'agente osserva lo stato corrente S_t
- Sceglie un'azione a_t
- L'azione modifica lo stato dell'ambiente
- L'ambiente restituisce:
 - Una ricompensa R_t
 - Un nuovo stato S_{t+1}

Lo stato rappresenta quindi la configurazione del mondo in un determinato istante temporale, mentre la reward quantifica quanto l'azione intrapresa sia stata positiva o negativa.

Nell'apprendimento per rinforzo l'agente inizialmente non possiede alcuna conoscenza dell'ambiente. Attraverso l'esperienza, l'agente prova differenti azioni e apprende progressivamente quali strategie portano a ricompense migliori. L'apprendimento avviene quindi mediante un processo di **trial and error**.

La ricompensa R_t può essere:

- positiva, per incentivare un comportamento
- negativa, per scoraggiare un comportamento

L'obiettivo dell'agente è massimizzare la **ricompensa cumulativa** nel tempo. Per questo motivo il reinforcement learning coinvolge differenti problematiche:

- **Ottimizzazione:** trovare la migliore sequenza di decisioni
- **Esplorazione:** esplorare nuove azioni e nuovi stati dell'ambiente
- **Sfruttamento** (*exploitation*): utilizzare le conoscenze già apprese per massimizzare la reward
- **Generalizzazione:** apprendere strategie valide anche in contesti differenti
- **Conseguenze ritardate:** una decisione può influenzare ricompense future anche molto lontane nel tempo

A differenza dell'apprendimento supervisionato e non supervisionato, nel reinforcement learning il concetto di **tempo** è fondamentale. La sequenza delle azioni influenza l'evoluzione futura del sistema e quindi anche le ricompense ottenute dall'agente.

16.1 Processo decisionale sequenziale

L'obiettivo è massimizzare un processo decisionale sequenziale. **Sequenziale** perché le azioni sono nel tempo e **decisionale** perché l'agente compie delle scelte. L'obiettivo è massimizzare la **somma cumulativa scontata delle ricompense future che riceve nel tempo**:

$$G_t = \sum_{i=t}^{H-1} \gamma^{i-t} \cdot r_i \quad , \quad (16.1)$$

dove G_t è una sommatoria pesata di ricompense che riceve dall'ambiente. Ad ogni timestamp t , l'agente compie un'azione a_t . A partire da quest'azione riceve un'osservazione o_t e una ricompensa r_t . La **storia del sistema** si può definire come:

$$h_t = (a_1, o_1, r_1, \dots, a_t, o_t, r_t) \quad (16.2)$$

L'agente sceglie l'azione da compiere come funzione della propria storia:

$$a_{t+1} = f(h_t) \quad (16.3)$$

L'obiettivo è massimizzare la **ricompensa attesa**, attesa perché non si conosce a priori il valore di ricompensa ad n timestamps, ma lo si può solo stimare.

Si definisce S il set di tutti i possibili stati in cui l'ambiente può trovarsi. Sia $\{s\}$ la sequenza di stati osservati dall'agente. Un'altra informazione utile per il

processo decisionale è la **dinamica di transizione dell'ambiente**, ovvero la probabilità di trovarmi nello stato s_{t+1} avendo compiuto s_t, a_t fino ad s_1, a_1 :

$$P(s_{t+1} | s_t, a_t, \dots, s_1, a_1) \quad (16.4)$$

E' quindi la probabilità di trovarmi in un certo stato futuro avendo compiuto una sequenza di azioni e osservazioni dall'ambiente. La probabilità di transizione è utile nella sua definizione perché non è detto che ogni volta che compiamo un'azione in un determinato stato ci ritrova in un certo stato futuro. Si può avere un ambiente probabilistico in cui non sempre compiendo un'azione ci si ritrova in un determinato stato futuro.

La **ricompensa immediata** ricevuta dalla transizione tra stati s_t e s_{t+1} avendo compiuto a_t , si indica con r_t . Si può schematizzare la funzione ricompensa, una funzione che ha in ingresso lo stato di partenza, l'azione compiuta e lo stato futuro di appartenenza:

$$R(s_t, a_t, s_{t+1}), \quad R: S \times A \times S \rightarrow \mathcal{R} \quad (16.5)$$

Date le tre variabili di ingresso, restituisce un valore scalare e va a segnare un valore ad ogni transizione da stati.

16.2 Descrizione del modello dell'ambiente

Un ambiente nell'apprendimento per rinforzo è descritto principalmente da:

- una **dinamica di transizione**
- una **funzione di ricompensa**

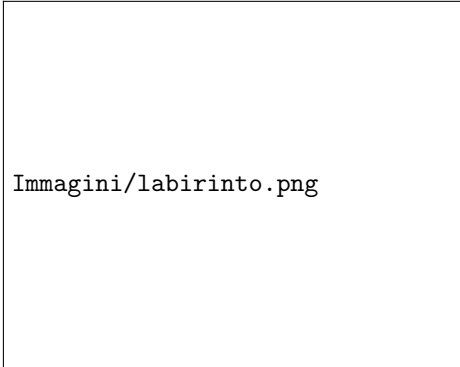
La dinamica di transizione descrive come evolve lo stato del sistema in seguito ad un'azione dell'agente, mentre la funzione di ricompensa assegna un valore numerico alle azioni intraprese. Se un agente conosce esplicitamente:

- le probabilità di transizione tra stati
- la funzione di ricompensa dell'ambiente

allora si parla di approccio **model-based**. In questo caso l'agente possiede un modello esplicito dell'ambiente e può pianificare le proprie azioni simulando l'evoluzione futura del sistema.

Molti algoritmi moderni, invece, sono **model-free**, cioè apprendono direttamente dall'interazione con l'ambiente senza conoscere a priori né la dinamica né la reward.

Consideriamo l'esempio di un labirinto:



Immagini/labirinto.png

L'obiettivo è raggiungere uno stato finale **Goal** partendo da uno stato iniziale **Start**. Si suppone che:

- ogni azione abbia reward pari a -1
- l'agente possa muoversi nelle direzioni:

$$N, E, S, W$$

- lo stato dell'agente sia rappresentato dalla coordinata:

$$(x, y)$$

Assegnare una reward negativa ad ogni passo incoraggia l'agente a trovare il percorso più breve possibile verso il goal. La **ricompensa cumulativa scontata** è pari a:

$$G_t = \sum_{i=t}^{H-1} \gamma^{i-t} r_i \quad (16.6)$$

con:

- r_i : la reward ottenuta all'istante i
- $\gamma \in [0, 1]$: il **fattore di sconto**
- H : l'orizzonte temporale

Il fattore di sconto γ stabilisce quanto peso dare alle ricompense future:

- $\gamma \approx 0$: l'agente privilegia ricompense immediate
- $\gamma \approx 1$: l'agente considera importanti anche le ricompense lontane nel tempo

Ad esempio:

- per $\gamma = 1$, tutte le reward future hanno lo stesso peso

- per $\gamma = 0.5$, le reward future vengono rapidamente attenuate

Nell'esempio del labirinto:

- con $\gamma = 1$ si può ottenere:

$$G_t = -10$$

- con $\gamma = 0.5$:

$$G_t \approx -2$$

Il fattore di sconto è fondamentale anche negli ambienti continui o infiniti, nei quali non esiste un vero stato terminale. In questi casi il discount factor evita che la reward cumulativa diverga e quindi impone un bound superiore a G_t

16.3 Proprietà di Markov

L'ipotesi di Markov afferma che la probabilità di transizione nel passare nello stato s_{t+1} avendo osservato la storia è pari alla probabilità di passare allo stato s_{t+1} avendo visto solamente il passo immediatamente precedente:

$$P(s_{t+1} | s_t, a_t, \dots, s_1, a_1) = P(s_{t+1} | s_t, a_t) \quad (16.7)$$

La storia non è più importante, ma conta solamente il passo immediatamente precedente. Un environment che soddisfa questa ipotesi è definito come **ambiente markoviano**. Un processo che non lo soddisfa è un modello non markoviano. Si può porre la storia del sistema pari a s_{t-1} , che permette di trasformare un ambiente non markoviano ad un ambiente markoviano, semplificando i calcoli matematici.

16.4 Processi di Markov

E' necessario schematizzare un processo decisionale sequenziale, Un paradigma è attraverso i **processi decisionali di Markov**. Consideriamo una breve panoramica dei processi markoviani e dei processi markoviani con ricompensa prima di analizzare i processi decisionali di Markov.

Un **processo di Markov** è un processo randomico senza memoria in cui si ha una sequenza di stati $s_1, s_2, \dots$ che soddisfano l'ipotesi di Markov (Figura 16.2):

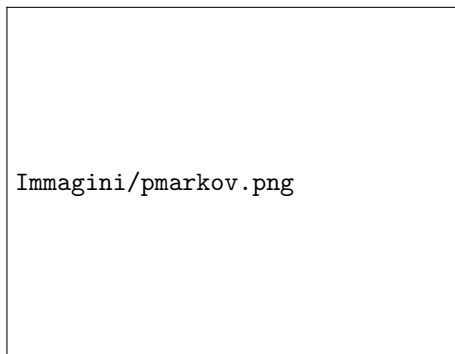


Figura 16.2: Processo di Markov

Un processo di Markov è definito da una tupla di due elementi (S, P) :

- S : un insieme finito di stati
- P : una matrice che rappresenta la dinamica di transizione tra gli stati

Nell'esempio in Figura 16.2, si hanno degli stati e delle probabilità che permettono di passare da uno stato all'altro.

Un **processo di Markov con ricompensa** è un processo simile a quello schematizzato in Figura 16.2, nel quale sono aggiunte delle ricompense. Si tratta quindi di catene di Markov con ricompensa (Figura 16.3), definite da una tupla di quattro elementi (S, P, R, γ) :

- S : un insieme finito di stati
- P : una matrice che rappresenta la dinamica di transizione
- R : la funzione di ricompensa
- $\gamma \in [0, 1]$: il fattore di sconto

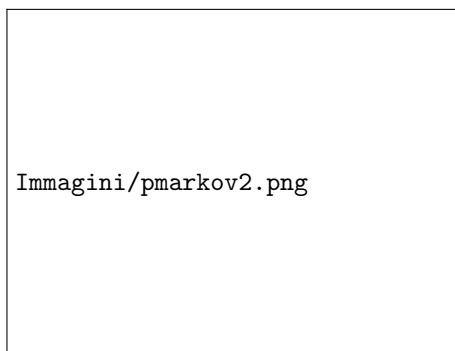


Figura 16.3: Processo di Markov con ricompensa

16.5 Processo decisionale di Markov

Alle formulazioni matematiche precedentemente presentate per poterle presentare ad un'agente di rinforzo mancano dell'azioni da compiere. Note le azioni da compiere, si passa ad un processo decisionale sequenziale con ricompensa che altro non è che un processo decisionale di Markov (MDP) (Figura 16.4).

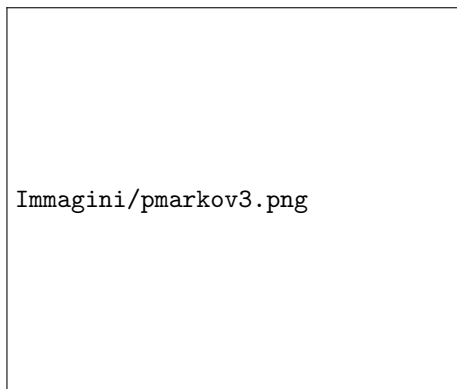


Figura 16.4: Processo decisionale di Markov

Un processo MDP è un processo di controllo stocastico a tempo discreto definito da una tupla (S, A, P, R, γ) con:

- S : un insieme finito di stati
- A : il set finito di azioni disponibili ad ogni stato nel sistema
- P : una probabilità di transizione
- R : la funzione di ricompensa
- $\gamma \in [0, 1]$: il fattore di sconto

γ limita la crescita esponenziale di G_t e consente di enfatizzare ricompense immediate piuttosto che ricompense a lungo termine.

16.5.1 Tipi di processi decisionali di Markov

Le tassonomie dei processi decisionali di Markov possono essere differenti. Si possono definire:

- **MDPs osservabili**: quando $s_t = o_t$, ossia lo stato del sistema corrisponde all'osservazione del sistema
- **POMDPs parzialmente osservabili**: quando $s_t \neq o_t$

Un esempio di sistema completamente osservabile è qualsiasi gioco, come gli scacchi. Le pedine sono note così come la griglia: l'osservazione corrisponde allo stato attuale. Un processo parzialmente osservabile è qualsiasi sistema nel mondo reale, come un'auto a guida autonoma o le informazioni cliniche che possono essere fornite ad un medico.

L'environment può inoltre essere:

- **Deterministico:** compiuta un'azione in un determinato stato, l'agente si troverà sicuramente in uno specifico stato futuro
- **Stocastico:** compiuta un'azione in un determinato stato, non è detto che l'agente finisca sempre nello stesso stato futuro con la stessa azione (es. *flip di una moneta*)

I task possono essere:

- **Episodici:** task che possono essere divisi in episodi indipendenti
- **Continui:** task che non terminano naturalmente. Ad esempio, un server rappresenta un task continuo: l'agente compie azioni fin tanto che il sistema non viene spento

16.5.2 Esempi di Processi Decisionali di Markov

Un esempio di processo decisionale di Markov è l'auto a guida autonoma. Gli **stati** possibili sono tutte le combinazioni delle informazioni ricevute dai sensori:

- Camera
- Quantità di carburante
- Velocità
- GPS
- $\vdots$

Gli stati sono quindi tutte le possibili combinazioni delle letture di questi **sensori**. Le **azioni** possibili di un'auto a guida autonoma possono essere:

- Accelerazione
- Decelerazione
- Sterzare verso destra
- Sterzare verso sinistra
- $\vdots$

La **ricompensa** potrebbe essere pari a -1 ad ogni step, così che l'agente sia incoraggiato a raggiungere l'obiettivo nel minor tempo possibile, ma una ricompensa fortemente negativa nel caso di collisioni o azioni illegali, in modo da scoraggiare tali comportamenti.

16.6 Agente

L'agente è colui che compie le azioni, compie il processo di trial and error ed è la componente che deve massimizzare la ricompensa cumulativa nel tempo G_t . Lo status dell'agente altro non è che una funzione della storia del sistema:

$$s_t^a = g(h_t)$$

Un agente può avere una o più rappresentazioni esplicite tra le seguenti:

- Una **policy**
- Una **funzione valore**
- Una **modello del mondo**

16.6.1 Agente: Policy

Una policy è una funzione che mappa lo stato in cui si trova l'agente ad un'azione da compiere. La policy altro non è che la politica, esplicita in ogni stato l'azione da compiere, come un manuale di istruzioni. Una policy può essere:

- **Deterministica:** è una funzione che, dato uno stato, fornisce l'informazione di quale azione compiere
- **Stocastica:** è una distribuzione di azioni possibili rispetto allo stato in cui si trova l'agente

16.6.2 Agente: Funzione Valore

La funzione valore è una funzione che afferma la bontà del comportamento dell'agente. Sono le ricompense future nel trovarmi in un determinato stato e/o compiendo una determinata azione, dando un'indicazione della bontà del comportamento dell'agente, dove il comportamento altro non è che la policy che sta seguendo.

Corrispondente ad una determinata azione π , si indica con:

$$V^\pi$$

Per uno stato $s \in S$, la funzione valore V^π di una determinata azione π è la sommatoria cumulativa scontata attesa ottenuta da un agente a partire da un determinato stato s nel seguire una determinata policy π (equazione di Bellman):

$$V^\pi(s_t = s) = \mathbb{E}_\pi[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots | s_t = s] = \mathbb{E}_\pi[G_t | s_t = s] = \mathbb{E}_\pi \left[\sum_{i=t}^{H-1} \gamma^{i-t} r_i | s_t = s \right] \quad (16.8)$$

- $\mathbb{E}_\pi$: valore atteso calcolato sugli stati incontrati seguendo la politica π

Si può definire in alcuni casi una funzione di qualità $Q^\pi(s, a)$, molto simile all'equazione (16.8) ma che considera in ingresso anche le azioni a_t :

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{i=t}^{H-1} \gamma^{i-t} r_i \mid s_t = s \wedge a_t = a \right] \quad (16.9)$$

L'equazione (16.9) dice qual è il valore atteso della ricompensa cumulativa scontata a partire da un determinato stato s_t , nel compiere una determinata azione a_t e successivamente nel seguire la policy π .

16.6.3 Agente: Modello

Un modello è la rappresentazione di come il mondo cambia in risposta al comportamento dell'agente. Il modello è la somma:

- **Probabilità di transizione tra stati**
- **Funzione di ricompensa**

Un agente di apprendimento per rinforzo può avere una o più rappresentazioni di queste tre entità.

16.7 Dilemma dell'esplorazione e dello sfruttamento

Nell'apprendimento per rinforzo un problema fondamentale è il dilemma dell'*exploration* ed *exploitation*. L'agente, ad ogni timestamp, deve scegliere quale azione intraprendere ed effettua la sua decisione mediante la policy π ottimale. Durante il processo di addestramento, l'agente apprende attraverso un processo di *trial and error*, guidato dalla ricompensa ricevuta dall'environment.

Ad ogni timestamp si deve decidere se:

- **Sfruttare** (*exploitation*) l'azione migliore trovata fino a quel momento
- **Esplorare** (*exploration*) l'ambiente alla ricerca di azioni potenzialmente migliori

Ad ogni istante temporale viene quindi effettuato un compromesso tra l'utilizzo della miglior policy π già nota e l'esplorazione di nuove azioni che potrebbero portare alla scoperta di policy migliori.

16.7.1 Strategie di Sfruttamento (Exploitation)

- **Epsilon-greedy**: ad ogni timestamp, con probabilità ε viene scelta un'azione randomica. Con probabilità $1 - \varepsilon$ viene invece scelta la migliore azione trovata fino a quel momento:

$$A_t \leftarrow \begin{cases} \arg \max_a Q^*(s, a), & 1 - \varepsilon \\ a \sim \text{Uniform}[\{a_1 \dots a_k\}], & \varepsilon \end{cases} \quad (16.10)$$

- **Epsilon-first:** dato un numero totale di N trial, per i primi εN trial viene scelta un'azione randomica, mentre per i successivi $(1 - \varepsilon)N$ trial viene scelta la migliore azione trovata fino a quel momento. Si effettua quindi una divisione tra una fase iniziale di esplorazione e una successiva fase di sfruttamento:

$$A_t \leftarrow \begin{cases} \arg \max_a Q^*(s, a), & (1 - \varepsilon)N \\ a \sim \text{Uniform}[\{a_1 \dots a_k\}], & \varepsilon N \end{cases} \quad (16.11)$$

- **Epsilon-decreasing:** è molto simile all'*epsilon-greedy*, ma il valore di ε decresce nel tempo. In questo modo si mantiene sempre una componente di esplorazione, ma il livello di esplorazione diminuisce progressivamente durante l'addestramento
- **Upper Confidence Bound (UCB):** a differenza delle strategie precedenti, utilizza un approccio dinamico nella scelta tra esplorazione e sfruttamento. Segue il principio dell'*ottimismo di fronte all'incertezza*: se un'azione è poco conosciuta, viene considerata potenzialmente vantaggiosa. L'azione da compiere A_t è:

$$A_t = \arg \max_a \left[\underbrace{Q_t(a)}_{\text{Exploitation}} + c \underbrace{\sqrt{\frac{\log t}{N_t(a)}}}_{\text{Exploration}} \right] \quad (16.12)$$

Si hanno quindi due componenti:

- $Q_t(a)$: valore stimato dell'azione a al timestep t
- $N_t(a)$: numero di volte che l'azione a è stata selezionata prima del tempo t nel processo di addestramento
- c : parametro di confidenza che controlla il livello di esplorazione

L'*Upper Confidence Bound* considera quindi il limite superiore di confidenza dell'azione. Man mano che il numero di osservazioni aumenta e l'incertezza diminuisce, viene effettuato un compromesso dinamico tra esplorazione e sfruttamento.

16.8 Tassonomie degli agenti

Un agente può avere una o più rappresentazioni esplicite di policy, funzione valore e modello. Sulla base della rappresentazione posseduta, un agente può essere classificato come:

- **Value Based**
- **Policy Based**

- **Model Based**



Figura 16.5: Tassonomie degli agenti

Tutti questi approcci possono essere sia **model-based** che **model-free**.

Gli **approcci basati sulla funzione valore** possono essere ulteriormente suddivisi nei seguenti paradigmi:

1. **Programmazione dinamica:** approcci *model-based* che utilizzano una rappresentazione esplicita della dinamica di transizione tra stati e della funzione di ricompensa per ottimizzare il comportamento dell'agente
2. **Approcci Monte Carlo:** approcci *model-free* che non utilizzano una rappresentazione esplicita del modello dell'ambiente
3. **Temporal Difference Learning:** approcci innovativi che rappresentano un compromesso tra programmazione dinamica e Monte Carlo. Sono *model-free* e ottimizzano il comportamento dell'agente mediante interazione diretta con l'environment, senza necessitare di una rappresentazione esplicita del modello
4. **Ricerca esaustiva**

Gli **approcci policy-based** non necessitano di una rappresentazione esplicita della funzione valore, ma ottimizzano direttamente la policy. La policy viene parametrizzata:

$$\pi \rightarrow \pi(\theta)$$

dove θ rappresenta i pesi della policy. L'obiettivo è quindi ottimizzare direttamente tali parametri senza passare attraverso una funzione valore esplicita. Per fare ciò esistono due grandi categorie di approcci:

- **Gradient Based:** utilizzano la discesa del gradiente per ottimizzare i parametri della policy
- **Gradient Free:** approcci di ottimizzazione che non utilizzano il gradiente ma delle ricerche euristiche e risultano particolarmente utili nel calcolo evoluzionistico

16.9 Algoritmi Value-Based

L'obiettivo è trovare la miglior funzione valore, la stima della ricompensa totale futura, in maniera tale da scegliere la migliore azione della funzione valore. Ottimizzata la funzione valore, si sceglie la migliore azione di quella funzione valore.

16.9.1 Value Iteration

Lo scopo del *Value Iteration* è stimare, per ogni stato, il valore atteso scontato ottenibile se l'agente si trovasse in quello stato e seguisse successivamente la politica ottima. Si tratta di un algoritmo **value based** appartenente agli approcci di **programmazione dinamica**.

La funzione valore ottima è associata alla policy ottima ed è definita come la soluzione dell'equazione di Bellman:

$$V^*(s) = \max_a \sum_{s', r} P(s', r | s, a) [r + \gamma V^*(s')] \quad (16.13)$$

dove:

- $V^*(s)$ rappresenta il valore ottimo dello stato s , ovvero il massimo rendimento atteso ottenibile partendo da quello stato
- $P(s', r | s, a)$ rappresenta la probabilità di raggiungere lo stato successivo s' ricevendo la ricompensa r , sapendo che l'agente si trova nello stato s ed esegue l'azione a
- r è la ricompensa immediata ottenuta
- γ è il fattore di sconto, con:

$$0 \leq \gamma \leq 1$$

che determina il peso attribuito alle ricompense future rispetto a quelle immediate

- $V^*(s')$ rappresenta il valore ottimo dello stato successivo s'

L'algoritmo aggiorna iterativamente la funzione valore utilizzando l'equazione di Bellman (16.13). Iterando il procedimento, la funzione valore converge empiricamente alla funzione valore ottima.

L'algoritmo segue i seguenti passi:

1. Per ogni stato $s \in S$ si inizializza la funzione valore $V(s)$
2. Si sceglie una soglia positiva molto piccola θ , utilizzata come criterio di arresto dell'algoritmo iterativo
3. Per ogni iterazione si pone inizialmente:

$$\Delta = 0$$

Successivamente, per ogni stato $s \in S$:

- si salva il valore precedente della funzione valore
 - si aggiorna il valore dello stato utilizzando l'equazione di Bellman (16.13);
 - si aggiorna Δ considerando la massima variazione osservata tra il vecchio e il nuovo valore della funzione valore
4. Il processo iterativo continua fino a quando:

$$\Delta < \theta$$

cioè quando la variazione massima della funzione valore tra due iterazioni successive risulta sufficientemente piccola

5. L'output finale dell'algoritmo è la funzione valore ottima $V^*(s)$. Nota la funzione valore ottima, è possibile ricavare la **policy ottima** π^* come:

$$\pi^*(s) \leftarrow \arg \max_a \sum_{s', r} P(s', r \mid s, a) [r + \gamma V^*(s')] \quad (16.14)$$

Esempio di Value Iteration Consideriamo un esempio in cui apri una start-up. In ogni stato, devi scegliere se risparmiare i soldi guadagnati (**S**, *Save*) oppure spenderli in pubblicità (**A**, *Advertising*). Il problema è modellato come un MDP (Processo di Decisione di Markov) con 4 stati:

- **PU** (Poor & Unknown): Povero e Sconosciuto (Ricompensa: +0)
- **PF** (Poor & Famous): Povero e Famoso (Ricompensa: +0)
- **RU** (Rich & Unknown): Ricco e Sconosciuto (Ricompensa: +10)
- **RF** (Rich & Famous): Ricco e Famoso (Ricompensa: +10)

Immagini/esempiovalueiteration.png

L'obiettivo è trovare la politica ottima π^* dato un **fattore di sconto** $\gamma = 0.9$, utilizzando l'algoritmo di **Value Iteration**:

$$V^{k+1}(s) \leftarrow \max_a \sum_{s',r} P(s', r | s, a) [r + \gamma V^k(s')]$$

Inizializzazione ($K = 0$): Tutti i valori degli stati sono inizializzati a zero:

$$V^0(PU) = 0, \quad V^0(PF) = 0, \quad V^0(RU) = 0, \quad V^0(RF) = 0$$

Iterazione $K = 1$: Calcoliamo il valore per ogni stato applicando l'equazione di Bellman.

Stato PU (Poor & Unknown):

- Azione S (Save): $1 \cdot [0 + 0.9 \cdot V^0(PU)] = 1 \cdot [0 + 0] = 0$
- Azione A (Advertising): $0.5 \cdot [0 + 0.9 \cdot V^0(PU)] + 0.5 \cdot [0 + 0.9 \cdot V^0(PF)] = 0 + 0 = 0$

$$V^1(PU) = \max(0, 0) = 0$$

Stato PF (Poor & Famous):

- Azione S : $0.5 \cdot [0 + 0.9 \cdot V^0(PU)] + 0.5 \cdot [10 + 0.9 \cdot V^0(RU)] = 0.5 \cdot [0] + 0.5 \cdot [10] = 5$
- Azione A : $1 \cdot [0 + 0.9 \cdot V^0(PF)] = 0$

$$V^1(PF) = \max(5, 0) = 5$$

Stato RU (Rich & Unknown):

- Azione S : $0.5 \cdot [10 + 0.9 \cdot V^0(PU)] + 0.5 \cdot [0 + 0.9 \cdot V^0(PF)] = 5 + 0 = 5$
- Azione A : $0.5 \cdot [0 + 0.9 \cdot V^0(PU)] + 0.5 \cdot [10 + 0.9 \cdot V^0(PF)] = 0 + 5 = 5$

$$V^1(RU) = \max(5, 5) = 5$$

Stato RF (Rich & Famous):

- Azione S : $0.5 \cdot [10 + 0.9 \cdot V^0(RU)] + 0.5 \cdot [10 + 0.9 \cdot V^0(RF)] = 5 + 5 = 10$
- Azione A : $1 \cdot [0 + 0.9 \cdot V^0(PF)] = 0$

$$V^1(RF) = \max(10, 0) = 10$$

L'algoritmo procede iterativamente aggiornando i valori. Ecco i risultati per i primi passi:

K	V(PU)	V(PF)	V(RU)	V(RF)
0	0	0	0	0
1	0	5	5	10
2	2.25	9.5	9.5	16.75
3	5.287	13.55	13.55	21.81
4	8.477	17.19	17.19	25.91

Una volta che i valori convergono, possiamo estrarre la politica ottima scegliendo l'azione che massimizza il valore atteso in ogni stato:

$$\pi^*(s) = \arg \max_a \sum_{s', r} P(s', r | s, a) [r + \gamma V(s')]$$

La politica ottima risultante è:

- $\pi^*(PU) = A$
- $\pi^*(PF) = S$
- $\pi^*(RU) = A$
- $\pi^*(RF) = S$

16.10 Algoritmi Policy-Based

16.10.1 Policy Iteration

L'algoritmo di *Policy Iteration* rappresenta un'estensione del *Value Iteration*. A differenza di quest'ultimo, non aggiorna direttamente la funzione valore ottima, ma alterna iterativamente due fasi distinte:

- Una fase di **policy evaluation**
- Una fase di **policy improvement**

Immagini/policyiteration.png

Figura 16.6: Schema del Policy Iteration

Nella fase di **policy evaluation** si assume fissata una policy π e si calcola la corrispondente funzione valore $V^\pi(s)$. Si stima il rendimento atteso ottenibile seguendo quella specifica policy.

Successivamente, nella fase di **policy improvement**, la policy viene migliorata scegliendo, per ogni stato, l'azione che massimizza il rendimento atteso calcolato attraverso la funzione valore appena stimata. La nuova policy ottenuta risulta quindi migliore o uguale rispetto alla precedente.

Le due fasi vengono ripetute iterativamente fino a quando la policy non varia più tra due iterazioni successive. In tale condizione si raggiunge la policy ottima π^* e la relativa funzione valore ottima $V^*(s)$.

Dal punto di vista dello pseudocodice, il *Policy Iteration* risulta più complesso rispetto al *Value Iteration*, poiché richiede l'alternanza delle due procedure di valutazione e miglioramento della policy. Tuttavia, empiricamente si osserva che tende a convergere in un numero minore di iterazioni rispetto al *Value Iteration*.

L'algoritmo segue i seguenti passi:

1. Si inizializzano arbitrariamente:
 - una policy iniziale π
 - una funzione valore $V(s)$
 - una soglia θ utilizzata come criterio di arresto
2. Si esegue una prima fase di **policy evaluation**. In questa fase si valuta la funzione valore associata alla policy corrente π . Il procedimento è analogo al *Value Iteration*, con la differenza che non viene effettuato il:

$$\max_a$$

poiché la policy è fissata e arbitraria. La funzione valore viene quindi aggiornata considerando solamente le azioni definite dalla policy corrente.

3. Terminata la fase di valutazione, si esegue la fase di **policy improvement**. Si inizializza un flag logico, ad esempio:

`policy_changed = true`

che indica se la policy è rimasta invariata oppure no.

4. Per ogni stato $s \in S$:

- si salva la vecchia policy nello stato b :

$$b \leftarrow \pi(s)$$

- si aggiorna la policy scegliendo l'azione che massimizza l'equazione di Bellman:

$$\pi(s) \leftarrow \arg \max_a \sum_{s', r} P(s', r \mid s, a) [r + \gamma V(s')] \quad (16.15)$$

5. Se la nuova policy risulta differente dalla precedente:

$$b \neq \pi(s)$$

allora il flag viene aggiornato:

`policy_stable = false`

6. Se la policy è cambiata almeno in uno stato, l'algoritmo ritorna alla fase di **policy evaluation** utilizzando la nuova policy aggiornata. Se invece la policy non cambia più, allora l'algoritmo è andato a convergenza e la policy ottenuta rappresenta la **policy ottima** π^* .

Esempio di Policy Iteration Riprendiamo l'esempio della start-up. A differenza della Value Iteration, l'algoritmo di **Policy Iteration** prevede di partire con una politica definita a priori e valutarne l'efficacia. Supponiamo di definire una politica iniziale π_0 in cui si sceglie sempre di risparmiare (Azione **S**):

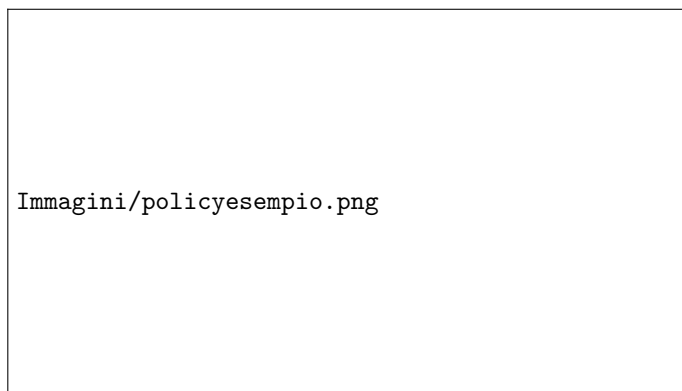
$$\pi_0(PU) = S, \quad \pi_0(PF) = S, \quad \pi_0(RU) = S, \quad \pi_0(RF) = S$$

Per valutare la politica π_0 , applichiamo l'equazione di Bellman. Notiamo che in questo caso **non c'è l'operatore max**, perché l'azione è già stata fissata dalla nostra politica:

$$V^{k+1}(s) \leftarrow \sum_{s',r} P(s',r \mid s, \pi_0(s)) [r + \gamma V^k(s')]$$

Utilizziamo un fattore di sconto $\gamma = 0.9$.

Inizializzazione ($K = 0$): Tutti i valori partono da zero.



Iterazione $K = 1$:

- $\pi_0(PU) = S \rightarrow 1 \cdot [0 + 0.9 \cdot 0] = 0$

- $\pi_0(PF) = S \rightarrow 0.5 \cdot [0 + 0.9 \cdot 0] + 0.5 \cdot [10 + 0.9 \cdot 0] = 5$
- $\pi_0(RU) = S \rightarrow 0.5 \cdot [0 + 0.9 \cdot 0] + 0.5 \cdot [10 + 0.9 \cdot 0] = 5$
- $\pi_0(RF) = S \rightarrow 0.5 \cdot [10 + 0.9 \cdot 0] + 0.5 \cdot [10 + 0.9 \cdot 0] = 10$

Iterazione $K = 2$:

- $\pi_0(PU) = S \rightarrow 1 \cdot [0 + 0.9 \cdot 0] = 0$
- $\pi_0(PF) = S \rightarrow 0.5 \cdot [0 + 0.9 \cdot 0] + 0.5 \cdot [10 + 0.9 \cdot 10] = 9.5$
- $\pi_0(RU) = S \rightarrow 0.5 \cdot [0 + 0.9 \cdot 0] + 0.5 \cdot [10 + 0.9 \cdot 5] = 7.25$
- $\pi_0(RF) = S \rightarrow 0.5 \cdot [10 + 0.9 \cdot 5] + 0.5 \cdot [10 + 0.9 \cdot 10] = 17$

Iterazione $K = 3$: (Usiamo i valori trovati al passo $K = 2$)

- $\pi_0(PU) = S \rightarrow 1 \cdot [0 + 0.9 \cdot 0] = 0$
- $\pi_0(PF) = S \rightarrow 0.5 \cdot [0 + 0.9 \cdot 0] + 0.5 \cdot [10 + 0.9 \cdot 17] = 12.65$
- $\pi_0(RU) = S \rightarrow 0.5 \cdot [0 + 0.9 \cdot 0] + 0.5 \cdot [10 + 0.9 \cdot 7.25] = 8.26$
- $\pi_0(RF) = S \rightarrow 0.5 \cdot [10 + 0.9 \cdot 7.25] + 0.5 \cdot [10 + 0.9 \cdot 17] = 20.91$

Iterazione $K = 4$:

- $\pi_0(PU) = S \rightarrow 1 \cdot [0 + 0.9 \cdot 0] = 0$
- $\pi_0(PF) = S \rightarrow 0.5 \cdot [0 + 0.9 \cdot 0] + 0.5 \cdot [10 + 0.9 \cdot 20.91] = 14.41$
- $\pi_0(RU) = S \rightarrow 0.5 \cdot [0 + 0.9 \cdot 0] + 0.5 \cdot [10 + 0.9 \cdot 8.26] = 8.72$

Tabella Riassuntiva della Policy Evaluation (π_0):

K	$V^k(PU)$	$V^k(PF)$	$V^k(RU)$	$V^k(RF)$
0	0	0	0	0
1	0	5	5	10
2	0	9.5	7.25	17
3	0	12.65	8.26	20.91
4	0	14.41	8.72	...

Una volta valutata π_0 , applichiamo l'operatore $\arg \max_a$ per vedere se esiste un'azione migliore rispetto a quella dettata attualmente da π_0 . L'equazione per estrarre la nuova politica π_1 è:

$$\pi_1(s) = \arg \max_a \sum_{s', r} P(s', r | s, a) [r + \gamma V^{\pi_0}(s')]$$

Verifichiamo ad esempio lo stato **PU** (per il quale π_0 prevedeva passivamente l'azione S):

- $a = S \rightarrow 1 \cdot [0 + 0.9 \cdot 0] = 0$
- $a = A \rightarrow 0.5 \cdot [0 + 0.9 \cdot 0] + 0.5 \cdot [0 + 0.9 \cdot 14.41] = 7.65$

Poiché il valore atteso dell'azione A (7.65) è maggiore di quello dell'azione S (0), aggiorniamo la politica per quello stato: $\pi_1(PU) \rightarrow A$.

Eseguendo lo stesso test sugli altri stati, otteniamo la nuova politica migliorata π_1 :

$$\pi_1(PU) = A, \quad \pi_1(PF) = S, \quad \pi_1(RU) = A, \quad \pi_1(RF) = S$$

Ora che abbiamo una nuova politica π_1 (sicuramente migliore o uguale alla precedente), dobbiamo valutarla ripetendo il processo di *Policy Evaluation* da capo. Azzeriamo i valori ($K = 0$) e applichiamo l'equazione di Bellman lineare, questa volta seguendo le azioni dettate da π_1 .

K	$V^k(PU)$	$V^k(PF)$	$V^k(RU)$	$V^k(RF)$
0	0	0	0	0
1	0	5	5	10
2	4.5	9.5	7.25	16.75
3	6.3	14.56	14.56	20.8
4	9.39	17.19	17.19	25.91

Con i nuovi valori di stato ottenuti valutando π_1 , proviamo a fare un ulteriore passo di *Policy Improvement* per cercare di estrarre una politica ancora migliore, che chiameremo π_2 :

$$\pi_2(s) = \arg \max_a \sum_{s', r} P(s', r | s, a) [r + \gamma V^{\pi_1}(s')]$$

Svolgendo i calcoli per tutti gli stati scopriamo che l'azione che massimizza il rendimento atteso in ogni stato coincide esattamente con l'azione che stavamo già prendendo:

$$\pi_2(PU) = A, \quad \pi_2(PF) = S, \quad \pi_2(RU) = A, \quad \pi_2(RF) = S$$

Poiché la politica non è cambiata ($\pi_2 = \pi_1$), significa che l'algoritmo ha raggiunto la **convergenza**. Possiamo affermare con certezza matematica di aver trovato la soluzione al problema: la politica finale è ottima:

$$\pi_2 = \pi^*$$

16.11 Temporal Difference Learning

Gli algoritmi precedentemente introdotti, come *Value Iteration* e *Policy Iteration*, sono algoritmi **model-based**, poiché richiedono la conoscenza esplicita del modello dell'ambiente. In particolare, utilizzano la matrice delle probabilità

di transizione tra stati e la funzione di ricompensa all'interno dell'equazione di Bellman.

In molti problemi reali, tuttavia, la matrice di transizione P non è nota oppure risulta troppo complessa da stimare. In questi casi si adottano approcci **model-free**, ovvero metodi che apprendono direttamente dall'esperienza senza disporre di un modello esplicito dell'environment.

Gli approcci *model-free* basati sulle **Temporal Difference** (TD) cercano di stimare il ritorno atteso G_t aggiornando progressivamente le stime a partire dalle ricompense osservate.

Supponendo di osservare una sequenza di ricompense:

$$r_1, r_2, \dots, r_k$$

si può stimare il valore medio mediante:

$$Q_k = \frac{r_1 + r_2 + \dots + r_k}{k}$$

Tale relazione può essere riscritta in forma ricorsiva:

$$\begin{aligned} kQ_k &= (r_1 + r_2 + \dots + r_k) \\ &= (r_1 + r_2 + \dots + r_{k-1}) + r_k \\ &= (k-1)Q_{k-1} + r_k \\ &= kQ_{k-1} + (r_k - Q_{k-1}) \end{aligned}$$

Dividendo entrambi i membri per k si ottiene:

$$Q_k = Q_{k-1} + \frac{1}{k}(r_k - Q_{k-1}) \quad (16.16)$$

Definendo:

$$\alpha_k = \frac{1}{k}$$

l'equazione (16.16) può essere interpretata come una **media mobile incrementale**, nella quale la nuova stima viene ottenuta correggendo la stima precedente sulla base dell'errore osservato.

Il termine:

$$\delta_k = r_k - Q_{k-1} \quad (16.17)$$

prende il nome di **Temporal Difference Error** e rappresenta la differenza tra il nuovo valore osservato e la stima precedente.

16.11.1 Q-Learning

Il **Q-Learning** è un algoritmo *value-based* appartenente alla famiglia dei metodi di *Temporal Difference Learning*. Si tratta di un approccio **model-free**, poiché non richiede una conoscenza esplicita della dinamica dell'ambiente né delle probabilità di transizione tra stati.

L'obiettivo dell'algoritmo è apprendere la funzione:

$$Q(s, a)$$

che rappresenta il valore associato all'esecuzione dell'azione a nello stato s .

In particolare, la funzione $Q(s, a)$ stima la ricompensa cumulativa futura massima ottenibile:

$$Q(s, a) = \max_a G_t = \max_a (r_t + \gamma G_{t+1}) \quad (16.18)$$

Poiché il ritorno futuro G_{t+1} non è noto, esso viene approssimato utilizzando il massimo valore stimato tra tutte le azioni future possibili:

$$Q_{\text{target}}(s, a) = r + \gamma \max_{a'} Q(s', a') \quad (16.19)$$

Il valore della funzione Q viene quindi aggiornato tramite la regola di apprendimento:

$$Q_{\text{new}} = Q_{\text{old}} + \alpha(Q_{\text{target}} - Q_{\text{old}})$$

Sostituendo l'equazione (16.19) si ottiene la forma classica dell'aggiornamento del Q-Learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (16.20)$$

In maniera equivalente:

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') \right] \quad (16.21)$$

L'algoritmo Q-Learning dipende principalmente da tre iperparametri fondamentali:

- **α (Learning Rate)**: controlla la velocità con cui le nuove informazioni modificano le stime precedenti della funzione Q . Valori elevati rendono l'apprendimento rapido ma meno stabile, mentre valori bassi producono aggiornamenti più gradualmente.
- **γ (Discount Factor)**: determina l'importanza delle ricompense future rispetto a quelle immediate. Per $\gamma \approx 0$ l'agente privilegia il guadagno immediato; per $\gamma \rightarrow 1$ l'agente assume un comportamento più lungimirante.

- ϵ (**Exploration Factor**): definisce la probabilità con cui l'agente seleziona azioni casuali per esplorare nuove strategie invece di sfruttare esclusivamente la migliore azione conosciuta. Generalmente si utilizza una strategia ϵ -greedy con ϵ inizialmente alto e progressivamente decrescente

L'algoritmo di Q-Learning segue i seguenti passaggi:

1. Si scelgono gli iperparametri:

$$\alpha \in (0, 1), \quad \gamma \in [0, 1], \quad \epsilon > 0$$

2. Si inizializza arbitrariamente la tabella:

$$Q(s, a)$$

per tutte le coppie stato-azione

3. Per ogni episodio si inizializza uno stato iniziale:

$$s_0$$

4. Finché lo stato corrente non è terminale:

- (a) si seleziona un'azione a mediante una policy derivata da Q (tipicamente ϵ -greedy)
- (b) si esegue l'azione osservando:

$$s', \quad r$$

ovvero il nuovo stato e la ricompensa immediata

- (c) si aggiorna la funzione valore tramite:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- (d) si aggiorna lo stato corrente:

$$s \leftarrow s'$$

5. Terminato l'episodio, il processo viene ripetuto fino alla convergenza della funzione Q

Una volta convergente, la policy ottima può essere ottenuta scegliendo, in ogni stato, l'azione con il valore Q massimo:

$$\pi^*(s) = \arg \max_a Q(s, a)$$

Il **Q-Learning** permette quindi all'agente di apprendere direttamente dall'esperienza accumulata durante l'interazione con l'ambiente.

- In ogni episodio, l'agente esplora l'ambiente ricevendo eventuali ricompense secondo la matrice delle ricompense R , fino al raggiungimento dello *goal state*
- Lo scopo dell'addestramento è migliorare progressivamente la conoscenza dell'agente, rappresentata dalla matrice:

$$Q(s, a)$$

- Man mano che la matrice Q viene aggiornata e migliorata, l'agente riduce gradualmente l'esplorazione dell'ambiente diminuendo il parametro ϵ . In questo modo l'agente sfrutta la conoscenza acquisita per individuare il percorso migliore e più rapido verso lo stato obiettivo

Esempio Pratico: Q-Learning in un Grid World Consideriamo un agente che si muove in una piccola griglia composta da 5 stati (da s_0 a s_4). L'obiettivo dell'agente è raggiungere lo **stato terminale** s_4 .

Immagini/griglia1.png

Impostazioni dell'ambiente:

- **Azioni disponibili (A):** Nord (N), Sud (S), Est (E), Ovest (W)
- **Ricompense (R):** Spostamento valido = -10 , Urtare un muro (mossa non valida) = -20 , Raggiungere l'obiettivo s_4 = $+100$
- **Parametri di apprendimento:** $\alpha = 1$, $\gamma = 0.9$.

Nota matematica: Poiché in questo esempio il tasso di apprendimento $\alpha = 1$, la formula di aggiornamento del Q-Learning si semplifica drasticamente. Sostituendo $\alpha = 1$ nella formula originale $Q = Q + \alpha(r + \gamma Q_{max} - Q)$, otteniamo semplicemente:

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a')$$

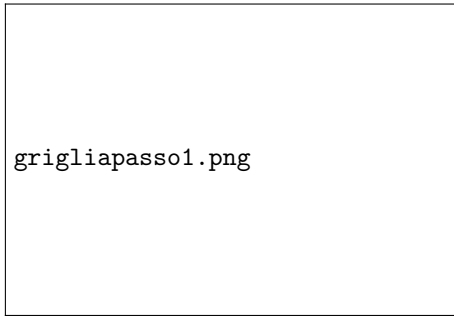
L'agente sovrascrive completamente la vecchia stima con la nuova.

Inizializzazione delle Matrici: La matrice delle Ricompense (R) è fissa e definita dalle regole dell'ambiente. La matrice dei Q-value (Q) viene inizializzata tutta a zero. Entrambe le matrici hanno gli stati sulle righe ($s_0 \dots s_4$) e le azioni sulle colonne (N, E, S, W).

$$R = \begin{pmatrix} -10 & -20 & -20 & -10 \\ -10 & -10 & -20 & -20 \\ -20 & -10 & -10 & -20 \\ -20 & 100 & -10 & -10 \\ 0 & 0 & 0 & 0 \end{pmatrix} \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \end{matrix} \quad Q = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \end{matrix}$$

Tracciamento dei primi passi:

- **Episodio 1 (Partenza da s_3):** L'agente tenta l'azione **N** (Nord). C'è un muro, quindi rimbalza e rimane in s_3 .



- Ricompensa ricevuta: $R(s_3, N) = -20$
- Nuovo stato: s_3
- Massimo Q-value nel nuovo stato: $Q_{max} = \max_a Q(s_3, a) = 0$
- Aggiornamento: $Q(s_3, N) = -20 + 0.9 \cdot (0) = -20$
- Aggiorniamo la matrice Q inserendo -20 nella riga s_3 , colonna **N**

- **Episodio 2 (Nuova partenza da s_1):**



- *Passo 1:* Stato s_1 , Azione **E** (Est) $\rightarrow$ Arriva in s_0 .
 $R(s_1, E) = -10$. Q_{max} in $s_0 = 0$.
 Aggiornamento: $Q(s_1, E) = -10 + 0.9(0) = -10$

- *Passo 2:* Stato s_0 , Azione **E** (Est) $\rightarrow$ Rimbalza sul muro, resta in s_0 .
 $R(s_0, E) = -20$. Q_{max} in $s_0 = 0$.
 Aggiornamento: $Q(s_0, E) = -20 + 0.9(0) = -20$
- *Passo 3:* Stato s_0 , Azione **W** (Ovest) $\rightarrow$ Arriva in s_1 .
 $R(s_0, W) = -10$. Q_{max} in $s_1 = 0$ (poiché il valore appena inserito -10 in s_1 non è il massimo, ci sono ancora gli 0).
 Aggiornamento: $Q(s_0, W) = -10 + 0.9(0) = -10$.

Conclusione (Dopo molte iterazioni): Continuando a esplorare l'ambiente per moltissimi episodi, la matrice Q converge ai valori ottimali.

$$Q_{finale} = \begin{pmatrix} 80 & 52 & 52 & 45.8 \\ 62 & 62 & 35.8 & 35.8 \\ 52 & 80 & 45.8 & 52 \\ 70 & 100 & 62 & 62 \\ 0 & 0 & 0 & 0 \end{pmatrix} \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \end{matrix}$$

La matrice Q ora **codifica il percorso più breve verso l'obiettivo**. Da qualsiasi stato, all'agente basterà guardare la riga corrispondente in Q e scegliere l'azione con il numero più alto. Ad esempio, partendo da s_1 , i valori sono ($N = 62, E = 62, S = 35.8, W = 35.8$). Il massimo è 62 (N o E): muovendosi a Est andrà in s_0 (dove il massimo è 80, N), salendo a Nord andrà in s_3 (dove il massimo è 100, E), e infine arriverà all'obiettivo s_4 .

E' possibile dimostrare che il Q-Learning converge allo stato ottimale $Q^*(s, a)$ se il processo di iterazione itera molte volte. Il numero di volte dipende dalla difficoltà del task in analisi.

Il Q-Learning classico si basa su un approccio **tabellare**: utilizza una matrice (la *Q-Table*) in cui ogni riga rappresenta un possibile stato dell'ambiente e ogni colonna una possibile azione. L'algoritmo popola le celle di questa matrice con i Q-values stimati. L'approccio tabellare funziona in modo eccellente in ambienti discreti e ristretti (come un *Grid World* 5x5), ma fallisce nel mondo reale. Se consideriamo un'auto a guida autonoma con sensori continui, o un videogioco in cui ogni configurazione di pixel è uno "stato", il numero di stati possibili diventa praticamente infinito.

- **Problema di Memoria:** Nessun computer può caricare in RAM una matrice con miliardi di miliardi di righe.
- **Problema di Tempo:** L'agente non avrebbe mai il tempo materiale per esplorare ogni singola combinazione e far convergere i valori della tabella.

Per ovviare a questo problema, il **Deep Q-Learning** abbandona la matrice e la sostituisce con una **Rete Neurale Profonda (DNN)**. Invece di memorizzare un valore per ogni combinazione stato-azione, la rete neurale apprende una funzione parametrizzata dai suoi pesi θ :

$$Q(s, a; \theta) \approx Q^*(s, a)$$

La rete prende in input la rappresentazione dello stato (es. i valori dei sensori) e restituisce in output un vettore contenente i Q-values stimati per tutte le azioni possibili. Il grande vantaggio è la **generalizzazione**: se l'agente incontra uno stato che non ha mai visto prima, ma che somiglia a uno stato noto, la rete neurale sarà in grado di intuire l'azione migliore. Una tabella, invece, non saprebbe cosa fare di fronte a una riga vuota.

16.12 Paradigmi di Apprendimento: Off-policy vs. On-policy

Nel Reinforcement Learning si possono identificare due diversi paradigmi per l'apprendimento dei valori:

- **Off-policy:** I valori di stato-azione vengono appresi in modo *indipendente* dalla politica utilizzata dall'agente per selezionare le azioni. L'agente impara il valore della mossa ottimale anche se nella realtà sta scegliendo mosse casuali per esplorare
- **On-policy:** I valori di stato-azione vengono appresi considerando le azioni *effettivamente eseguite* dall'agente in quel momento, seguendo la sua politica corrente (inclusa la sua esplorazione)

L'algoritmo **Q-Learning** è un classico esempio di apprendimento **off-policy** (infatti usa l'operatore max per aggiornare la stima supponendo di fare la scelta perfetta, ignorando la reale politica ϵ -greedy che sta usando per muoversi).

Il rischio degli algoritmi Off-policy:

Ci sono situazioni in cui ignorare le azioni reali dell'agente durante la fase di esplorazione è pericoloso. In altre parole, **negli algoritmi off-policy, le azioni esplorative potrebbero far incorrere in ricompense negative molto grandi** (penalità severa). Questo accade perché il Q-learning valuta il percorso *ottimo* supponendo di non sbagliare mai. Ad esempio, potrebbe calcolare che il percorso ottimale è camminare esattamente sul bordo di un precipizio. Tuttavia, siccome l'agente sta ancora esplorando (es. con probabilità ϵ fa un passo a caso), fare un passo casuale vicino a un precipizio porta a un disastro. Un algoritmo on-policy, invece, si accorgerebbe del pericolo legato all'esplorazione e imparerebbe a tenersi alla larga dal bordo.

16.12.1 Paradigma On-Policy: Approccio SARSA

SARSA è un algoritmo di apprendimento *Temporal Difference* (TD) **on-policy**. A differenza del Q-Learning (che è *off-policy*), SARSA stima i valori di stato-azione basandosi sulla politica che l'agente sta effettivamente seguendo in quel momento, compresa la sua esplorazione.

- Un'esperienza in SARSA è definita dalla tupla $\langle s, a, r, s', a' \rangle$. Questo significa che l'agente si trovava nello stato s , ha eseguito l'azione a , ha ricevuto

la ricompensa r , è finito nello stato s' e **da lì ha deciso di eseguire l'azione a'**

- Questa intera sequenza fornisce i dati per la nuova esperienza utile ad aggiornare la stima $Q(s, a)$
- Il **Target** (il nuovo valore stimato verso cui far convergere l'aggiornamento) fornito da questa esperienza è:

$$r + \gamma Q(s', a')$$

Mentre il Q-learning assume sempre che al passo successivo si prenderà l'azione ottimale (usando l'operatore $\max$), SARSA usa il valore dell'azione a' che l'agente ha *realmente* scelto di eseguire al passo successivo seguendo la sua politica attuale (che potrebbe essere una scelta sub-ottimale o esplorativa).

Q-learning (Off-policy):

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') \right]$$

SARSA (On-policy):

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha [r + \gamma Q(s', a')]$$